QA Test Guide — Step by Step

Printable walkthrough generated from docs/QA_TEST_PLAN.md. Each case shows its Gherkin scenario and the plain-English steps. Record results on the companion QA Run Log sheet. Generated 2026-10-11.

> End-to-end manual test plan covering the **entire** functionality of the app across all three clients: **Web** (Blazor WASM), **Desktop** (MAUI / Windows), and **Android** (MAUI). Each case is written twice — a **Gherkin** scenario (Given/When/Then, matching this project's convention so it can later seed the automated `E2E.Tests` Playwright suite) and a **plain-English walkthrough** a manual tester can follow step by step.
>
> Brand in examples is "JiggerJot"; substitute your app's brand if rebranded.

How to use this document

- **Run the Smoke suite (§4) first.** It's the ~15-minute critical path. If any smoke case fails, stop and report — deeper suites will likely cascade.
- **Each case has an ID** (e.g. `QA-AUTH-03`). Record the result against the ID using the **sign-off sheet (§16)**: Pass / Fail / Blocked / N-A, plus tester, build/commit, date, notes.
- **Priority:** • Smoke Smoke (critical path) · • Core Core (run every regression) · • Edge Edge (run on full regression or when the area changed).
- **Automated in CI** on a case title means a Playwright journey in `tests/E2E.Tests` now exercises the same path on every push (the `e2e` job in `.github/workflows/ci.yml`). Human QA can **spot-check** these rather than run them in full each cycle; they still need a manual pass on Desktop/Android (the E2E job runs Web only) and whenever the area changes. The marker on each case title is authoritative (≈35 cases); §15 maps the major journeys. See §17 for the run procedure.
- **Both formats describe the same test.** Read whichever suits you; the Gherkin is the source of truth for automation.
- **"App" = whichever client the suite header names.** Most behavior is identical across clients (same API, same shared RCL UI); the per-client suites (§12–§13) cover the auth transport and session persistence **plus the full per-feature native parity added by NATIVE-6** (share-sheet downloads, hardware back, refresh-on-return, theme/locale persistence — QA-DSK-08..15, QA-AND-07..15).

1. Environment & prerequisites

1.1 Backing services + API (host machine)

```
docker compose up -d # Postgres + Mailpit
dotnet run --project src/Api --launch-profile https # binds https:7360 (web/desktop) AND http:5438 (android)
```

- API health/liveness check: `curl -k https://localhost:7360/health` -> **200** (`Healthy`); `curl -k https://localhost:7360/health/ready` -> **200** when the database is reachable (503 if not). *(The older `curl -k -X POST https://localhost:7360/api/auth/refresh` -> **401** reachability check still works.)*
- **Mailpit UI: http://localhost:8027** — this is the dev mail trap. Every magic link, OTP code, and invitation email lands here. Keep it open in a tab throughout testing.

(!) **Email only reaches Mailpit if SMTP points at it.** If your repo-root `.env` has the `Email__Smtp__*` lines set to a real provider (e.g. Brevo), the API sends auth emails there and **Mailpit stays empty** — every email-based case below will appear to "fail." For QA, route mail to Mailpit by **either**:
- commenting out the `Email__Smtp__*` lines in `.env` (unset -> defaults to Mailpit `localhost:1027`), **or**
- leaving `.env` untouched and overriding on the command line (command-line config beats `.env`):

```bash
dotnet run --project src/Api --launch-profile https -- \
--Email:Smtp:Host=localhost --Email:Smtp:Port=1027 --Email:Smtp:Username= --Email:Smtp:Password=
```

Verify by triggering one OTP (QA-SMK-01) and confirming it appears in Mailpit before running the suite.
(!) **Email delivery is now asynchronous.** Requesting a code/link/invite **enqueues** the email and a background dispatcher (the outbox) sends it — so it appears in Mailpit **a few seconds later, not instantly**. **Wait briefly before assuming failure. The send request now** always returns success** (reliability moved to the background): if an email never arrives while SMTP points at Mailpit, the message is retrying or dead-lettered in the `OutboxMessages` table — it is **no longer** surfaced as a request error.

- Web app: `dotnet run --project src/Web --launch-profile https` -> **https://localhost:7208**.

(!) Always use the **https** profile for both web and API. Chrome treats `http://localhost` and `https://localhost` as different sites, so the refresh cookie is dropped over http and sign-in silently fails to persist. (See `docs/DECISIONS.md` / the schemeful-same-site note.)
(!) **The passwordless endpoints are rate-limited** per client IP: the **send** endpoints (`/otp/send`, `/magic-link/send`) default to **5/minute**; the **verify** endpoints (`/otp/verify`, `/mfa/verify`) have their **own, larger budget** (the OTP attempt cap + headroom, default **10/minute**) so the cumulative lockout's distinct message (QA-AUTH-04) surfaces before the throttle can mask it. Exceeding a budget returns **HTTP 429**, shown on `/login` as "Too many requests. Please wait a minute, then try again." — that's the abuse guard working (QA-AUTH-11), **not** a bug. Pace requests, or wait ~1 minute for the window to reset.

1.2 Test accounts & data

Need	What to prepare
Two real Google accounts	e.g. `qa.owner@gmail.com`, `qa.member@gmail.com` — for OAuth + linking + multi-user household flows.
One Microsoft personal account	for the Microsoft OAuth path (provider pinned to the *consumers* tenant).
Throwaway email addresses	any address works for magic-link / OTP — mail is trapped by Mailpit, so the address need not be real. Use distinct ones per test to keep inboxes clean.
Two browser contexts	a normal window **and** an incognito/second-profile window. The household invite flow needs two *different* signed-in users at once; incognito gives you an isolated session + cookie jar.
A brand-new household, unticked	§10h needs a shelf nobody has touched. Use an email address that has never signed in — the platform provisions a fresh tenant on first sign-in (§6). You cannot get back to this state by unticking, because unticking records a decision rather than erasing one (JJ-023).
A screen reader	NVDA (Windows) or VoiceOver (macOS), for QA-SHELF-12 and QA-CHROME-03/05. The shelf's controls are hidden checkboxes styled through their labels, so "does it still announce as a checkbox" is not something you can see.
A Spanish reader	QA-CHROME-09 asks whether Marga's lines read as written Spanish rather than as a translation. That is a judgement, not a check.

Which catalog is loaded matters. The app ships the **curated catalog** (JJ-043): 644 recipes, one per drink, every one with a glass — the IBA list plus the Savoy recipes it does not have. Cases in §10d–§10i name specific drinks; counts are the curated catalog's. No shipped recipe has a single required line, so an empty shelf is one bottle away from nothing (QA-MAKE-10). A database seeded before JJ-043 carries the old 969 or 31 until the `ReseedCuratedCatalog` migration runs — record the catalog size on the sign-off sheet if it is not 644.

> **Database reset between full runs (optional but recommended):** to retest "new user" onboarding cleanly, you need users that don't yet exist. Either use fresh email addresses each run, or reset the dev DB: `docker compose down -v && docker compose up -d`, then re-apply migrations by starting the API. A volume wipe destroys all test data — only do it on the dev environment.

1.3 Desktop (MAUI Windows) — additional setup

- Run the app from Visual Studio (Windows Machine target) or `dotnet build src/Maui -t:Run -f net10.0-windows...`.
- API must be running on `https://localhost:7360` (the desktop client's base URL).
- OAuth uses a **loopback browser flow** — your default system browser will open a tab during OAuth.

1.4 Android (MAUI) — additional setup

Follow `docs/MOBILE_TESTING.md`. The essential bits:

- Emulator (AVD) or USB device running.
- **`adb reverse tcp:5438 tcp:5438` — re-run every time the device/emulator restarts** (it does not persist). Verify with `adb reverse --list`.
- API started with the **https** profile (binds the cleartext `:5438` leg the device uses).
- Provider redirect URIs registered: `http://localhost:5438/signin-google` and `http://localhost:5438/signin-microsoft`.

1.5 Environment B — deployed staging (DEPLOY, ADR-017)

Everything above is **Environment A** (local). The same suite also runs against the **deployed staging** environment — one Render container serving the API + WASM **single-origin** over real HTTPS, backed by Neon Postgres and Brevo email. Point the browser at the staging URL (e.g. `https://<app>-staging.onrender.com`) and execute the cases exactly as written. What differs from local:

- **One origin, real TLS.** Web + API share the host, so there's no separate API port and no CORS step.
- **Email is real (Brevo), not Mailpit.** Use **real or plus-addressed inboxes** you can open (e.g. `you+qa1@gmail.com`); there's no mail-trap UI. If a code/link doesn't arrive, check **Brevo -> Transactional -> Logs** (and that a verified sender + `Email__Smtp__FromAddress` are set). Email cases are therefore slower than local — pace them (the passwordless rate limit still applies).
- **Cold start.** Free instances sleep after ~15 min idle; the **first** request of a session can take ~30–60 s. That's expected, not a failure — retry once it wakes.
- **OAuth** buttons appear **only** for providers configured on staging (Google/Microsoft need their redirect URIs registered against the staging host). If unconfigured, the buttons are simply absent (by design) — record those provider cases **N-A** for this environment.
- **Billing** uses Stripe **test mode** — exercise webhooks with `stripe trigger …` against the staging `/api/billing/webhook`.

Deploy + smoke gate (ADR-031). A merge to `develop` deploys nothing. Staging is deployed from **GitHub** -> Actions -> `CI` -> *Run workflow* on `develop` with `deploy` = staging: the run re-runs every web gate, and only if all pass fires the Render hook, waits for the new build to be live (`/api/version` reports the commit) and runs the automated post-deploy smoke (liveness/readiness, SPA shell + deep-link, `/api` returns an API-shaped 404, `/api/auth/providers`). A red gate or smoke blocks — so a broken deploy is caught before manual QA starts. Manual QA on staging complements it (the human-only paths: real email, OAuth, billing, visual checks).

2. Scope

In scope — the platform: authentication (all methods, all clients), new-user onboarding, household/tenant management, invitations & joining, account settings (linked providers), localization (EN/ES), transactional emails, cross-cutting security (tenant isolation, auth guards, token lifecycle), and **native (MAUI) feature parity** — the shared-RCL feature surface exercised per platform on Windows + Android (§12–13) with a first-run iOS/macCatalyst smoke (§13b) and a per-release native checklist (§13c).

In scope — JiggerJot itself (§10d–§10i). Every flow in `docs/FEATURES.md` §7–§15 is built and covered here: the shelf and custom ingredients (§10d), browsing, filtering, a recipe and the measurement preference (§10e), the makeable

engine, one-ingredient-away and the ranked bottle (§10f), forking and authoring (§10g), the onboarding wizard (§10h), and Marga plus the responsive chrome (§10i). **§10d comes first among these on purpose** — until a household has told the app what it owns, "what can I make right now" has no answer, so a failure there cascades into §10f and §10h.

Out of scope (per `docs/PROJECT_BRIEF.md` OUT list & current state): SMS OTP, OAuth providers beyond Google/Microsoft, FR/DE/PT languages (scaffolded but not translated — see `docs/LOCALIZATION.md`), and the app's own deferred items — publishing household cocktails to a community pool, household-specific substitutions, custom ingredients in the substitution graph, brand/product granularity, recipe-level substitutions, a manual "primary classification", and "running low" inventory. **Editing a cocktail after forking or writing it (`AUTHORING-2`) is not built**, so there is no case for it.

Platform services with no client UI (API-/operational-level, not manually testable through the app yet): the append-only audit log, OpenTelemetry telemetry, the health endpoints, the background outbox/inbox/scheduled-jobs, and **file storage** — the `IFileStorage` seam + the signed download endpoint `GET /api/files/{token}` (anonymous, the token *is* the authorization; local-disk only — cloud backends hand out native presigned URLs). These are **covered by automated tests** (`tests/Api.Tests`); browser E2E for them is deliberately out of scope (headless machinery). The **billing page** is NO longer in this list — BILLING-8 shipped `/billing` (§10c, QA-BILL-01/02) with the fake-provider E2E journey (`BillingJourneyTests`). Health has a smoke check (QA-SMK-07); manual cases for the rest will be added when client UI exists. **GDPR data export** (`POST /api/household/export`) and **account erasure** (`DELETE /api/auth/me`) now **have a web UI** (UI-1: owner Household -> Data, Settings -> Danger zone) — covered by the manual cases QA-HH-13 + QA-SET-07. **RBAC role management now has a web UI** (RBAC-3) — covered by the household cases QA-HH-09..12. **MFA now has a web UI** (UI-2) — enrollment/ disable in Settings and the sign-in step-up on Login — covered by QA-MFA-01..03. The **in-app notification center now has a web UI** (UI-3) — the header bell (list, unread count, mark-read, delete/clear) and Settings delivery-preference switches — covered by QA-NOTIF-01..04. The **platform-staff admin surface now has a web UI** (UI-4) — a staff-only `/admin` console (tenant list/detail + impersonation + targeted/broadcast announcements + plan comp/revert + MFA reset) — covered by QA-ADMIN-01..07. The **public API (PUBAPI)** and **outbound webhooks (HOOKS)** are intentionally UI-less (they're for machines) and **config-gated off** — they have **manual curl/Postman cases in §14b** (QA-API-01..06), in addition to automated tests.

Automated in CI (Web): a Playwright/NUnit E2E suite (`tests/E2E.Tests`, currently **54 journeys**) runs against the real booted stack on every push — the `e2e` job in `.github/workflows/ci.yml`. Every case it covers is marked **Automated in CI** on its title (across auth, MFA, i18n, household/roster, invitations, notifications, admin, billing, theme and GDPR on the platform side, and the shelf, catalog, filters, makeable, one-away, forking, authoring, onboarding and the responsive shell on the app side — the case titles are authoritative; §15 maps the major journeys). **17 of those journeys are the app's own.** Human QA can spot-check those on Web and focus effort on the un-automated cases and the native clients — where CI runs two smoke canaries (Android boots the real app and drives the OTP journey on an emulator; Windows is a boot-to-login probe only).

The app's cases are the least automated part of the plan by design. What a browser journey cannot check is what most of §10d–§10i is for: whether a count is *honest*, whether a suggestion is *useful*, whether her Spanish reads as Spanish. Those need a person.

3. Test data conventions

- **Owner user** = the account you sign in with first; it auto-creates and owns a household.
- **Member user** = a second account invited into the owner's household.
- Household auto-named on creation; rename it to something recognizable (e.g. "QA House") early so you can spot it in the header tenant badge.

4. Smoke suite • Smoke (run first — ~15 min)

The critical path: can a user get in by each method, reach the app, and get out.

QA-SMK-01 — Web: OTP sign-in happy path • Smoke (Web) Automated in CI

Gherkin

```
Given I am an anonymous user on the /login page of the web app
When I enter my email and request a 6-digit code
And I retrieve the code from Mailpit and submit it
Then I am signed in and land on the home page
And the header shows my household name and my display name
```

Walkthrough

1. Open https://localhost:7208 -> you're redirected to `/login`.
2. In the email field enter `qa-smoke@example.com`; click **Email me a 6-digit code**.
3. **Expected:** the form switches to a code-entry view ("Enter the 6-digit code sent to …").
4. Open Mailpit (http://localhost:8027); open the newest mail; copy the 6-digit code.
5. Enter the code; click **Verify code**.
6. **Expected:** you land on the home page; the top header shows a tenant badge (household name) and your display name, plus **Household**, **Settings**, **Sign out** buttons.

QA-SMK-02 — Web: Google OAuth sign-in • Smoke (Web)

Gherkin

```
Given I am on the /login page
When I click "Continue with Google" and complete Google consent
Then I am returned to the app, signed in, on the home page
```

Walkthrough

1. From `/login`, click **Continue with Google**.
2. **Expected:** full-page redirect to Google's consent screen.
3. Choose `qa.owner@gmail.com`, approve.
4. **Expected:** redirected back into the app, signed in, on the home page with the header chrome.

QA-SMK-03 — Web: Sign out • Smoke (Web) Automated in CI

Gherkin

```
Given I am signed in to the web app
When I click "Sign out"
Then I am returned to the /login page
And navigating to /settings redirects me back to /login
```

Walkthrough

1. While signed in, click **Sign out** in the header.
2. **Expected:** you land on `/login`.
3. In the address bar go to `/settings`.
4. **Expected:** you're bounced back to `/login` (no access without a session).

QA-SMK-04 — Web: Session persists across reload ("remember me") • Smoke (Web)

Gherkin

```
Given I am signed in to the web app
When I reload the page (or reopen the tab)
Then I am still signed in without re-authenticating
And a tab left open past the access token's hour stays signed in
And a server that is still starting up does not sign me out
```

Walkthrough

1. Signed in, press F5 / reload.
2. **Expected:** brief load, then the app shell — still signed in, no trip to `/login`. (A silent refresh exchanges the refresh cookie for a new access token on load.)
3. Leave the tab open and untouched for **more than 65 minutes** (the access token lives 60), then navigate to another page. **Expected:** the page loads its data; no trip to `/login`, no error toast. (The session renewed itself a minute before expiry — ADR-002 addendum 2026-09-22.)

4. Stop the API, reload the tab: the loading spinner stays up for about 30 s, then `/login`. Start the API again and reload. **Expected:** signed in without re-authenticating — a server that couldn't be reached never cost the session. *(Native: `SessionKeepAliveTests` pin the same for the stored token.)*

QA-SMK-05 — Desktop: OTP sign-in • Smoke (Desktop) — see QA-DSK-01

QA-SMK-06 — Android: OTP sign-in • Smoke (Android) — see QA-AND-01

QA-SMK-07 — API health & readiness • Smoke (Platform)

Gherkin

```
Given the API is running
When I GET /health and /health/ready
Then /health returns 200 "Healthy"
And /health/ready returns 200 when the database is reachable, 503 when it is not
```

Walkthrough

1. `curl -k https://localhost:7360/health` -> **200**, body `Healthy` (liveness — process up).
2. `curl -k https://localhost:7360/health/ready` -> **200** (readiness — Postgres reachable).
3. *(Optional)* stop the DB (`docker compose stop db`) and re-run step 2 -> **503**; then `docker compose start db` and confirm it returns to **200**.
4. **Expected:** liveness is 200 whenever the process runs; readiness tracks DB reachability. Responses are **status-only** (no connection details leaked).

5. Web — Authentication • Core

QA-AUTH-01 — Magic-link sign-in happy path • Core (Web) Automated in CI

Gherkin

```
Given I am an anonymous user on /login
When I enter my email and request a magic link
And I open the emailed link from Mailpit
Then I am signed in and landed in the app
```

Walkthrough

1. On `/login` enter `qa-magic@example.com`; click **Email me a magic link**.
2. **Expected:** a success panel — "Check your inbox." + "A sign-in link was sent to qa-magic@example.com. It expires in 15 minutes." — with a **Use a different email** link.
3. In Mailpit open the newest mail; click the sign-in link (or copy it into the same browser).
4. **Expected:** the link resolves and you end up signed in, in the app.

QA-AUTH-02 — Microsoft OAuth sign-in • Core (Web)

Gherkin

```
Given I am on /login
When I click "Continue with Microsoft" and complete consent
Then I am returned signed in
```

> **Account resolution (read before testing):** signing in with a provider whose email matches an existing account links the provider to that account rather than creating a duplicate — but only when the email is **verified**. Google asserts this; Microsoft on the **`consumers`** (personal MSA) tenant is trusted to have a verified email even though it omits the claim (`IProviderEmailTrust`). For **work/school** tenants (`organizations`/`common`/a tenant GUID) and any other provider, a same-email auto-link is **refused** (fail-closed takeover guard, audit MITI-3) — the user signs in with their original method and links the provider from **Settings** instead (QA-SET-02). To test the plain first-time path below, use a Microsoft account whose email has **no** prior account.

Walkthrough

1. Click **Continue with Microsoft**; sign in with a **personal** Microsoft account.
2. **Expected:** returned to the app signed in. A brand-new email creates a new account; a personal Microsoft account whose email already exists auto-links to that account (consumers tenant). (If you see a tenant/reply-URL error, the provider registration is the cause — out of app scope, note it.)

QA-AUTH-03 — OTP wrong code is rejected • Core (Web)

Gherkin

```
Given I requested an OTP code for my email
When I submit an incorrect 6-digit code
Then I see an "incorrect code" error and remain on the code-entry screen
```

Walkthrough

1. Request an OTP (as QA-SMK-01) but **do not** use the real code.
2. Enter `000000`; click **Verify code**.
3. **Expected:** inline error ("code is incorrect / verification failed"); you stay on the code-entry view and can retry. You are **not** signed in.

QA-AUTH-04 — OTP cumulative lockout & code expiry • Edge (Web)

Gherkin

```
Given I requested an OTP code
When I submit wrong codes up to the cumulative limit (default 5 failures per email within a 15-min
window)
Then the email is locked — further attempts, INCLUDING a freshly requested code, are rejected until
the window elapses
And separately, an unused code is rejected once it passes its lifespan (default 10 minutes)
```

Walkthrough

1. Request an OTP for an email; enter a **wrong** 6-digit code until you hit the cumulative limit (default 5, `Auth:Otp:MaxAttempts`, counted **per email across the `Auth:Otp:LockoutWindowMinutes` = 15-min window**, not per code).
2. **Expected:** after the limit, a "too many attempts" lockout error.
3. Now request a **fresh** code and submit it — even the **correct** one.
4. **Expected:** still rejected — requesting a new code does **NOT** reset the budget (this is the brute-force defense; a resend can't hand the attacker another N guesses). The lock clears after the 15-min window.
5. **Expiry sub-case:** separately, request a code and wait past `Auth:Otp:CodeLifespanMinutes` (default 10) — the stale code is rejected.

> **Note (enumeration):** OTP verify returns the **same** generic "invalid code" error whether the code was wrong or there is no active code — it never reveals whether an address has an outstanding OTP. *(Long-wait cases — lower the config to test fast. The verify throttle's budget deliberately sits **above** the attempt cap — QA-AUTH-11 — so you reach this lockout and see its distinct message before any 429; only hammering faster than ~10 verifies/minute trips the throttle.)*

QA-AUTH-11 — Passwordless endpoints are rate-limited • Core (Web)

Gherkin

```
Given I rapidly request codes/links for the same client
When I exceed the per-IP send limit (default 5 per minute)
Then further requests are rejected with HTTP 429 until the window resets
```

Walkthrough

1. From `/login`, request an OTP (or magic link) repeatedly in quick succession — more than 5 within a minute.
2. **Expected:** after the limit the request is throttled (**HTTP 429**), surfaced on `/login` as **"Too many requests. Please wait a minute, then try again." OTP verify** has its own, larger per-IP budget — max(send limit, OTP attempt cap + 5) = 10/min by default (so the lockout cap in QA-AUTH-04 is reachable) — with the same 429 + message when exceeded.
3. Wait ~1 minute; requests succeed again.

> Protects against email-bombing and OTP brute-forcing. Expected behavior, not a defect — see the §1.1 pacing note.

QA-AUTH-05 — Magic link is single-use • Edge (Web) Automated in CI

Gherkin

```
Given I signed in by clicking a magic link
When I click the same link a second time
Then it is no longer valid
```

Walkthrough

1. Complete QA-AUTH-01. Then revisit the *same* link from Mailpit.
2. **Expected:** it does not grant a second session — you're sent to `/login` with an invalid-link indication (or simply not signed in). The token is consumed on first use.

QA-AUTH-06 — Magic link expiry • Edge (Web)

Walkthrough: request a link, wait past `Auth:MagicLink:TokenLifespanMinutes` (default 15), then open it. **Expected:** rejected as expired; user lands on `/login`. *(Lower the config to test fast.)*

QA-AUTH-07 — User-enumeration protection • Core (Web)

Gherkin

```
Given an email address that has no account
When I request a magic link or OTP for it
Then the UI shows the same success/"check your inbox" response as for a real account
```

Walkthrough

1. Request a magic link **and** an OTP for a never-before-used address.
2. **Expected:** identical success messaging to a known address — the app never reveals whether an account exists. (Mailpit will still show whatever the system chose to send.)

QA-AUTH-08 — Login error banners • Edge (Web)

Gherkin

```
Given the login page is loaded with an error query parameter
Then a human-readable error banner is shown
```

Walkthrough — visit each URL and confirm a red banner with sensible copy:

- `/login?error=external_failed` -> external sign-in failed message.
- `/login?error=email_unverified` -> email-unverified message.
- `/login?error=invalid_link` -> invalid/expired link message.
- `/login?error=somethingelse` -> generic "something went wrong" fallback.

QA-AUTH-09 — Email-format validation • Edge (Web) Automated in CI

Walkthrough: on `/login`, click a send button with an empty or malformed email (e.g. `abc`). **Expected:** inline "enter a valid email" validation; no request sent.

QA-AUTH-10 — Magic link & OTP available on web; OAuth always • Edge (Web)

Walkthrough: confirm the web login page shows **both** "Email me a magic link" and "Email me a 6-digit code", plus Google/Microsoft buttons. (Native clients hide magic link — covered in §11–12.)

6. Web — Onboarding & new tenant • Core

QA-ONB-01 — First-ever sign-in auto-creates a household, user is owner • Smoke (Web)

Gherkin

```
Given an email/account that has never signed in before
When I authenticate for the first time (any method)
Then a new household is created and I am its owner
And the header shows that household and my name
```

Walkthrough

1. Sign in with a brand-new account/email (fresh address or post-DB-reset).
2. **Expected:** you reach the app immediately (no "create household" prompt — provisioning is automatic). Open **Household**: you are listed with the **Owner** badge and are the only member.

QA-ONB-02 — Returning user keeps their household • Core (Web)

Walkthrough: sign out and back in with the same account. **Expected:** same household, same role, data intact.

7. Web — Household management • Core

> Roles: **owner** > **admin** > **member** (ADR-009). **Owner + admin** can rename and invite/remove members; **owner-only**: change member roles (promote/demote), transfer ownership, dissolve. Members see a read-only name and a **Leave** button. The owner's row never shows action buttons.

QA-HH-01 — Owner renames the household • Core (Web)

Gherkin

```
Given I am the household owner on /household
When I change the name and click Rename
Then the new name is saved and reflected in the header tenant badge
```

Walkthrough

1. **Household** -> edit the name field (e.g. "QA House") -> **Rename**.
2. **Expected:** success banner; the header tenant badge updates to the new name (a silent refresh updates the claim).

QA-HH-02 — Member sees read-only household, no owner controls • Core (Web) Automated in CI

Precondition: signed in as a *member* (use QA-INV flow to create one). **Walkthrough:** open **Household**. **Expected:** household name shown as read-only heading; no rename/invite/transfer controls; a **Leave** button is present.

QA-HH-03 — Owner removes a member • Core (Web) Automated in CI

Gherkin

```
Given I am the owner and another member exists
When I click Remove next to that member and confirm
Then they are removed from the household member list
```

Walkthrough

1. As owner with a member present, click **Remove** by the member's row.
2. **Expected:** a confirm dialog ("Remove <name>?"); on confirm, success banner and the member disappears from the list. (The removed user is re-homed to a fresh solo household — verify by signing in as them: they now own an empty household — see QA-HH-08.)

QA-HH-04 — Owner cannot remove themselves • Edge (Web)

Walkthrough: as owner, confirm there is **no Remove button on your own row**. (Owner departs via transfer or dissolve, not removal.)

QA-HH-05 — Transfer ownership • Core (Web) Automated in CI

Gherkin

```
Given I am the owner and at least one other member exists
When I select that member and click Transfer
Then they become owner and I become a regular member
```

Walkthrough

1. As owner with >=1 other member, in **Transfer ownership** pick the member -> **Transfer**.
2. **Expected:** success banner. The chosen member now shows the **Owner** badge; your row shows **Member**. The owner-only controls (invite/transfer/dissolve) are no longer available to you, and a **Leave** button now is.

QA-HH-06 — Owner with other members cannot directly leave/dissolve • Edge (Web)

Walkthrough: as owner **with** other members present, confirm the bottom card shows the **Transfer ownership** control (with "you must transfer before leaving" note) — **not** a leave/dissolve button. The single-owner invariant is enforced in the UI.

QA-HH-07 — Sole owner dissolves the household • Core (Web) Automated in CI

Gherkin

```
Given I am the only member and owner of my household
When I click "Leave and delete" and confirm
Then the household is dissolved and I am re-homed to a fresh solo household
```

Walkthrough

1. As sole owner (no other members), bottom card ("Leave & dissolve") -> **Leave and delete**.
2. **Expected:** a confirm dialog warning the household will be deleted; on confirm, the app reloads and you land signed in with a **new empty household you own** (you're never left tenant-less).

QA-HH-08 — Member leaves the household • Core (Web) Automated in CI

Gherkin

```
Given I am a non-owner member
When I click Leave and confirm
Then I leave the household and am re-homed to a fresh solo household I own
```

Walkthrough

1. As a member, **Household** -> **Leave** -> confirm.
2. **Expected:** app reloads; you now own a brand-new empty household. The household you left still exists for its remaining members (verify as the owner: the leaver is gone from the member list).

QA-HH-09 — Owner promotes a member to admin • Core (Web) Automated in CI

Precondition: signed in as the owner with at least one **member** present (use the QA-INV flow). **Gherkin**

```
Given I am the owner and another member exists
When I click "Make admin" next to that member
Then their badge changes to Admin
```

Walkthrough

1. On **Household**, find a member's row -> click **Make admin**.
2. **Expected:** success banner ("Role updated."); the member's badge flips from **Member** to **Admin**, and the button becomes **Make member**. (Verify as that user — see QA-HH-11.)

QA-HH-10 — Owner demotes an admin to member • Core (Web) Automated in CI

Gherkin

```
Given I am the owner and an admin exists
When I click "Make member" next to that admin
Then their badge changes back to Member
```

Walkthrough

1. On **Household**, on an **Admin** row -> click **Make member**.

2. **Expected:** success banner; the badge returns to **Member** and the button becomes **Make admin**.

QA-HH-11 — Admin sees management controls but not role/ownership controls • Core (Web) Automated in CI

Precondition: signed in **as the admin** promoted in QA-HH-09. **Gherkin**

```
Given I am an admin (not the owner)
When I open Household
Then I can rename the household and invite/remove members
But I see no promote/demote (role) controls and no transfer/dissolve — only Leave
```

Walkthrough

1. As the admin, open **Household**.
2. **Expected:** the **rename** field and the **Invitations** card are available; member rows show a **Remove** button **but no Make admin/Make member** buttons (role changes are owner-only). The bottom card shows **Leave** (no Transfer ownership / Leave & delete). The owner's row shows **no** action buttons.

QA-HH-12 — Member sees no management controls • Edge (Web) Automated in CI

Walkthrough: signed in as a plain **member**, open **Household**. **Expected:** read-only name, no Invitations card, no per-row action buttons (no Remove/role controls), only a **Leave** button — unchanged from QA-HH-02 (a member is never shown management controls regardless of the admin tier).

QA-HH-13 — Owner exports household data • Core (Web)

Gherkin

```
Given I am the household owner on /household
When I click "Download household data"
Then I get a link to a JSON export of the household's data
```

Walkthrough

1. As owner, open **Household** -> the **Data** card -> **Download household data**.
2. **Expected:** a success row with a **Download** link; following it downloads a JSON bundle containing the tenant, members and invitations (and each feature's data). Members/admins don't see the Data card (owner-only). No secrets (invitation token hashes) appear in the file.

QA-HH-14 — Seat quota blocks inviting past the plan limit • Core (Web) Automated in CI

Precondition: the platform ships example seat caps (Free = 3 seats, counting members + pending **unexpired** invites — a lapsed invite reserves nothing, v4 T34; `PlanCatalog`). A Free household at its cap (e.g. 3 members, or 2 members + 1 pending invite). **Gherkin**

```
Given my Free plan's seats are all used (members + pending invites)
When I invite another member
Then I'm told my seat limit is reached and to upgrade (nothing is invited)
And raising the limit (Pro plan / editing PlanCatalog) lets the invite through
```

Walkthrough

1. As owner of a Free household at the seat cap, **Household** -> invite a new email.
2. **Expected:** an error — "Your plan's seat limit is reached. Upgrade your plan to invite more members." (HTTP **402**); no invitation is created and no email is sent. **With billing gated off** (`Billing__Enabled` unset — GATES-1) the same 402 reads "Your household is full. Remove a member or revoke a pending invitation to free a seat." instead, since there is no plan to upgrade to.
3. Free up a seat (revoke a pending invite / remove a member) **or** move to a higher-seat plan — the next invite succeeds. (Seats count members **plus** pending invites, so invites can't over-provision.)
4. **Note:** limits are `PlanCatalog` data; `null`/absent = unlimited. Metered-usage caps (`IQuotaService.TryConsumeAsync`, monthly) are wired the same way where an app calls them.

8. Web — Invitations & joining • Core

QA-INV-01 — Owner invites by email; token revealed + email sent • Smoke (Web) Automated in CI

Gherkin

```
Given I am the household owner
When I invite an email address
Then an invitation appears in the pending list with an expiry date
And a one-time join token is revealed in the UI
And an invitation email with a /join link is delivered to Mailpit
```

Walkthrough

1. **Household** -> Invitations -> enter `qa.member@gmail.com` -> **Invite**.
2. **Expected:** a green panel reveals the raw token + a **Copy** button and notes a join link was emailed; the address shows under **Pending** with an expiry date.
3. Check Mailpit: an invitation email addressed to that user, containing a `/join?token=…` link.

QA-INV-02 — Invitee accepts and joins (two-user flow) • Smoke (Web) Automated in CI

Gherkin

```
Given an invitation exists for a second user
When that user opens the /join link, signs in, and accepts
Then they become a member of the inviter's household
```

Walkthrough

1. Copy the `/join?token=…` link from QA-INV-01.
2. In an **incognito window**, open the link.
3. **Expected:** "You've been invited… sign in to accept." Click **Sign in to accept**; complete any sign-in method as `qa.member@gmail.com`.
4. **Expected:** after sign-in you're returned to the join flow automatically, it processes, and you see **"You're in"** with a **Go to household** button.
5. Click it -> **Household** shows you as a **Member** of "QA House".
6. Back in the owner window, reload **Household**: the new member appears and the pending invite is gone.

QA-INV-03 — Already-authenticated user accepts directly • Core (Web)

Walkthrough: while already signed in as a *different* fresh user, open a valid `/join?token=…`. **Expected:** it accepts immediately (no sign-in step) and shows success. *(Note: a user already in a household who accepts another invite moves to the new household — verify their old membership is replaced, honoring the one-tenant invariant.)*

QA-INV-04 — Join page with no token offers manual code entry • Edge (Web)

Walkthrough: open `/join` with no `?token=`. **Expected:** the **manual invite-code entry** state ("Join with an invite code" + input + submit — the NATIVE-4b path for emailed codes), no crash. Pasting a garbage code shows the inline error and lets you retry (automated: `MembershipLifecycleTests.Pasting_An_Invalid_Code_Shows_An_Inline_Error`).

QA-INV-05 — Join with invalid/expired/used token • Core (Web)

Gherkin

```
Given an invitation token that is invalid, already used, or expired
When an authenticated user opens its /join link
Then they see an error state, not a successful join
```

Walkthrough: while signed in, open `/join?token=garbage123` (or reuse a token already accepted in QA-INV-02). **Expected:** the error state with a **Back to household** link; no membership change.

QA-INV-06 — Invite an existing member is rejected • Core (Web)

Gherkin

```
Given a user is already a member of my household
When I invite their email again
Then I get an "already a member" error and no duplicate invite is created
```

Walkthrough: as owner, invite the email of someone already in the household. **Expected:** a red "already a member" status (HTTP 409); nothing added to pending.

QA-INV-07 — Regenerate a pending invitation • Core (Web)

Gherkin

```
Given a pending invitation exists
When I click Regenerate
Then a new token is issued (revealed) and the old token no longer works
```

Walkthrough

1. On a pending invite, click **Regenerate**.
2. **Expected:** a new token is revealed. The **previous** token's `/join` link now fails (QA-INV-05), the new one works.

QA-INV-08 — Revoke a pending invitation • Core (Web) Automated in CI

Gherkin

```
Given a pending invitation exists
When I click Revoke
Then it is removed from pending and its token no longer works
```

Walkthrough: click **Revoke** on a pending invite. **Expected:** "invitation revoked" status; it leaves the pending list; opening its old link gives the error state.

QA-INV-09 — Copy token button • Edge (Web)

Walkthrough: click **Copy** on a revealed token; paste elsewhere. **Expected:** the token is on the clipboard. (If the browser blocks clipboard access, the token is still visible to copy manually — no error shown.)

QA-INV-10 — Accepting an invite after a downgrade is refused (seat re-check) • Core (Web) Automated in CI

Precondition: a way to change the tenant's plan — staff comp/revert (QA-ADMIN-06) or the fake provider webhook (E2E does the latter). **Gherkin**

```
Given a household on Pro invited more members than the Free plan allows
And the subscription has since lapsed or been reverted to Free
When an invitee opens a still-valid invitation link
Then joining is refused with a "This household is full" message
And the invitation stays pending (it works again if the owner re-upgrades)
```

Walkthrough

1. Comp the household to Pro (QA-ADMIN-06); as the owner, invite members until members + pending exceeds the Free limit (currently 3).
2. Revert the household to Free.
3. Sign in as an invitee (fresh browser) and open the invite link / paste the code on `/join`.
4. **Expected:** the join page shows **"This household is full"** (402 `seat_limit_reached`) — the invitee does not join and no membership changes. Existing members are untouched; new invites are also blocked (QA-HH-14).
5. Comp back to Pro and retry the same link. **Expected:** it joins — the refused token self-heals (BILLING-9, ADR-006 addendum).

9. Web — Settings / linked accounts • Core

QA-SET-01 — View linked providers • Core (Web)

Gherkin

```
Given I signed up with Google
When I open /settings
Then Google shows as "Connected" and Microsoft shows a "Link" button
```

Walkthrough: sign in with Google, open **Settings**. **Expected:** a row per provider; the one you used shows a **Connected** badge + **Unlink**; the other shows **Link**.

QA-SET-02 — Link a second provider • Core (Web)

Gherkin

```
Given I am signed in and Microsoft is not linked
When I click Link on Microsoft and complete consent
Then Microsoft becomes Connected on my account (no new account is created)
```

Walkthrough

1. **Settings** -> **Link** on Microsoft -> complete consent with a Microsoft account **whose email isn't already used by another app user**.
2. **Expected:** returned to Settings with a success banner ("Microsoft linked"); Microsoft now shows **Connected**. You can subsequently sign in with either provider into the *same* account.

QA-SET-03 — Linking a provider already used by another account is rejected • Core (Web)

Gherkin

```
Given a provider identity is already linked to a different user
When I try to link it to my account
Then I get an "already in use" error and it is not linked
```

Walkthrough: try to **Link** a Google/Microsoft identity that another app user already owns. **Expected:** Settings shows an "already in use" banner (`?link_error=in_use`); no link made. *(Expired link-token path shows `?link_error=expired` — exercise if you can stall the flow past the token lifetime.)*

QA-SET-04 — Unlink a provider • Core (Web)

Gherkin

```
Given two providers are linked to my account
When I click Unlink on one and confirm the dialog
Then it returns to a "Link" state
```

Walkthrough: with two providers connected, click **Unlink** on one. **Expected:** a confirm dialog ("Unlink this sign-in method?"); on **confirm** the row reverts to **Link**. **Cancelling** the dialog leaves it **Connected** — no change. *(The confirm fails closed: if the browser dialog can't run, the unlink is cancelled, never silently performed. The same guarded confirm protects remove-member / leave / dissolve in §7.)*

QA-SET-05 — Unlinking your only provider never locks you out • Edge (Web)

Gherkin

```
Given Google is my only linked provider
When I unlink it
Then I can still sign in by email (magic link / OTP)
```

Walkthrough: unlink your sole provider, sign out, and sign back in with **OTP/magic link** using the same email. **Expected:** you reach the **same** account/household. (Email sign-in is always available by design, so provider removal can't lock you out.)

QA-SET-06 — Settings requires auth • Edge (Web)

Walkthrough: signed out, navigate to `/settings`. **Expected:** redirect to `/login`.

QA-SET-07 — Delete my account • Core (Web) Automated in CI

Precondition: use a **throwaway** account (this is destructive). Easiest: a member of another owner's household (so no dissolve). **Gherkin**

```
Given I am signed in on /settings
When I use the Danger zone "Delete my account" and confirm
Then my account and personal data are deleted and I'm signed out
```

Walkthrough

1. **Settings** -> **Danger zone** -> **Delete my account** -> confirm the dialog.
2. **Expected (member):** account deleted; you're signed out and land on `/login`. Signing in again creates a brand-new account.
3. **Owner with other members:** an error tells you to **transfer ownership first** (nothing deleted).
4. **Sole owner:** a second confirm warns it also **dissolves the household**; on confirm, the account + household are deleted.

QA-SET-08 — Theme: dark mode applies, persists, and follows the user • Core (Web) Automated in CI

Gherkin

```
Given I am signed in
When I pick "Dark" in the header theme switcher
Then the app restyles dark immediately without a reload
And a reload first-paints dark (no light flash)
And signing in on a fresh browser profile renders dark as part of the sign-in
```

Walkthrough

1. Signed in, pick **Dark** in the header switcher (login page and **Settings -> Preferences** have the same control). **Expected:** the page flips dark instantly — background, cards, buttons, and the brand lockup swaps to its dark variant; no reload.
2. Reload. **Expected:** the very first paint is dark — no white flash.
3. Sign in as the same user in a fresh browser profile (or private window). **Expected:** dark applies as part of signing in, no reload needed — the choice is stored per user server-side (PREFS-1, ADR-022).
4. Pick **Auto**. **Expected:** the app follows the OS scheme live (flip the OS setting to check). Auto is stored per user too: a second browser showing Dark returns to the OS scheme on its next sign-in/reload.

QA-MFA-01 — Enable two-factor (authenticator TOTP) • Core (Web) Automated in CI

Precondition: signed in; an authenticator app (Google Authenticator, 1Password, Authy, …) to hand. **Gherkin**

```
Given I am signed in on /settings with two-factor Off
When I enable it, scan the QR (or enter the key) and confirm with a 6-digit code
Then two-factor turns On and I'm shown one-time recovery codes
```

Walkthrough

1. **Settings** -> **Two-factor authentication** shows an **Off** badge -> **Enable two-factor**.
2. A **QR code** renders next to a **manual key** (Base32). Scan it (or type the key) into the app.
3. Enter the app's current **6-digit code** -> **Verify & enable**.
4. **Expected:** a success banner, the badge flips to **On**, and a grid of 10 **recovery codes** appears (shown once) — short, typeable `xxxxx-xxxxx` codes (e.g. `k7m2q-9xr4t`; no ambiguous `0/O/1/I/L` glyphs). **I've saved my codes** returns to the On state.
5. **Wrong code:** an inline error ("that code is incorrect or has expired"); nothing changes.

QA-MFA-02 — Two-factor is required at sign-in • Core (Web) Automated in CI

Precondition: an account with two-factor **On** (QA-MFA-01). **Gherkin**

```
Given my account has two-factor enabled
When I sign in with an email OTP
Then I'm asked for an authenticator code before the session starts
And entering a valid code completes sign-in
```

Walkthrough

1. Sign out. On `/login`, request an **email code**, enter it.

2. **Expected:** instead of landing signed-in, a **second prompt** asks for the authenticator code.
3. Enter the current 6-digit code -> you're signed in (lands on `/`).
4. **Wrong/expired code:** an inline error; you stay on the step-up prompt (no session).
5. **Note:** OAuth and magic-link sign-ins enforce the step-up too (QA-MFA-04); native (MAUI) too (QA-MFA-05).

QA-MFA-03 — Use a recovery code, then disable two-factor • Core (Web)

Gherkin

```
Given my account has two-factor enabled
When I sign in and enter a recovery code at the step-up
Then sign-in completes (that code is now spent)
And I can disable two-factor from Settings with a valid code
```

Walkthrough

1. Sign in as in QA-MFA-02; at the step-up, enter one **recovery code** instead of a TOTP -> signs in. Entry is forgiving: wrong case, a dropped hyphen, or stray spaces still match (`K7M2Q9XR4T` ≡ `k7m2q-9xr4t`).
2. **Settings** -> **Two-factor authentication** (On) -> **Disable two-factor** -> enter a current TOTP (or another recovery code) -> **Disable**.
3. **Expected:** the badge flips to **Off**; a subsequent sign-in no longer asks for a second step.

QA-MFA-04 — Redirect logins (OAuth / magic link) enforce the step-up • Core (Web)

Precondition: an account with two-factor **On**, reachable via an OAuth provider and/or magic link. **Gherkin**

```
Given my account has two-factor enabled
When I sign in with Google/Microsoft or a magic link
Then I am redirected to the authenticator step-up before any session is issued
And entering a valid code completes sign-in
```

Walkthrough

1. Sign out. Sign in via **Google/Microsoft** (or click a **magic link**).
2. **Expected:** instead of landing signed-in, you arrive on `/login` showing the **authenticator code prompt** (the URL carries a one-time `?mfa=` challenge). No session exists yet.
3. Enter the current 6-digit code (or a recovery code) -> you're signed in (`/auth-callback` -> `/`).
4. **Security check:** confirm you are **not** signed in until the code is accepted — a wrong/expired code keeps you on the prompt with no session. (This closes the gap where redirect logins skipped MFA.)

QA-MFA-05 — Native (MAUI) sign-in enforces the step-up • Core (Desktop/Android)

Precondition: an account with two-factor **On**; run the desktop/Android shell (see `docs/MOBILE_TESTING.md`). Native has no magic link — use **email OTP** or **OAuth**. **Gherkin**

```
Given my account has two-factor enabled
When I sign in on the native app with an email code or OAuth
Then the app shows the authenticator step-up in-app before completing sign-in
And a valid code (or recovery code) finishes sign-in
```

Walkthrough

1. In the native app, sign in with an **email code** (or a provider).
2. **Expected:** the app stays on the login screen and shows the **authenticator code prompt** (it does not sign in yet). No session/token is stored.
3. Enter the current 6-digit code (or a recovery code) -> you're signed in (lands on `/`).
4. **Wrong/expired code:** inline error; you remain on the prompt (no session). Tokens arrive in the response body (native transport), same as a normal native login.

QA-NOTIF-01 — Notification bell + list • Core (Web) Automated in CI

Note: the platform has no built-in producer; to see items, a feature must call `INotificationService.NotifyAsync` (seed one in dev, or exercise a downstream feature that notifies). **Gherkin**

```
Given I am signed in
When I open the notification bell in the header
Then I see my notifications newest-first, with unread ones marked
And the bell shows an unread count when I have unread notifications
```

Walkthrough

1. In the header, click the **bell**. **Expected:** a dropdown opens; with none, it reads "You're all caught up."
2. With unread notifications present: a red **count badge** shows on the bell; unread rows carry a dot + bold title; each shows a relative time ("just now", "5m", "3h", "2d").
3. Click the backdrop (outside the panel) -> it closes.

QA-NOTIF-02 — Mark read / mark all read • Core (Web) Automated in CI

Precondition: at least one unread notification (see QA-NOTIF-01 note). **Gherkin**

```
Given I have unread notifications
When I click one (and, separately, "Mark all read")
Then that one clears its unread mark and the count drops
And "Mark all read" zeroes the count
```

Walkthrough

1. Open the bell -> click an **unread** row. **Expected:** its dot/bold clears; the count decrements.
2. Click **Mark all read**. **Expected:** the count badge disappears; all rows show as read.
3. Reload the page -> the counts/read state persist (server-side).

QA-NOTIF-04 — Delete a notification / clear read / clear all • Edge (Web)

Precondition: a mix of read and unread notifications (see QA-NOTIF-01 note). **Gherkin**

```
Given I have read and unread notifications
When I click a row's trash icon (and, separately, "Clear read" / "Clear all")
Then that row is removed (unread ones also drop the count)
And "Clear read" removes only the read rows; "Clear all" empties the list
```

Walkthrough

1. Open the bell -> click the **trash icon** on the left of a row. **Expected:** the row disappears (without marking anything read — the trash doesn't trigger the row click); if it was unread the count decrements.
2. Footer -> **Clear read**. **Expected:** only the already-read rows vanish; unread ones (and the badge) survive. The button is disabled when nothing is read.
3. Footer -> **Clear all**. **Expected:** the list empties ("You're all caught up."), badge gone.
4. Reload -> deletions persist (server-side; `DELETE /api/notifications/{id}`; bulk requires an explicit scope — `?read=true` clears read, `?read=false` clears all, omitted -> 400 `scope_required`).

QA-NOTIF-03 — Delivery preferences • Core (Web) Automated in CI

Gherkin

```
Given I am signed in on /settings
When I toggle the In-app or Email notification switches
Then the choice is saved and survives a reload
```

Walkthrough

1. **Settings** -> **Notifications** card shows two switches (**In-app**, **Email**), both on by default.
2. Toggle one off. **Expected:** it saves immediately (optimistic; reverts with an error if it fails).
3. Reload -> the switch keeps its new state. (Email-off suppresses the email channel on future notifies; in-app-off suppresses the in-app row.)

10. Web — Localization (i18n) • Core

QA-I18N-01 — Switch language on the login page • Core (Web) Automated in CI

Gherkin

```
Given I am on /login in English
When I choose Español in the language switcher
Then the page text renders in Spanish
```

Walkthrough: on `/login`, use the language switcher (bottom of the card) -> **Español**. **Expected:** titles, button labels, and prompts switch to Spanish; the choice persists on reload.

QA-I18N-02 — Language persists per user across sessions • Core (Web) Automated in CI

Gherkin

```
Given I am signed in and set my language to Spanish
When I sign out and sign back in (even in a fresh browser)
Then the app loads in Spanish
```

Walkthrough

1. Signed in, switch to **Español** in **Settings -> Preferences**. (This saves the locale to your user record via `PUT /api/auth/locale` and into the JWT. A pre-auth pick on the login page also works: it's adopted into your user record when you sign in — PREFS-1, ADR-022.)
2. Sign out; sign back in (a fresh browser/private window is the stronger check).
3. **Expected:** app comes up in Spanish — the preference followed the *user*, not just the browser. On a mismatch the app persists the saved locale and reloads once.

QA-I18N-03 — In-app UI is fully translated (no English leaks) • Edge (Web)

Walkthrough: in Spanish, walk Home -> Household -> Settings -> invite flow. **Expected:** all visible labels/buttons/validation/status messages are Spanish; flag any English string that leaks.

QA-I18N-04 — Email language matches the requester's UI language • Core (Web)

Gherkin

```
Given my UI language is Spanish
When I trigger an OTP / magic link / invitation email
Then the email arrives in Spanish
```

Walkthrough

1. Set UI to **Español**. Trigger an OTP (and a magic link, and send an invite).
2. In Mailpit, open each. **Expected:** subject + body in Spanish. Repeat in English to confirm both.

10b. Web — Admin console (platform staff) • Core

Precondition: your account's email must be in the staff allowlist — set `Admin__StaffEmails__0` in the repo-root `.env` (see `.env.example`) and restart the API. Non-staff accounts must **not** see any of this.

QA-ADMIN-01 — Staff sees the console; non-staff don't • Core (Web) Automated in CI

Gherkin

```
Given I am signed in as a platform-staff user
When I look at the header
Then I see an "Admin" link, and /admin lists every tenant with member counts
And a non-staff user sees no Admin link and is refused at /admin
```

Walkthrough

1. Signed in as **staff**: an **Admin** button shows in the header -> open it (or go to `/admin`).
2. **Expected:** the **Tenants** list shows every tenant (name + member count). Click one -> detail panel shows members (name/email + role), subscription status, created date, audit-event count.
3. Sign in as a **non-staff** user: **no Admin link**; navigating directly to `/admin` shows "You don't have access to the admin console."

> Automated: staff console + tenant list (announcement journey) and the non-staff `/admin` refusal.
> The header **Admin-link visibility** checks (steps 1 and 3's "no Admin link") remain manual.

QA-ADMIN-02 — View a tenant is audited in that tenant • Core (Web)

Gherkin

```
Given I am staff viewing a tenant's detail in /admin
When the detail loads
Then an audit event (admin.tenant.viewed) is recorded in that tenant
```

Walkthrough

1. As staff, open a tenant's detail in `/admin`.
2. **Expected:** an `admin.tenant.viewed` event is written **in that tenant** (visible to that tenant's own audit trail) — the global tenant filter is never loosened; the read enters the target tenant.

QA-ADMIN-03 — Impersonate a user, then stop • Core (Web)

Gherkin

```
Given I am staff on a tenant's detail
When I "Sign in as" a member and confirm
Then I browse the app as that user with a persistent impersonation banner
And "Stop impersonating" returns me to my own staff identity
```

Walkthrough

1. In a tenant detail, click **Sign in as** on a member -> confirm the dialog.
2. **Expected:** you land on `/` **as that user** (their name/household in the header); a yellow **impersonation banner** is pinned at the top; the **Admin** link is hidden while impersonating.
3. Click **Stop impersonating**. **Expected:** you're back as yourself (staff); the banner is gone.
4. The impersonation token is **short-lived (15 min) and non-refreshable** — a full page reload also returns you to your own identity. When it **expires** (leave the impersonated session open 15 min), the app reloads to `/` as yourself: no call from the target's page is ever made as staff, and the client never renews the impersonation (v4 T31, `SessionKeepAliveTests.ImpersonationExpiry_*`). Impersonation is **audited** in the target's tenant.

QA-ADMIN-04 — Staff announcement reaches a tenant's members • Edge (Web) Automated in CI

Gherkin

```
Given I am staff on a tenant's detail in /admin
When I send an announcement (title + body) and confirm
Then every member of that tenant is notified through their preferred channels
And each member's bell shows the announcement; reading it clears the badge
And an admin.announcement.sent audit event is recorded in that tenant
```

Walkthrough

1. As staff, open a tenant's detail in `/admin` -> fill **Send announcement** (title + message) -> **Send to all members** -> confirm. **Expected:** "Announcement sent to N member(s)."
2. Sign in as a member of that tenant. **Expected:** the bell badge shows the unread announcement; opening it shows title + message; clicking it marks it read and the badge clears. Members with email delivery on also get the email (Mailpit).
3. The send is audited **in that tenant** (`admin.announcement.sent`, with the member count).
4. **Targeted variant:** check one or more member rows (new checkbox column) — the button becomes **"Send to N selected"** and only those members are notified (the count in the confirmation and the "sent to N" ack match the selection). Unchecking all reverts to all-members. The selection clears after a send and when switching tenants.

QA-ADMIN-05 — Platform-wide broadcast reaches every user • Edge (Web)

Gherkin

```
Given I am staff on /admin
When I send an "Announce to everyone" broadcast and confirm
Then the request is acknowledged as queued
And every user of every tenant receives it once the outbox delivers
```

Walkthrough

1. As staff on `/admin`, the **Announce to everyone** card sits above the tenant grid (it is not tenant-scoped). Fill title + message -> **Send to everyone** -> the confirm spells out the blast radius (EVERY user of EVERY tenant).
2. **Expected:** "Broadcast queued for delivery." — the fan-out is **asynchronous** (outbox): give the dispatcher a few seconds; delivery is not instant by design.
3. Sign in as users of **two different tenants**. **Expected:** both bells show the announcement.
4. There is **no in-tenant audit row** for a broadcast (the audit trail is per-tenant and this spans all tenants); the durable outbox message is the record.

QA-ADMIN-06 — Comp a tenant to Pro and revert • Edge (Web)

Gherkin

```
Given I am staff on a tenant's detail in /admin
When I use "Upgrade to Pro (comp)" (and later "Revert to Free")
Then the tenant's entitlements match the plan immediately, with no payment involved
And a provider-managed (Stripe-backed) subscription refuses the override
```

Walkthrough

1. As staff, open a **Free** tenant's detail. The header shows a **plan badge** (`free`) next to the status badge; the **Subscription** section shows **Upgrade to Pro (comp)**.
2. Comp it -> confirm. **Expected:** "Subscription updated."; badge flips to `pro` / `active`; the button is replaced by **Revert to Free**. The comp **never lapses** (no period end) and is audited in-tenant (`admin.subscription.comped`).
3. As that tenant's owner, check `/billing`. **Expected:** plan **Pro**, seat limit 10 — same plan + entitlements as a completed checkout, but **no Manage subscription button** (a comp has no provider customer, so the portal is absent). This simulates payment completion for entitlement purposes; QA-BILL/QA-HH-14 behaviors follow the plan.
4. **Revert to Free** -> confirm. **Expected:** badge back to `free`; entitlements fall back (absence => Free, fail-closed); audited (`admin.subscription.reverted`). Reverting again is a no-op.
5. **Provider-managed guard:** for a tenant with a **real Stripe subscription** (QA-BILL-02), the section shows "Managed by the billing provider — change the plan there." and **no** comp/revert buttons; the API refuses with 409 (Stripe stays the source of truth — ADR-006).

QA-ADMIN-07 — Reset a locked-out user's MFA • Edge (Web)

Gherkin

```
Given a user has MFA enabled but lost both their authenticator and recovery codes
When staff clicks "Reset MFA" on that user's row in the tenant detail and confirms
Then the user's MFA secret and recovery codes are wiped and they can sign in with primary auth alone
And the reset is audited in the user's tenant and the user is notified in-app and by email
```

Walkthrough

1. As a member user, enable MFA in Settings (QA-MFA-01), then pretend the authenticator and codes are lost — do **not** disable it. Sign out. Confirm sign-in now demands the code you "lost" (QA-MFA-02).
2. As staff on `/admin`, open the tenant's detail. Each member row now shows a red **Reset MFA** button next to **Sign in as**. Click it for the locked-out user. **Expected:** the confirm spells out the blast radius (authenticator + recovery codes stop working immediately, the user is notified) and reminds you to verify identity **out-of-band** first. Confirm.
3. **Expected:** "Two-factor authentication was reset for …" above the member table.
4. As the affected user, sign in again. **Expected:** primary auth alone signs you in — **no MFA step-up** (the second factor is gone; primary credentials are untouched). Settings shows MFA off; re-enrolling from scratch works (fresh QR + fresh recovery codes).
5. **Visibility:** the user's bell shows "Two-factor authentication was reset" and the email copy arrives in Mailpit; the tenant's audit trail (owner data export, QA-HH-13) contains an `admin.mfa.reset` event naming the staff actor.

6 **No silent no-op noise:** clicking **Reset MFA** for a member who never enrolled still reports success (idempotent) but writes **no** audit event and sends **no** notification.

10c. Web — Billing page • Core

> With no Stripe keys configured (dev default) the **FakeBillingProvider** is active: checkout/portal return `https://billing.test/...` URLs (a dead domain — expected), and the webhook accepts the signature `valid`. With real Stripe test keys, use Stripe's hosted test checkout instead.

QA-BILL-01 — Billing page shows plan + seats; owner-only • Edge (Web) Automated in CI

Gherkin

```
Given I am the household owner on /billing
Then I see my current plan, its status, and seat usage vs the plan limit
And a member opening /billing sees only the "ask your owner" notice
```

Walkthrough

1 As **owner**, open **Billing** in the header. **Expected:** current plan shown as its localized label (e.g. **Free** — the raw `free` token stays wire-only), status badge, and seats (e.g. `1 of 3 used`); an **Upgrade to Pro** button on the free plan; **Manage subscription** only when a provider customer exists (a comped tenant has a subscription row but no portal button).

2 As a **member**, open `/billing`. **Expected:** no plan/usage — just the owner-only notice.

QA-BILL-02 — Upgrade via checkout + provider webhook lands on Pro • Core (Web) Automated in CI

Gherkin

```
Given the owner on the free plan clicks Upgrade to Pro
When the checkout redirect fires and the provider webhook lands (subscription active)
Then the provider returns me to /billing/success — the Billing page with a "Payment received" banner
— and the page refetches on its own until it shows plan pro, status active, the pro seat limit, and
the portal button
And cancelling at the provider returns me to /billing/cancel — the same page with a "nothing changed"
banner and the plan untouched
```

Walkthrough

1 Click **Upgrade to Pro**. **Expected:** redirect to the provider checkout (fake: `billing.test/checkout/{tenantId}/pro` — the page won't load, which is fine).

2 Simulate payment completion by POSTing the webhook (fake provider): `POST {api}/api/billing/webhook` with header `Stripe-Signature: valid` and a PascalCase JSON body (`EventId`, `TenantId`, `PlanKey: "pro"`, `Status: "active"`, `StripeCustomerId`, `OccurredAt`). **Expected:** 200.

3 Open `/billing/success` (where a real provider returns the browser). **Expected:** the **Payment received** banner, then — within a few seconds, no reload — plan **Pro** / status **Active** (localized labels; raw tokens on the wire), seats `x of 10`, **Manage subscription** now visible, and the owner's bell (+ outbox email) carries a **Subscription active** notice naming the plan and its renewal date. `/billing/cancel` shows the **Checkout cancelled — nothing changed** banner over the unchanged plan.

4 Cancel from the portal (or delete the customer in Stripe) -> the webhook lands. **Expected:** plan **Free**, badge **Canceled**, the line reads **Your paid subscription ended on <date>** (never "renews"), and **Manage subscription** is gone — **Upgrade to Pro** is the way back (a fresh checkout).

10d. Web — The shelf (INV) • Smoke

> **JiggerJot's own flows begin here (§10d–§10i).** Everything above is the platform: sign-in, households, invitations, settings, billing, admin. Everything below is the product — FEATURES §7–§15.
>
> **The shelf comes first because nothing else works without it.** Until a household has said what it owns, "what can I make right now" has no answer, so a failure here cascades into §10f and §10h.
>
> **The QA shelf.** Several cases below need a known shelf. Unless a case says otherwise, tick **London dry gin**, **Campari** and **Sweet vermouth** — that is a Negroni, and it is the shortest path to a non-empty makeable list. Cases that name a specific drink use the curated catalog's names (JJ-043): the Savoy's "Cocktail" suffix is gone, so *Hawaiian*, not *Hawaiian Cocktail*.

QA-SHELF-01 — The shelf lists the whole catalog, grouped • Smoke (Web) Automated in CI

Gherkin

```
Given a household that has ticked nothing
When I open the shelf
Then I see every ingredient the catalog has, grouped by category
And nothing is ticked
```

Walkthrough

1. **Shelf** in the header (or the bottom tab bar on a narrow window).
2. **Expected:** cards per category — Gin, Rum, Vermouth, Juice and so on — each holding pills. The footer reads **"0 on the shelf"**.
3. **Expected:** the list is the WHOLE catalog, not just what you own. You cannot tick what you cannot see.

QA-SHELF-02 — Ticking is remembered • Smoke (Web) Automated in CI

Gherkin

```
When I tick an ingredient
And I reload the page
Then it is still ticked
```

Walkthrough

1. Click the **London dry gin** pill. **Expected:** it fills copper immediately, and the footer count goes up before any round trip finishes.
2. Reload. **Expected:** still filled, count unchanged. *(The tick is optimistic in the browser, so the reload is what proves it reached the database rather than only the screen.)*

QA-SHELF-03 — Unticking puts it back • Core (Web) Automated in CI

Walkthrough: click a filled pill. **Expected:** it returns to the outline style and the count drops. Reload — still unticked. *(Under the hood the row is updated rather than deleted, so "checked and I do not have it" stays distinguishable from "never looked". Not visible in the UI; see `Api.Tests`.)*

QA-SHELF-04 — A failed save rolls the tick back and says so • Edge (Web)

Walkthrough: stop the API (`Ctrl-C` in its terminal), then click a pill. **Expected:** the pill fills, then reverts, and a red banner appears. *(Optimism is only honest if it is undone — a tick that stayed put after a failed write is a lie the household finds out about later, when a drink they were promised turns out not to be makeable.)* Restart the API before continuing.

QA-SHELF-05 — "Only what I have" • Core (Web) Automated in CI

Walkthrough: with the QA shelf ticked, switch **Only what I have** on. **Expected:** exactly the three ticked ingredients remain, in their own cards. Switch it off — the full catalog returns.

QA-SHELF-06 — Search narrows by name and by category • Edge (Web)

Walkthrough: type `verm` in the shelf search. **Expected:** the vermouth pills, and any card whose CATEGORY matches. Type something absurd — **Expected:** "No ingredients match that." plus an **Add your own** button with the search text already in the name field.

QA-SHELF-07 — Add a custom ingredient inline • Core (Web) Automated in CI

Gherkin

```
Given a bottle the catalog has never heard of
When I add it from inside a category card
Then that category is already chosen
And it appears on my shelf, already ticked
```

Walkthrough

1. In any category card, click **+ Add your own**.
2. **Expected:** a form appears INSIDE that card with the **Category** dropdown already set to it.
3. Enter a name that cannot exist (`Homemade coffee liqueur <today's date>`), leave the subcategory blank, **Add to my shelf**.
4. **Expected:** it appears as a filled pill in that category, marked **yours**, and the count went up. You add a bottle to your shelf because it is on your shelf.
5. Reload — still there, still ticked.

QA-SHELF-08 — A duplicate name is refused and points at the original • Core (Web) Automated in CI

Walkthrough: add the same name again, in any case (`HOMEMADE COFFEE LIQUEUR …`). **Expected:** a warning saying it is already in the list, and the shelf search is set to that name so it is showing above — rather than leaving you to hunt for a name you just typed. **Expected:** the form reappears at the TOP of the page, since the search may have hidden the card it was opened from.

QA-SHELF-09 — Counts are honest, and the jump bar agrees • Core (Web)

Walkthrough

1. Note a category card's count, e.g. **Gin — 1 of 5**.
2. **Expected:** the same numbers appear on that category's chip in the scrollable jump bar above.
3. Tick another gin. **Expected:** BOTH move together — they read one value, so they cannot disagree.
4. Type a search that hides some of the category. **Expected:** the count still describes the WHOLE category, not what is left on screen. It measures progress against the shelf, and a number that changed meaning when you typed would measure nothing.

QA-SHELF-10 — The jump bar stays on the shelf • Core (Web)

Walkthrough: click a chip in the jump bar. **Expected:** the page scrolls to that category card with its heading clear of the header — and you are **still on the shelf**. *(A fragment-only link would resolve against `<base href="/">` and land on the home page; this case exists because it did.)*

QA-SHELF-11 — The payoff footer moves as you tick • Core (Web)

Walkthrough

1. With the QA shelf ticked, look at the sticky footer: **"3 on the shelf — that's N drinks so far"** and a **Show them** button.
2. Tick **Dry vermouth**. **Expected:** the bottle count moves at once; the drinks figure follows a moment later, once the ticking stops.
3. Tick three more in quick succession. **Expected:** the drinks figure updates once at the end rather than flickering per tick — the ask is deliberately deferred so clicking down the shelf does not cost a request per checkbox.
4. **Show them** -> the makeable list, with the toggle already on.

QA-SHELF-12 — A pill is still a checkbox • Edge (Web, accessibility)

Walkthrough: Tab to a pill. **Expected:** a visible focus ring; **Space** toggles it. With a screen reader (NVDA/VoiceOver), **Expected:** it is announced as a checkbox with the ingredient's name and its checked state — not as a button. *(The control is a real hidden `input[type=checkbox]` styled through its label, precisely so the semantics are the browser's.)*

QA-SHELF-13 — Ice and water never appear • Edge (Web)

Walkthrough: search the shelf for `ice`, then `water`. **Expected:** no ingredient row for either (matches on other names like "Soda water" are fine — what must not appear is plain ice or plain water as something to tick). *(They are always assumed available; a shelf that asks whether you have water is a shelf nobody trusts.)*

QA-SHELF-14 — Deleting a bottle I added, and the refusal while a recipe uses it • Core (Web)

Gherkin

```
Given a bottle my household added
When one of our recipes uses it and I delete it
Then I am told which recipe, and it stays
And once no recipe uses it, deleting takes it off the shelf
```

Walkthrough

1. Add a bottle (QA-SHELF-07), e.g. `Test orgeat <today's date>`. **Expected:** a small **×** beside its pill. No catalog bottle has one.
2. Click the **×** (not the pill) and **Cancel**. **Expected:** nothing changed, and the pill was not ticked or unticked by the click.
3. Write a cocktail that uses it (QA-MINE-01). Back on the shelf, **×** and confirm. **Expected:** a warning: *Can't delete Test orgeat…: it's used in <your cocktail>. Take it out of that recipe first.* The bottle stays.
4. Delete that cocktail (QA-MINE-10), then **×** the bottle and confirm. **Expected:** it leaves the shelf, its category count drops, and the payoff footer re-counts. Reload — still gone.
5. Switch to **Español** and repeat step 3 with a second bottle. **Expected:** the warning reads in Spanish.

10e. Web — Browsing the catalog (CKTL, FILTER, PREFS-2, PREFS-3) • Core

QA-CAT-01 — The catalog is browsable, searchable and paged • Smoke (Web) Automated in CI

Gherkin

```
When I open the catalog
Then I see cocktails with a count, ordered by name
And searching narrows the list
And paging moves through it
```

Walkthrough

1. **Cocktails** in the header. **Expected:** a count ("N cocktails") and a list, each row showing the glass, method, ingredient count and **the book it came from**.
2. **Expected:** the source is on every row and is load-bearing — several names appear in both books, and one appears twice in the Savoy alone, so without it the list shows duplicate rows with no way to tell them apart.
3. Search `negroni`. **Expected:** the list narrows and the count follows it.
4. Clear the search, go to page 2 and back. **Expected:** the list changes and the ordering is stable.

QA-CAT-02 — A recipe opens with its lines, method and credit • Smoke (Web) Automated in CI

Walkthrough: open any cocktail. **Expected:** every line in the author's order with amounts, the method and glass where the recipe states them, the instructions, and the source credited by name — written per source rather than from a template.

QA-CAT-03 — Every recipe names its glass; an unstated method says nothing • Edge (Web)

Walkthrough: page through the catalog. **Expected:** every row names a glass (JJ-043). Open the **Mojito**. **Expected:** a Highball glass — one of the five IBA drinks whose glass the curator chose, since the IBA names none. Find a recipe whose method is blank and open it. **Expected:** the detail page omits the method entirely rather than showing a guess or a placeholder (JJ-034).

QA-CAT-04 — Filtering by ingredient is wider than searching • Core (Web) Automated in CI

Gherkin

```
When I filter by an ingredient
Then I get drinks that do not have that word in their name
And a parent category catches its children
```

Walkthrough

1. **Filters** -> ingredient `gin` -> apply.
2. **Expected:** many more results than searching the NAME for "gin", because it reads the recipe lines. **Expected:** a Negroni appears; its name contains no "gin".
3. Try a CATEGORY name, e.g. `rum`. **Expected:** drinks using white, dark and spiced rum all appear — a parent matches every child.

QA-CAT-05 — Filters combine, and a dead end is an empty page • Core (Web) Automated in CI

Walkthrough: apply ingredient + method + glass together. **Expected:** the list narrows with each, the active-filter count on the button matches, and **Clear** resets everything. Combine filters that can have no result. **Expected:** an empty list with a message, never an error.

QA-CAT-06 — Every filter option returns something • Edge (Web)

Walkthrough: open each dropdown and pick a few options at random. **Expected:** none returns an empty list. *(The options are derived from the catalog rather than the curated lookups, exactly so no filter is a dead end.)*

QA-CAT-07 — Measurement preference changes what the recipe says • Core (Web) Automated in CI

Walkthrough

1. As a brand-new account, open **Settings** -> Preferences. **Expected:** two choices, **Imperial** and **Metric**, with Imperial already selected — there is no "As written" (JJ-041).
2. Open the Negroni. **Expected:** it reads **1 oz** of each, with fractions wherever ounces need them.
3. Switch to **Metric** and reopen it. **Expected: 30 ml** of each — the bar's 30 ml to the ounce, never 29.6. Any 3/4 oz line reads **22.5 ml**; millilitres are never shown as a fraction.
4. Switch back to **Imperial**. **Expected:** ounces again.
5. **Expected in both:** teaspoons, tablespoons and neutral units — dash, barspoon, piece, to taste — read exactly the same.

QA-CAT-08 — Another household's cocktail is not found • Core (Web) Automated in CI

Walkthrough: in a second household, fork or write a cocktail and copy its URL. Open that URL as the first household. **Expected:** a not-found state, not a permission error. *(The filter simply does not return another household's rows, so "forbidden" would be a claim the page is in no position to make — and saying it would confirm the row exists.)*

QA-CAT-09 — A search with no matches • Edge (Web)

Walkthrough: search the catalog for nonsense with no filters on. **Expected:** "No cocktails match that." and **no** illustration, **no** shelf button and **no** bottle suggestion — an unfiltered search returning nothing says something about the search, not about the household.

QA-CAT-10 — Every amount reads as a pour, never as parts or glasses • Core (Web)

Walkthrough

1. As Imperial, open the Savoy's **Absinthe (Special) Cocktail**. **Expected: 2 oz** absinthe, **1/2 oz** gin, **1/2 oz** anisette, then the dashes — no line says "part" (JJ-041).
2. Open the Savoy's **Hawaiian** (the book's 4 : 2 : 1). **Expected: 1 3/4 oz**, **3/4 oz**, **1/2 oz**.
3. Switch to **Metric** and reopen both. **Expected:** 60 / 15 / 15 ml and 52.5 / 22.5 / 15 ml.
4. Open any recipe that used a glass measure in the book (the batch recipes). **Expected:** ounces or millilitres, never "glass", "wineglass" or "liqueur glass".
5. **Write a cocktail** as a Metric reader. **Expected:** the unit list starts with **ml** and offers no oz, part or glass; write 45 ml of gin and save. **Expected:** it reads **45 ml**, and switching to Imperial reads **1 1/2 oz**.
6. On an existing database (staging): **Expected:** the recipes written before this release read in ounces too — the migration converted them.

10f. Web — What I can make (MAKE, ALMOST) • Smoke

> **The headline feature.** Run §10d first; these cases assume the QA shelf.

QA-MAKE-01 — What I can make follows my shelf • Smoke (Web) Automated in CI

Gherkin

```
Given nothing on my shelf
Then nothing is makeable
When I tick the three ingredients of a Negroni
Then the Negroni is makeable
And when I untick one, it is not
```

Walkthrough

1. With an empty shelf: **Cocktails** -> **I can make now**. **Expected:** an empty state with her illustration and a **Tick what you have** button — not a bare "no results".
2. Tick the QA shelf, return, toggle it on. **Expected:** the Negroni is listed.
3. Untick **Campari**, return. **Expected:** the Negroni is GONE — but the list is **not** empty. A household holding gin and sweet vermouth can still make several things. Nothing was recomputed or invalidated; it is derived at query time.

QA-MAKE-02 — A drink shown thanks to a substitute says what you would pour • Core (Web) Automated in CI

Walkthrough: tick **London dry gin** and **Curaçao** (not Cointreau) plus **Lemon juice**. **Expected:** the White Lady appears, with Marga saying which bottle stands in for which. Open it — **Expected:** the same swap is marked on the one line it applies to, and the card and the line agree.

QA-MAKE-03 — One ingredient away names the bottle • Smoke (Web) Automated in CI

Walkthrough: tick **London dry gin** and **Campari** only. **Cocktails** -> **One ingredient away**. **Expected:** the Negroni appears and the row NAMES **Sweet vermouth**. *("You cannot make this" on its own is not a shopping list.)*

QA-MAKE-04 — Buying it moves the drink across • Core (Web) Automated in CI

Walkthrough: from the state above, tick **Sweet vermouth**, then check both lists. **Expected:** the Negroni is now under **I can make now** and gone from **One ingredient away**. The two are adjacent and can never overlap — the same predicate, counted rather than negated.

QA-MAKE-05 — Both toggles at once is an empty page, not a winner • Edge (Web)

Walkthrough: switch **I can make now** and **One ingredient away** on together. **Expected:** an empty list — you asked for drinks that are both zero short and one short. Neither flag quietly wins.

QA-MAKE-06 — The one-away list leads with the bottle that opens the most • Core (Web) Automated in CI

Walkthrough

1. With a partial shelf, open **One ingredient away**.
2. **Expected:** a card above the list naming ONE bottle and how many drinks it opens, then those drinks by name.
3. **Expected:** the number and the names agree. If more than four are named, the display stops at four and counts the rest ("and 9 more") — the number still describes them all.
4. Reload twice. **Expected:** the same bottle each time; ties break on name so the card cannot appear to change its mind.

QA-MAKE-07 — The recipe says where it stands and which line is short • Core (Web) Automated in CI

Walkthrough: with gin and Campari only, open the Negroni. **Expected:** a badge saying it is one away, and a mark on the **sweet vermouth** line specifically — a reader wants to know WHICH line to do something about, not merely that one of them needs attention. **Expected:** the badge and the browse filters never disagree.

QA-MAKE-08 — The home screen answers the question • Smoke (Web) Automated in CI

Walkthrough

1. With the QA shelf, open **Home**.
2. **Expected:** Marga's line naming one purchase, the makeable COUNT as the headline, three of those drinks, and a card for the bottle that would open the most.
3. **Expected:** her sentence quotes a number from each half and each agrees with the list beneath it.
4. Click **Show me them** -> the makeable list with the toggle already on, and its count matches the headline. Click **See what else is close** -> the one-away list, likewise.

QA-MAKE-09 — Nothing makeable points at the shelf • Core (Web)

Walkthrough: with an empty shelf, open **I can make now**. **Expected:** her illustration, a line about the shelf rather than the catalog, and **Tick what you have** as a PRIMARY button. **Expected:** no bottle is suggested here — with nothing ticked, the next step is telling the app what you have, not going shopping.

QA-MAKE-10 — Nothing one away names a bottle to start from • Core (Web)

Walkthrough: with an empty shelf on the **shipped starter catalog**, open **One ingredient away**. **Expected:** the empty state names a bottle to start with and says how many recipes ASK for it — never how many it would unlock, because one bottle on an empty shelf makes very nearly nothing. **Expected:** turn the search box on with the filter still active and the suggestion DISAPPEARS — an empty list under a search says the search found nothing, not that you own nothing. *((!) If the full 969-recipe catalog is enabled this state is unreachable: single-ingredient recipes mean an empty shelf is one bottle away from several drinks. Record **N-A** and note the catalog size.)*

10g. Web — Making it mine (FORK, AUTHORING) • Core

QA-MINE-01 — Forking opens my own copy • Core (Web) Automated in CI

Gherkin

```
When I fork a cocktail from the shared catalog
Then I get my own copy, marked as mine
And it says what it was based on
```

Walkthrough

1. Open any shared cocktail -> **Create my own version**.
2. **Expected:** you land on a NEW cocktail marked **yours**, carrying "Based on <original>".
3. **Expected:** it has its own copy of every line, with amounts exactly as authored.
4. **Expected:** it carries **no book credit**. The book wrote the original, not the household's version.

QA-MINE-02 — The copy is a snapshot, not a reference • Edge (Web)

Walkthrough: fork a cocktail, then confirm the ORIGINAL still appears unchanged in the catalog and still shows its own source. *(Edits to the original never propagate; deleting it leaves the copy standing — the provenance link is deliberately not a foreign key. Proven in `Api.Tests`.)*

QA-MINE-03 — Writing my own cocktail • Core (Web) Automated in CI

Walkthrough

1. **Write a cocktail** from the catalog toolbar.
2. Give it a name, instructions, and two ingredient lines with amounts and units.
3. Save. **Expected:** you land on it, marked **yours**, with no source credit.
4. **Expected:** it appears in the catalog list for your household and NOT for any other.

QA-MINE-04 — A written drink joins makeable and filtering immediately • Core (Web) Automated in CI

Walkthrough: write a cocktail using two ingredients you have ticked. **Expected:** it appears under **I can make now** straight away, and filtering by one of its ingredients finds it. *(Both derive from the recipe lines at query time, so this is free rather than a second thing to keep in step.)*

QA-MINE-05 — The form refuses what it cannot store • Edge (Web)

Walkthrough: try each in turn — save with no ingredient lines; a unit with no amount; a name left blank. **Expected:** an inline message for each and nothing written. **Expected:** the method can be left blank and it still saves; the glass cannot (QA-MINE-11).

QA-MINE-06 — The authoring form's lists are not the filter lists • Edge (Web)

Walkthrough: compare the **glass** dropdown on the authoring form with the one in the catalog filters. **Expected:** the form offers MORE — the whole curated set, so someone writing down what they pour can reach a glass no seeded recipe happens to use. The filter offers only what the catalog uses, so no filter is a dead end. *(Two endpoints on purpose; they must not be merged.)*

QA-MINE-07 — The ingredient suggests the line's role • Edge (Web) Automated in CI

Walkthrough

1. **Write a cocktail.** Add four lines and pick, in order: **London dry gin**, **Campari**, **Lime juice**, **Mint**. **Expected:** without touching the role dropdowns they read **Base**, **Modifier**, **Juice**, **Garnish** — and the mint line's **Required** box is unticked.
2. Change the second line to **Cognac**. **Expected:** it reads **Modifier** — only the first spirit is the base.
3. Remove the gin line. **Expected:** the cognac line becomes **Base**.
4. Set the lime line's role to **Other** by hand, then change its ingredient to **Soda water**. **Expected:** it stays **Other** — a choice made by hand is never overwritten.
5. Save. **Expected:** the recipe page lists each line under the role it had on the form.

QA-MINE-08 — The ingredient is a searchable picker • Edge (Web) Automated in CI

Walkthrough

1. **Write a cocktail.** Click a line's ingredient box. **Expected:** a list of every bottle, grouped by category then name; bottles you have ticked say **on your shelf**.
2. Type **gin**. **Expected:** only gins, names starting with "gin" first. Type **juice**. **Expected:** every juice.
3. Pick one with the mouse. **Expected:** the box shows its name and its role fills in (QA-MINE-07).
4. On another line, type a few letters, press ↓ twice and **Enter**. **Expected:** the second match is picked. Type again and press **Escape**. **Expected:** the line keeps the bottle it had.
5. Type **yuzu** (not on the list). **Expected:** "No bottle by that name. Add it on your shelf first."
6. With a screen reader: **Expected:** the box is announced as a combobox, and each option is read as the arrow keys reach it.

QA-MINE-09 — Editing a cocktail I wrote or forked • Core (Web) Automated in CI

Walkthrough

1. Open a cocktail your household **wrote**. **Expected:** an **Edit** button beside *Create my own version*. Open a book's recipe (the Negroni). **Expected:** no Edit — only *Create my own version*.
2. On your own cocktail, **Edit**. **Expected:** "Edit cocktail", every field and line filled in, roles as they were.
3. As a **Metric** member, edit a recipe with a 1½ oz line. **Expected:** the line reads **45 ml**.
4. Change the name, remove a line, add one, save. **Expected:** back on the recipe, which says exactly what you saved, lines in your order; its makeability badge follows the new lines.
5. **Fork** the Negroni, then **Edit** the copy and save. **Expected:** it still says **Based on Negroni**; the book's Negroni is unchanged.
6. Save an edit with the name blank. **Expected:** the inline message, and nothing changed.
7. In two browsers as two members, edit the same recipe and save both. **Expected:** the second save is what the recipe says.

QA-MINE-10 — Deleting a cocktail I wrote or forked, and the actions row • Core (Web) Automated in CI

Gherkin

```
Given a cocktail my household owns
When I choose Delete and confirm
Then it is gone and I am back on the catalog
And a book's recipe offers no Delete at all
```

Walkthrough

1. Open a cocktail your household **wrote**. **Expected:** under the ingredients, ONE row: **Create my own version** (copper), **Edit**, and a red **Delete** link at the far end. Open the Negroni. **Expected:** only *Create my own version*.
2. On a phone width (390px), open your own cocktail. **Expected:** the fixed bar at the bottom holds all three on a single line, nothing clipped.
3. **Delete**, then **Cancel** in the dialog. **Expected:** nothing happens; the recipe is still there.
4. **Delete** again and confirm. **Expected:** the dialog named the cocktail; you land on the catalog and it is no longer listed. Its old URL shows the not-found state.
5. Write a cocktail, fork it, then delete the **original**. **Expected:** the fork is still there with every line.

QA-MINE-11 — No glass, no cocktail: Marga says so beside the field • Core (Web) Automated in CI

Gherkin

```
Given I am writing a cocktail with a name and its lines
When I save without choosing a glass
Then nothing is sent, and Marga says beside the glass that it needs one
When I choose a glass
Then she goes, and the cocktail saves
```

Walkthrough

1. **Write a cocktail. Expected:** the glass list opens on **Choose a glass**, and there is no "Not stated" glass; the method list still has *Not stated*.
2. Give it a name and a line, leave the glass, and **Save**. **Expected:** you stay on the form; the glass is outlined red and Marga, small, beside it: *"A Martini without its glass is just cold gin. Pick one."* (ES: *"Un Martini sin su copa es solo ginebra fría. Elige una."*). Nothing in the summary above Save.
3. Choose **Rocks glass**. **Expected:** she goes at once. **Save. Expected:** you land on the recipe, which names its glass.
4. **Edit** it, set the glass back to *Choose a glass*, and save. **Expected:** the same line, nothing written.
5. *(API)* Send the create request with `glassTypeId` null. **Expected:** **400 `glass_required`**.

10h. Web — The first minute (ONBOARD) • Smoke

> **Needs a brand-new household.** Sign in with an address that has never been used, so the platform provisions a fresh tenant with an untouched shelf (§6, QA-ONB-01).

QA-START-01 — A new household is offered the guided route • Smoke (Web) Automated in CI

Gherkin

```
Given a household that has ticked nothing
When I land on the home page
Then it leads with setting up my shelf
And the raw checklist is still reachable
```

Walkthrough

1. Sign in as a brand-new user. **Expected:** the home page shows Marga's empty line and **Set up my shelf** as the primary button, with **Tick what you have** below it as a quieter link.
2. **Expected:** nothing redirects you. A wizard you cannot dismiss is a modal, not an offer.

QA-START-02 — The suggestions arrive ticked • Smoke (Web) Automated in CI

Walkthrough: click **Set up my shelf**. **Expected:** step 1 of 2, "The usual suspects", about a dozen bottles ALREADY filled, and a count reflecting them. *(Answering "which of these is wrong" is far faster than picking a dozen bottles out of 191.)* **Expected:** the bottles shown are the ones the most recipes ask for — not a hand-curated list.

QA-START-03 — Nothing is written until Finish • Core (Web) Automated in CI

Walkthrough

1. Untick one suggestion, click **Next**, then — **without finishing** — navigate to **Shelf**.
2. **Expected:** the shelf is still empty. **"0 on the shelf".**
3. *(Had the wizard written as it went, it would have recorded a shelf you never confirmed — and left half of one behind for anyone who closed the tab midway.)*

QA-START-04 — Step two is the rest of the catalog • Core (Web)

Walkthrough: back in the wizard, go to step 2. **Expected:** "Anything else on your shelf" — the full catalog by category, the **same pill control as the shelf screen**, with a search. **Expected:** anything selected on step 1 is still selected here, and **Back** returns without losing anything.

QA-START-05 — Finish lands on something to pour • Smoke (Web) Automated in CI

Walkthrough

1. Click **Save my shelf**.
2. **Expected:** you land on **what I can make**, with the filter already applied rather than on the whole catalog — and, on a shelf built from the suggestions, it is populated.
3. Open **Shelf**. **Expected:** exactly what you confirmed, with the one you unticked absent.
4. *(A single request carries the whole shelf, so it either arrives as you left it or not at all.)*

QA-START-06 — I can leave • Core (Web)

Walkthrough: on a fresh household, open the wizard and click **Skip for now**. **Expected:** you are in the catalog and the shelf is still empty — nothing was written. **Expected:** the home page still offers the wizard; skipping is "later", not "never".

QA-START-07 — A member joining by invitation never sees it • Core (Web)

Walkthrough: have a second user accept an invitation into a household that already has a shelf (§8). **Expected:** their home page shows the household's makeable COUNT, not the setup offer. *(The household already has an inventory, so there is nothing to seed — handled by the same condition rather than by a special case.)*

10i. Web — Marga and the chrome (MARGA, SHELL) • Edge

> **Marga is a drawn character, not an assistant.** Every sentence she says is a localized resource string with real data in its placeholders. There is no model behind her and nothing generates text. These cases check that the data is real and the wording is fixed.

QA-CHROME-01 — Her number and the list beneath it agree • Core (Web)

Walkthrough: on Home with a partial shelf, read her line, then check each number against the list directly under it. **Expected:** they match exactly. *(If her sentence and the list disagree, the bug is that they came from two queries.)*

QA-CHROME-02 — She is optional furniture • Edge (Web)

Walkthrough: stop the API, then load Home. **Expected:** the screen renders WITHOUT her rather than with an empty speech line. Restart the API.

QA-CHROME-03 — She is decorative to a screen reader • Edge (Web, accessibility)

Walkthrough: with a screen reader, move through Home. **Expected:** her picture is skipped entirely — the sentence beside her carries the whole meaning, and "photo of a bartender" ahead of it would only get in the way.

QA-CHROME-10 — She is where I spend my time • Core (Web)

Gherkin

```
Given a shelf with something ticked
Then she names the one bottle that would open the most
And she does not repeat the count the footer already shows
```

Walkthrough

1. Open the **shelf** with the QA shelf ticked. **Expected:** she is at the TOP, naming a bottle and how many more drinks it would open.
2. **Expected:** she does **not** repeat the footer. The footer counts what you HAVE; she names what you are SHORT of. If the two say the same thing, that is the defect.
3. Untick everything. **Expected:** she switches to suggesting where to start — the one-away set is empty with nothing ticked, so there is nothing to rank.

QA-CHROME-11 — She reads the catalog's count against the shelf • Core (Web) — MARGA-5, amended by MARGA-6

Walkthrough: open the catalog under **I can make now** and note the number. Switch to **Everything**. Then open Filters and type "gin" in Made with. Then switch to **One ingredient away**. **Expected:** under I can make now she gives the number in her voice. Under Everything she says "N cocktails here, and you can pour M of them tonight", and **M is the number I can make now showed**. With the gin filter both numbers drop, and M is what I can make now shows with the same filter on. A household that can pour none of them hears "none you can pour yet", never "0". Under One ingredient away the count is plain "N cocktails" — her panel below already speaks there, and two of her on one screen is one too many.

QA-CHROME-12 — She can say yes • Edge (Web)

Walkthrough: open a recipe you **can** pour, then one you are **one bottle** short of, then one **two or three** bottles short, then one **four or more** short. **Expected:** "you can pour this now" on the first; the missing bottle named on the second; **every missing required bottle named** on the third ("You are 2 bottles short: Campari and Sweet vermouth") and never an optional garnish; on the fourth the count and the first two. Each name matches a line marked "not on your shelf". **Expected:** where a substitution is in play, that outranks all of these, since it is the reason the drink qualified at all.

QA-CHROME-13 — She says nothing rather than something empty • Edge (Web)

Walkthrough: stop the API, then reload the shelf. **Expected:** she is absent — not a blank speech bubble, and not a confident line she has nothing behind. **Expected:** the footer's bottle count is unaffected; it is a fact the screen owns rather than something she supplies.

QA-CHROME-26 — She offers the fix on an empty search, and counts the shelf on the write form • Edge (Web) — MARGA-6

Walkthrough

1. On the **shelf**, search for "yuzu". **Expected:** a small inline Marga (32px, no label) says "Nothing called “yuzu” on my list. If you have it, add it below.", with **Add your own** under it; her line at the top of the shelf is still the only large one.
2. Turn on **Only what I have** and search for a bottle you have NOT ticked. **Expected:** the plain "No ingredients match that." and **no Marga** — the bottle is on the list, just hidden.
3. Open **Write a cocktail**. **Expected:** she says your shelf has N bottles to write with, and N is the "N on the shelf" figure the shelf footer shows. With a household that has ticked nothing she says to write it anyway.
4. Switch to Español and repeat 1 and 3. **Expected:** her lines read as written Spanish, and the term and the number survive.

QA-CHROME-27 — She pages through the next bottles • Core (Web) — MARGA-7

Walkthrough — a household with a few bottles ticked, and a second one with nothing ticked.

1. **Home.** In the panel under her line, press **›**. **Expected:** her line changes to "Or pick up …", the box directly under her line lists that bottle's drinks, and the position reads **2 of N**. Keep pressing: it stops at the last bottle — never more than **10** — and **›** is greyed out. **‹** at the first bottle is greyed out too.
2. **Shelf.** Press **›** under her line. **Expected:** "Or {bottle}: one bottle from N more drinks", and the **outlined pill** moves to that bottle. Tick any bottle. **Expected:** her suggestion goes back to **1 of N**.
3. **Catalog -> One ingredient away.** Press **›**. **Expected:** her line and the drink names under it follow the bottle, while the **list below does not change**.

4. **The household with nothing ticked.** On the **shelf** and on the catalog's **One ingredient away** empty state, press **›**. **Expected:** later bottles say how many recipes **ask for** them — never how many drinks they open.
5. With only one bottle to suggest: **Expected:** no arrows at all.
6. Keyboard only: Tab to the arrows and press Enter. **Expected:** a visible focus ring, and a screen reader announces her new line and the arrows as "Previous bottle" / "Next bottle".
7. Switch to Español and repeat 1. **Expected:** "O consigue …", "2 de N".

QA-CHROME-04 — The destinations follow the width • Core (Web) Automated in CI

Walkthrough

1. At a wide window: **Expected:** Home, Shelf and Cocktails sit in the header beside the brand, and they read at least as strongly as Household/Billing/Settings on the right.
2. Narrow the window below ~990px. **Expected:** they move to a **bottom tab bar**, reachable without opening anything, and the hamburger now holds only the account cluster.

QA-CHROME-05 — The current destination is obvious and announced • Edge (Web, accessibility)

Walkthrough: navigate between the three. **Expected:** the current one is **bolder** and carries a drawn indicator bar — distinguishable without relying on colour. With a screen reader, **Expected:** it is announced as the current page. **Expected:** opening a recipe keeps **Cocktails** current.

QA-CHROME-06 — The tab bar does not cover the page • Core (Web)

Walkthrough: on a narrow window, scroll to the bottom of the **shelf** and of the **catalog**. **Expected:** the last thing on each page is fully readable above the bar, and the shelf's payoff footer sits clear of it rather than behind it.

QA-CHROME-07 — The boot state shows real progress • Edge (Web)

Walkthrough: hard-reload with the cache disabled (DevTools -> Network -> Disable cache) and watch the first second. **Expected:** the illustration, gently rocking, at up to 480px wide, with the brass arc and its percentage **centred on the drawing, on a small night disc**, and the ring does not rock with her. **Expected:** the arc's length matches the number printed inside it — a quarter turn at 25%, a full ring at 100%. *(Also verify on Desktop and Android; the native hosts show a sweeping arc instead, because a WebView has no download to measure.)*

QA-CHROME-08 — Reduced motion is respected • Edge (Web, accessibility)

Walkthrough: turn on the OS "reduce motion" setting, then hard-reload. **Expected:** nothing on the boot screen moves on its own. *(A boot screen is the one thing nobody can choose to skip.)*

QA-CHROME-14 — The display face is the app's own, in both themes • Edge (Web) — BACKBAR-1

Walkthrough: open the app with the browser's network panel filtered to fonts, sign in, and open Home; then switch the theme to Dark, then Light (§9 QA-SET-08). **Expected:** every font request goes to the app's own origin (`_content/JiggerJot.Shared.Ui/fonts/…`) and none to a font host; the serif that Marga's card and the headlines use renders as a real serif, not the system fallback, within a second of the page appearing; nothing set in it is bold or smeared (the face has one weight). In Dark the page ground is the night colour, panels are one step lighter with a hairline and **no shadow**; in Light the ground is warm off-white and panels are white with a soft shadow. Buttons are pills and fields have rounded corners in both. Tab to any button or field: one copper focus ring with a gap in the ground colour, and no ring on a mouse click.

QA-CHROME-15 — The chrome sits on the page, not on a copper band • Edge (Web) — BACKBAR-2

Walkthrough: sign in and look at the header at 1280 wide in Light, then Dark (§9 QA-SET-08); then narrow to 390 and look at the tab bar in both. **Expected:** the bar is the page's surface colour with a 1px hairline beneath it (above it, for the tab bar) — never copper, never a shadow in Dark. The mark reads on both grounds (copper on light, bone on dark). Home / Shelf / Cocktails are in the page's text colour and the current one is bold with a **2px copper line** under it (over it, in the tab bar); the account buttons beside them are quieter, muted with a hairline border, and "Sign out" looks exactly like the "Household" and "Settings" buttons next to it — no button is painted the link colour. Hover a destination: it turns copper. The household name is a copper-tinted pill.

QA-CHROME-16 — Home is rules, not cards, and never a zero • Core (Web) — BACKBAR-3

Walkthrough: with a partial shelf, open Home at 1280 wide, then at 390; then untick everything on the shelf and come back. **Expected:** at 1280 two columns — on the left, Marga at 96px with her label above her line, directly under it the copper-tinted box with that bottle's drinks and the ‹ › arrows (**the only boxed thing on the page**, with no headline repeating her), then the big serif count and its two pill buttons; on the right, up to eight drinks as rules with the name in the serif and the facets right-aligned, the two columns ending near each other. **In Dark as well as Light, a drink's name is in the page's ink and turns copper only under the pointer** — the same on every catalog row (a dark-theme link rule once repainted all of them copper, and only a browser showed it). At 390 the same in one column. With nothing ticked: her scene beside her line, "Set up my shelf" as the primary pill and "Tick what you have" as a text link, and **no zero anywhere**. Reload with the network throttled: a thin brass bar under the header and a dimmed dash where the count will be, no spinner.

QA-CHROME-17 — The shelf is sections, and the named bottle can be found • Core (Web) — BACKBAR-3

Walkthrough: open the shelf with a partial shelf that leaves something one bottle away (§10d QA-SHELF-01 fixture). **Expected:** the title is in the serif; Marga at 96px with "MARGA" above her line; each category is a serif heading on a hairline with "n of m" on the right — **no boxes** around the pills; the bottle her line names is outlined in copper and is **not ticked**; ticking it fills it copper like any other. Press "+ Add your own" inside a category: the form opens in place under that category's pills as a rounded panel. The sticky footer's count is in the serif and copper, the drinks phrase beside it in the sans, and it still clears the tab bar at 390 (QA-CHROME-06).

QA-CHROME-18 — The catalog's three chips are one control • Core (Web) — BACKBAR-4

Walkthrough: open the catalog with a partial shelf. Click "I can make now", then "One ingredient away", then "Everything"; then Tab to the chip row and use the arrow keys; then follow Home's "Show me them". **Expected:** exactly one chip is filled copper at any time and only that chip carries a count ("· 14"); the arrow keys move the selection the way a radio group does and a screen reader announces "radio, 1 of 3"; the deep link lands with the right chip selected. The search is a pill; the rows are hairlines with the name in the serif and the book on the right, never wrapping; a substituted drink shows Marga at 32px with her line in copper; the pager is text on a hairline. At 390 the chip row stays at the top while the list scrolls. No box anywhere except the one-away panel. **At 600 wide** (a tablet, a narrow window): the search takes the row, Filters and Write a cocktail sit **left-aligned** under it rather than pushed to the right edge, and the three chips stay **on one row** that scrolls sideways if it must — "Everything" never drops to a line of its own.

QA-CHROME-19 — A recipe reads by its measures, and prints • Core (Web) — BACKBAR-4

Walkthrough: open a drink you are one bottle short of, whose recipe also has a substitution and an optional garnish (§10f QA-MAKE-05 fixture), at 1280 and at 390; then print preview. **Expected:** the title in the serif at up to 52px, wrapping rather than cut; status pill beside the facets; "Based on X" in the facet line for a fork, not as its own paragraph; Marga at 72px with her line in the serif. Amounts in a 104px column in the serif and copper; the garnish shows a dash, a muted name and the word "optional" as plain text — no badge. Exactly two marks: "you'd pour X" on the substituted line and "not on your shelf" on the missing one; pourable lines say nothing. At 1280 the method sits left and the ingredients right; at 390 the ingredients come first and "Create my own version" is a bar fixed above the tab bar. Print preview: white ground, no header, no tab bar, no fork button. A bad id shows her scene, the not-found line and a link back — not a yellow alert.

QA-CHROME-20 — Her two full appearances, and the two screens that borrow them • Core (Web) — BACKBAR-5

Walkthrough: sign out and open `/login` at 1280 wide in Light, then Dark; then at 390×812. Send a 6-digit code. Then sign in as a new household and open `/welcome`, step 1 and step 2. Then open `/join` with no token, and `/auth-error`. **Expected:** on login **a framed card centred on the page ground** (not edge to edge): her scene on the left is **square, so the whole drawing shows** — shaker, glass and bar counter included — with her sentence in the serif over the counter and the "Marga · behind the bar" label above it; the form on the right is centred, the wordmark over "Sign in" and "Choose how you'd like to continue" beneath it, then the buttons; there is **nothing over her**: the wordmark is on the form side only, on all three of login, join and the error page. **Her panel stays night in both themes**, only the form side flips. Join and the error page keep their framed split. **At 390 the card stacks: her scene is a full-width square at the top and the form starts under it** — no cropped face, nothing sliding up over her; at 768×1024 the same, with the form below the fold. The code step swaps the two buttons for one 24px spaced field and she does not move. The wizard is the same split with a warm base, "Step 1 of 2" over a thin brass rule at half, full at step 2; step 2's

categories are the shelf's own sections with "n of m", pre-ticked bottles filled copper; the running count is the serif copper figure. **On step 2 at 1280, scroll the whole list: her scene stays one screen tall and follows you down under the header, never a face stretched to the height of the list**, and the column below her is the warm night ground. Join and the auth error page use the same split; the error page's line is the error heading with no "Marga" label. No box anywhere; the app header stays on the wizard and is absent on login.

QA-CHROME-21 — Writing a cocktail is two columns, and the amounts preview the recipe • Edge (Web) — BACKBAR-6

Walkthrough: open "Write a cocktail" at 1280 and at 390. Press Save with nothing filled in; then give it a name and press Save with no ingredient chosen; then choose an ingredient, type an amount, pick a unit, add a second line, remove it. **Expected:** the drink's fields on the left, the ingredients on the right, no boxes; the title in the serif. Save with nothing filled in marks the **name field** red with its message under it and shows no alert at the top; a name with no complete line shows the message under the lines. Each line is one hairline row: the amount in the serif and copper beside its unit, the ingredient with role and required visible on a sub-line — nothing to open — and an × at the end (disabled when it is the only line). At 390 it is one column and "Save this recipe" is a bar fixed above the tab bar. A server rejection still shows above Save.

QA-CHROME-22 — Settings is two columns of rows, and the choices are visible • Core (Web) — BACKBAR-7

Walkthrough: open Settings at 1280 and at 390 with a provider linked and 2FA off. Click the "Dark" pill, then "Light"; click "oz" then "As written"; toggle a notification switch; press "Delete my account" and cancel the dialog. **Expected:** sign-in providers and two-factor on the left, preferences and notifications on the right, each a labelled group of hairline rows with no box; the title in the serif. Theme and Measurements are three pills each with the current one filled copper, and each click applies at once (the theme flips live, the header's own switcher follows) and sends one save. Language is still a dropdown. Unlink and Link are text links, Unlink red. "Delete my account" is a single red text link at the foot, no red box, and it still asks first. Enable 2FA: the flow still expands in place — QR on white, manual key, code field, then the codes in a panel.

QA-CHROME-23 — Household is four groups, and the owner is the only copper badge • Core (Web) — BACKBAR-7

Walkthrough: as owner with one member and one pending invitation, open Household at 1280 and at 390; then as a member. **Expected:** the name and the members on the left; invitations, transfer, data and leave on the right; no boxes. Owner's badge is copper-tinted; Admin and Member are neutral. Make admin / Remove, Regenerate / Revoke and Download household data are text links, the destructive ones red, each still asking first. At 390 a member row shows "⋯"; tapping it reveals that row's links beneath the row and tapping again hides them. As a member: the household name in the serif, the roster and nothing else — no rename, no invite, no transfer — and "Leave" as a red text link. The notification bell's list is hairline rows with a brass dot on unread.

QA-CHROME-24 — The pages nobody drew look like the rest • Edge (Web) — BACKBAR-8

Walkthrough: with billing enabled, open Billing as owner and as a member; as platform staff open the console, pick a tenant, send an announcement; then visit a URL that does not exist; then sign in through a provider and watch the callback; then impersonate someone. **Expected:** Billing is two labelled groups — the plan name in the serif with its status as a green/amber/neutral pill, the seats as the serif copper figure — with no box; a member sees a muted one-line pointer, not an alert. The console: the broadcast as the one panel; tenants as hairline rows, the chosen one copper; the tenant's name in the serif; members as rows with text-link actions, the reset red; every status a single inline notice. A missing page shows her scene, the line and "Go home". The callback shows a thin brass bar, not a spinner. The impersonation banner is a thin amber strip under the header. Force an unhandled error (§14): the error bar at the foot is the red-on-rose error colour.

QA-CHROME-25 — The sweep: nothing gives the old app away • Edge (Web) — BACKBAR-9

Walkthrough: in both themes, walk every screen (Home, Shelf, Cocktails with each chip, a recipe, Write, Welcome, Settings, Household, Billing, Admin, Login, Join, the auth error page, a missing URL) at 1440, 768 and 390 wide. On Household, Join and the console, look at Rename, Invite, Transfer, Join and "Send to everyone" BEFORE typing anything. Open the bell. Then switch the OS to reduce motion and reload the shelf, the wizard and the boot screen. **Expected:** no box with a border on any screen except the copper-tinted unlock panel on Home and Cocktails — the bell's list is a floating panel with hairline rows, not a card. Every disabled primary button is copper at reduced opacity,

never blue. Join's heading is the serif, like Login's. With motion reduced, ticking a pill changes colour instantly, the wizard's progress rule jumps rather than slides, and the boot arc steps to each figure. Tab through any screen: one copper ring on every control, no element skipped and none with the browser's blue outline.

QA-CHROME-09 — The app speaks Spanish throughout • Core (Web)

Walkthrough: switch the language to Español (§10) and walk the app's own screens — shelf, catalog, a recipe, makeable, one-away, the wizard, both empty states. **Expected:** no English leaks, and **Marga's lines read as written Spanish rather than as a translation** — her lines are voiced, so a literal translation reads like a machine. **Expected:** the numbers and names in her placeholders survive the rewrite.

11. Emails (Mailpit) — branding & content • Core

> **Delivery is asynchronous** (the outbox dispatcher) — emails land in Mailpit a few seconds after the triggering action, and the send request succeeds regardless (see §1.1). Wait briefly before opening Mailpit; a missing email is a delayed/retrying/dead-lettered outbox message, not a request failure.

QA-MAIL-01 — OTP email is branded & correct • Core

Gherkin

```
Given I request an OTP code
When I open the email in Mailpit
Then it looks like the app: the lockup, the warm ground and copper, the heading in the display serif,
the code, and the tagline
```

Walkthrough: trigger an OTP; open it in Mailpit, then in Gmail if you have it. **Expected:** the JiggerJot lockup renders (CID-embedded, not a broken image) **on the warm ground with no white box around it**; a white card on a hairline; the heading and Marga's line in the **display serif**, never bold (Gmail and Outlook on Windows drop the embedded face and show **Georgia** — that is the fallback, not a bug); body text in the app's sans; the code large in a hairline field; the footer reads "JiggerJot · Mix what you have." Nothing in the old cool grey, no bold copper heading.

QA-MAIL-02 — Magic-link email • Core

Walkthrough: trigger a magic link; open in Mailpit. **Expected:** the same look as QA-MAIL-01; a working sign-in button that is a **copper pill** (square in Outlook on Windows, which ignores the radius); sensible subject.

QA-MAIL-03 — Invitation email • Core

Walkthrough: send an invite; open in Mailpit. **Expected:** branded layout; the recipient's address; a `/join?token=…` link that works (QA-INV-02).

QA-MAIL-04 — Logo renders in a real client • Edge

Walkthrough (optional, deliverability): forward/inspect one email in a real Gmail/Outlook client. **Expected:** the CID lockup and Marga still render, and the heading falls back to Georgia rather than Times New Roman. *(Note: from a non-verified domain, real delivery may land in spam — a domain/DKIM concern, not an app bug.)*

QA-MAIL-05 — Marga is on the emails a person asked for, and only those • Core

Gherkin

```
Given an email I asked for — a sign-in code, a sign-in link, an invitation
Then Marga says one line near the top, above the thing I came for
But given a system notification — a failed payment, a security alert
Then she is not there at all
```

Walkthrough

1. Trigger an OTP, a magic link and an invitation. **Expected in each:** her avatar at 72px with "MARGA" as a small copper label above her line, the line in the serif, all above the code/button — and **Inline images (2)** in Mailpit's header, the lockup plus her.
2. Trigger a notification email (a billing or announcement one — §10b, §10c). **Expected:** no Marga, and **Inline images (1)**. She is not merely hidden; her 19 KB does not ride along on an email that never shows her.

3 *(Why: that template wraps arbitrary system messages, including "your subscription is past due" and a security alert. A bartender character on those undercuts the message.)*

QA-MAIL-06 — Her email lines read as written Spanish • Edge

Walkthrough: set the language to Español, request an OTP and have someone invite you. **Expected:** her line is written Spanish, not a translation of the English, and the rest of the email is Spanish too. *(Her lines are voiced; a literal translation of a voiced line reads like a machine.)*

QA-MAIL-07 — A client that blocks images loses only her • Edge

Walkthrough: open an OTP email with images blocked (Mailpit's Text tab, or Gmail with "ask before displaying"). **Expected:** her sentence is still there and still makes sense; no "photo of a bartender" alt text sits in front of it; the code and the button are unaffected. *(Her avatar has an empty `alt` for the same reason it does in the app — the line beside her carries the whole meaning.)*

12. Desktop — MAUI / Windows • Core

> The desktop client reuses the same UI; only the **auth transport** (loopback browser flow, body token, OS secure storage) differs. Magic link is intentionally **absent** on native.

QA-DSK-01 — OTP sign-in • Smoke (Desktop) — CI covers boot-to-login only

Gherkin

```
Given the desktop app is on the login screen
When I request an OTP and enter the code from Mailpit
Then I am signed in within the app (no browser)
```

Walkthrough

1 Launch the desktop app -> login screen. **Expected:** Google/Microsoft buttons + email field with **Email me a code**; **no magic-link button** (native hides it).
2 Enter an email -> request code -> get it from Mailpit -> enter it.
3 **Expected:** signed in, app shell shown, household + name in header.

QA-DSK-02 — Google OAuth via loopback • Core (Desktop)

Gherkin

```
Given the desktop login screen
When I click Continue with Google and complete consent in the system browser
Then the browser shows a "you can close this" page and the app completes sign-in
```

Walkthrough

1 Click **Continue with Google**. **Expected:** your **system browser** opens to Google consent.
2 Approve. **Expected:** the browser tab shows a "you can close this tab" page; **focus returns to the app**, now signed in. (Repeat for **Microsoft**.)

QA-DSK-03 — "Remember me" across app restart • Core (Desktop)

Gherkin

```
Given I am signed in to the desktop app
When I fully close and reopen it
Then I am still signed in
```

Walkthrough: close the app completely; relaunch. **Expected:** lands signed in (refresh token held in Windows secure storage / DPAPI, silently exchanged on startup).

QA-DSK-04 — Household & Settings load (authorized API calls) • Core (Desktop)

Gherkin

```
Given I am signed in to the desktop app
When I open Household and Settings
Then both load their data without an auth error
```

Walkthrough: open **Household** (members load) and **Settings** (linked providers load). **Expected:** both populate — no spinner-forever, no "couldn't load" error. *(This is the Bearer-token-handler path that previously failed on native; it must work.)*

QA-DSK-05 — Link a provider from Settings • Edge (Desktop)

Walkthrough: Settings -> **Link** Microsoft -> system browser loopback flow -> returns to app. **Expected:** Microsoft shows **Connected**; the app shell was never navigated away.

QA-DSK-06 — Sign out • Edge (Desktop)

Walkthrough: Sign out. **Expected:** back to the login screen; reopening the app does **not** auto-sign-in (secure storage cleared).

QA-DSK-07 — Core household flows work on desktop • Edge (Desktop)

Walkthrough: spot-check rename, invite (token revealed), and leave on desktop. **Expected:** same behavior as web (§7–8) — the UI is shared.

> **NATIVE Wave 2 (ADR-018, `docs/NATIVE_PARITY.md`):** the cases below verify the parity fixes per-feature on desktop — join-by-code (G5), culture persistence (G6), export share (G1), and the billing return-trip (G2) — plus the feature areas the old plan never exercised natively.

QA-DSK-08 — Join a household by pasted invite code • Core (Desktop)

Gherkin

```
Given a member account signed in on desktop and an invite code from the owner
When I open Household -> "Have an invite?" and paste the code
Then I join the owner's household exactly as the emailed link would
```

Walkthrough

1. As the **owner** (web is fine): Household -> invite the member's email -> copy the revealed token.
2. On **desktop** as the member: **Household** -> **Have an invite?** -> paste the code -> **Join**. **Expected:** success state -> **Go to household** -> roster lists both members.
3. Negative: paste a garbage code. **Expected:** inline error, the form stays for a retry.

QA-DSK-09 — Language choice survives an app restart • Core (Desktop)

Gherkin

```
Given the desktop app on the login screen (signed out)
When I switch the language to Español and fully restart the app
Then it launches in Spanish
```

Walkthrough

1. Signed out, on the login screen: switch the language selector to **Español**. **Expected:** the UI re-renders in Spanish (no restart needed).
2. Fully close the app; relaunch. **Expected:** still Spanish — the choice is read from OS Preferences before first render (NATIVE-5). Switch back to English; same persistence.
3. Note: after sign-in, a **server-saved** locale wins (the account preference follows you across devices) — that's by design.

QA-DSK-10 — Data export downloads via the OS • Core (Desktop)

Gherkin

```
Given I am the owner, signed in on desktop
When I request the data export and click Download
Then the file is offered through the platform share/save UI, not a dead WebView navigation
```

Walkthrough

1. **Household** -> **Data** -> **Export my data** -> wait for the ready alert -> **Download**.
2. **Expected:** the **Windows share flyout** opens with the JSON bundle staged (server-named `…-<id>.json`); the app page is not navigated away. Save it and open — valid JSON, no secrets (spot-check: no token hashes).

QA-DSK-11 — Billing: checkout leaves, summary refreshes on return • Core (Desktop)

Gherkin

```
Given I am the owner on the desktop Billing page (Stripe test mode configured)
When I click Upgrade, complete checkout in the system browser, and return to the app
Then the plan summary refreshes by itself — no re-navigation needed
```

Walkthrough

1. **Billing** -> **Upgrade**. **Expected:** the **system browser** opens the hosted checkout; the app stays on Billing (it does not navigate away).
2. Complete checkout with a test card. **Expected:** the browser lands on the **web** app's billing page (by design — emailed/redirect links are web; `docs/NATIVE_PARITY.md` G2).
3. Click back into the desktop app. **Expected:** the summary **refetches on focus** — the plan flips to **pro** without touching navigation. (Without Stripe keys the fake provider's checkout URL is a stub — then verify only: browser opened, app stayed, and refocusing refetches.)

QA-DSK-12 — MFA: enroll and native step-up • Core (Desktop)

Gherkin

```
Given I am signed in on desktop without MFA
When I enable the authenticator in Settings, sign out, and sign in again
Then the app itself prompts for the 6-digit code before completing sign-in
```

Walkthrough

1. **Settings** -> **Two-factor** -> **Enable**. **Expected:** the QR renders inside the app (the vendored QR script ships in the native host too); manual key shown; confirm with a code; **recovery codes** displayed once.
2. Sign out -> OTP sign-in. **Expected:** after the OTP, the app shows the **in-app MFA prompt** (native step-up, MFA-4) — enter the authenticator code -> signed in. A recovery code also works (single-use).

QA-DSK-13 — Notification bell & preferences • Edge (Desktop)

Walkthrough: header **bell** -> **Expected:** opens with the empty state (or your items), badge matches unread count. **Settings** -> notification preferences -> toggle one off -> reload the page -> **Expected:** the switch state persisted. (For a real item end-to-end, a staff announcement — see QA-ADMIN-04 — lands in the native bell too; spot-check when staff is configured.)

QA-DSK-14 — Admin console on desktop (staff only) • Edge (Desktop)

Walkthrough: with your email in `Admin:StaffEmails`, sign in on desktop. **Expected: Admin** appears in the nav; the console lists tenants; opening a detail works. Impersonate a user -> **Expected:** banner appears; **Stop** restores your staff session in-app (native refresh path).

QA-DSK-15 — Theme: dark mode survives an app restart • Edge (Desktop)

Walkthrough: pick **Dark** in the header switcher -> **Expected:** the WebView restyles dark immediately. Quit the app fully and relaunch -> **Expected:** it boots dark with no light flash (the theme bootstrap reads the WebView's own localStorage pre-paint — verifies WebView storage persists across restarts on this platform). Pick **Auto** -> **Expected:** follows the OS light/dark setting live.

13. Android — MAUI • Core

> Prereq every run: **`adb reverse tcp:5438 tcp:5438`** + API on the https profile. See `docs/MOBILE_TESTING.md`.

QA-AND-01 — OTP sign-in • Smoke (Android) Automated in CI

Gherkin

```
Given the Android app is on the login screen with adb reverse set
When I request an OTP and enter the code from Mailpit
Then I am signed in
```

Walkthrough

1. Confirm `adb reverse --list` shows `tcp:5438`. Launch the app.
2. Login screen: email field + **Email me a 6-digit code**; Google/Microsoft buttons; **no magic link**.
3. Enter an email -> request code -> read it in Mailpit (host) -> enter it.
4. **Expected:** signed in, app shell shown.

QA-AND-02 — Google OAuth via custom scheme • Core (Android)

Gherkin

```
Given the Android login screen
When I tap Continue with Google and complete consent
Then the in-app browser returns to the app via the jiggerjot:// scheme, signed in
```

Walkthrough

1. Tap **Continue with Google**. **Expected:** a browser tab opens to Google consent.
2. Approve. **Expected:** the tab returns control to the app (`jiggerjot://auth` intent), now signed in. (Repeat **Microsoft** if its `:5438` redirect is registered.)

QA-AND-03 — "Remember me" across app restart • Core (Android)

Walkthrough: swipe-close the app; reopen. **Expected:** still signed in (refresh token in the Android Keystore). *(If `adb reverse` was lost on a device reboot, re-run it first — a startup failure after reboot is an environment issue, not an app bug.)*

QA-AND-04 — Household & Settings load • Core (Android)

Walkthrough: open **Household** and **Settings**. **Expected:** both load their data (the native Bearer path works), same as desktop QA-DSK-04.

QA-AND-05 — Navigation drawer / header is reachable • Edge (Android)

Gherkin

```
Given I am signed in on Android
When I open the navigation (hamburger)
Then the menu is tappable and not hidden under the status bar
```

Walkthrough: tap the hamburger; use **Household/Settings/Sign out**. **Expected:** the header sits below the status bar (safe-area padding) and every item is tappable.

QA-AND-06 — Core flows on Android • Edge (Android)

Walkthrough: spot-check language switch, invite (token revealed), and leave. **Expected:** parity with web.

> **NATIVE Wave 2 (ADR-018, `docs/NATIVE_PARITY.md`):** the cases below verify the parity fixes on Android — including the two that only exist on this platform (hardware back, share sheet).

QA-AND-07 — Hardware back navigates in-app history • Smoke (Android)

Gherkin

```
Given I am signed in on Android and have navigated Home -> Household -> Settings
When I press the hardware/gesture back button repeatedly
Then it walks back through the app's pages and only leaves the app at the root
```

Walkthrough

1. Navigate **Home -> Household -> Settings** (three distinct pages).

2. Press **back**. **Expected:** Settings -> Household. Again: -> Home.
3. Press **back at Home (root)**. **Expected:** the app backgrounds/exits — the default, but only at the root. *(Before NATIVE-4, any back press exited the app.)*
4. Note: a full WebView reload (e.g. the language switch) restarts the in-page history — back exiting right after one is acceptable.

QA-AND-08 — Join a household by pasted invite code • Core (Android)

Walkthrough: as QA-DSK-08, on Android: owner invites (web) -> member on Android: **Household** -> **Have an invite?** -> paste -> **Join** -> roster shows both. Garbage code -> inline error, retryable. *(This was parity gap G5 — a native member previously had no way to join at all.)*

QA-AND-09 — Language choice survives an app restart • Core (Android)

Walkthrough: as QA-DSK-09: signed out, switch to **Español** (re-renders) -> **swipe-close** the app -> relaunch. **Expected:** still Spanish (OS Preferences bootstrap, NATIVE-5). Signed-in accounts reconcile to their server-saved locale — by design. **Signed in (2026-09-16):** Settings -> Language -> **Español**. **Expected:** the app reloads once, in Spanish, and **stays** Spanish — before the fix it read the old language from the sign-in token, saved it back and reloaded again.

QA-AND-10 — Data export via the share sheet • Core (Android)

Walkthrough: owner -> **Household** -> **Data** -> **Export my data** -> **Download**. **Expected:** the **Android share sheet** opens with the JSON bundle (server-named); share to Files/Drive and open — valid JSON. The app page is not navigated away. *(Before NATIVE-3 this click did nothing — the WebView silently dropped it.)*

QA-AND-11 — Billing: checkout leaves, summary refreshes on return • Core (Android)

Walkthrough: as QA-DSK-11: **Billing** -> **Upgrade** -> **Expected:** the system browser/Custom Tab opens; complete checkout (test mode); switch back to the app (app switcher or back). **Expected:** the summary **refetches on resume** — plan shows **pro** without re-navigation (NATIVE-4; Android's return path is the Activity resume).

QA-AND-12 — MFA: enroll and native step-up • Core (Android)

Walkthrough: as QA-DSK-12 on Android — QR renders in-app (scan it with a second device or use the manual key), recovery codes shown once; after sign-out, OTP sign-in prompts the **in-app** MFA step-up (MFA-4).

QA-AND-13 — Edge-to-edge / status bar on Android 15 • Edge (Android)

Gherkin

```
Given a device or emulator on Android 15 (API 35, edge-to-edge enforced)
When I use the app in portrait and landscape
Then no control is hidden under the status bar or gesture areas
```

Walkthrough: check the header/hamburger (extends QA-AND-05), the bell dropdown, and a page with bottom-of-screen buttons (Settings danger zone) in both orientations. **Expected:** nothing sits under the status bar or the gesture-nav pill; everything tappable. *(Flagged [verify] by the parity audit — if this fails, it becomes a small safe-area fix slice.)* **Also, in both themes (2026-09-16):**

- The **status bar** is the header's colour — white with dark icons in Light, the dark surface with light icons in Dark — never the old sage green; on the sign-in screen it is the page's warm ground.
- **No empty band** between the status bar and the header.
- The **Shelf** payoff footer, **Write**'s Save bar and a **recipe**'s action bar sit flush on the tab bar, with no strip of page showing between them.
- **Cocktails:** the selected chip ("Everything · N" by default) is fully on screen; picking another chip brings that one into view.

QA-AND-14 — Theme: dark mode survives an app restart • Edge (Android)

Walkthrough: as QA-DSK-15 on Android — pick **Dark** (hamburger -> header controls), force-stop the app (or swipe it away) and relaunch. **Expected:** boots dark, no light flash (Android WebView localStorage persists). **Auto** follows the system dark theme toggle live. **Then (2026-09-16):** signed in, pick **Light** in Settings and immediately switch the language (which reloads the app). **Expected:** still Light after the reload — and the status bar follows each change.

QA-AND-15 — OAuth sign-in survives process death (NATIVE-12) • Core (Android)

Gherkin

```
Given I tapped Continue with Google and the consent tab is open
When Android kills the app process before I finish consent
Then approving still signs me in — the redirect relaunches the app and completes on startup
```

Walkthrough

1. Tap **Continue with Google**. **Expected:** the browser tab opens to Google consent.
2. With the tab in the foreground, kill the app process: `adb shell am kill com.jiggerjot.app` (works because the app is backgrounded behind the browser; *Don't keep activities* + memory pressure reproduces it the organic way).
3. Approve consent in the still-open tab. **Expected:** the `jiggerjot://auth` redirect cold-starts the app, which **stays open** and lands signed in on Home (the stashed code is exchanged during startup — no "flash open and close").
4. (MFA account) same steps. **Expected:** the app opens on the Login MFA code prompt; entering the TOTP completes sign-in.
5. (Staleness) repeat 1–2, wait > 5 min before approving. **Expected:** the app opens on Login with "Your sign-in took too long to complete. Please try again." — no crash, retry works.

13b. iOS + macCatalyst — first-run smoke • Core

> **These platforms compile in CI but had never been RUN before this pass.** Prereqs: a Mac with **Xcode 26.5** (the CI pin), the repo, and the API + Postgres + Mailpit running on that Mac (the iOS **simulator** shares the host network, so `https://localhost:7360` works; a **physical device** needs the API bound to a LAN address + the dev cert trusted). Launch: `dotnet build src/Maui -t:Run -f net10.0-ios` (simulator) / `-f net10.0-maccatalyst`. The **G7 fix is required** (PR #109) — before it, both platforms crashed at first resolve (no `IOAuthInitiator` registered). OAuth is wired via `ASWebAuthenticationSession` + the `jiggerjot` scheme in Info.plist; first exercised **and passed** in the 2026-07-06 §13b run (PR #125 fixed the two gaps it found) — QA-IOS-04 re-verifies it each pass.

QA-IOS-01 — App boots to the login screen • Smoke (iOS)

Walkthrough: launch on the simulator. **Expected:** the app opens (no startup crash — this *was* parity gap G7), the login screen renders: email + **Email me a 6-digit code**, provider buttons, **no magic-link button**, nothing under the notch/safe areas.

QA-IOS-02 — OTP sign-in • Smoke (iOS)

Walkthrough: request a code -> read it from Mailpit (on the Mac) -> enter it. **Expected:** signed in, app shell + household load (the native Bearer/body-token path works on iOS).

QA-IOS-03 — Core flows spot-check • Core (iOS)

Walkthrough: spot-check on the simulator: **Household** roster + invite (token revealed) + **join-by-code**, **Settings** (providers list, MFA card renders its QR), **language switch** -> re-render + relaunch persistence, **bell** empty state, **export** -> iOS share sheet. **Expected:** parity with Android (§13) — same shared RCL.

QA-IOS-04 — OAuth via ASWebAuthenticationSession • Core (iOS)

Walkthrough: Continue with Google. **Expected:** the system auth session sheet opens to Google consent; approving returns to the app via the `jiggerjot://auth` scheme, signed in. Also verify session-across-restart (Keychain-backed secure storage).

QA-MAC-01 — App launches at a usable window • Smoke (macCatalyst)

Walkthrough: launch the macCatalyst build. **Expected:** no startup crash (G7); the window opens at a sensible default size and is resizable; login renders. *(The audit's [verify] desktop-window-sizing cell for macOS.)*

QA-MAC-02 — OTP sign-in • Smoke (macCatalyst)

Walkthrough: as QA-IOS-02. **Expected:** signed in; Household + Settings load.

QA-MAC-03 — Core flows + OAuth spot-check • Core (macCatalyst)

Walkthrough: as QA-IOS-03 (share = macOS share menu) + one OAuth round-trip (QA-IOS-04 flow) + restart persistence. **Expected:** parity.

13c. Native release checklist

Per release that ships a native client, run the platform's • Smoke cases plus **one** • Core feature spot-check, and record results in §16:

Platform	Always (• Smoke)	Plus one of (• Core)
Windows desktop	DSK-01, DSK-03, DSK-06	DSK-08..12
Android	AND-01, AND-03, AND-07	AND-08..15
iOS	IOS-01, IOS-02	IOS-03, IOS-04
macCatalyst	MAC-01, MAC-02	MAC-03

Full per-feature native regression (every case in §12–13b) is for releases that changed native glue (`src/Maui/**`, the RCL seams: `IculturePersistence` / `IFileDownloadLauncher` / `AppResumeNotifier`) or bumped the .NET/MAUI toolchain.

> **Two things to know before handing a tester an Android build (v4 T51).** (1) Android upgrades an app in place only when the new build carries the **same signing key** as the installed one: a store-signed build will not install over a debug-signed sideload, nor the reverse — the tester uninstalls the old one first (DEPLOYMENT §9). (2) Nothing the app stores is backed up off the device (`allowBackup="false"` + data-extraction rules), so after an uninstall or on a new phone the tester signs in again and re-picks language and theme; the account and its data are on the server.
>
> **(!) The native legs run ON REQUEST, never per push (ADR-031).** A pull request runs the web gates only. The MAUI builds and the Windows / Android / Apple smokes run from GitHub -> Actions -> `CI` -> *Run workflow* -> `devices` (android / windows / apple / all) — macOS bills at 10× and Windows at 2× on a private repo.
>
> **What that means for a release:** a green pull request says nothing about the native apps. Before shipping a native client, run the device legs for that platform (or all) on the commit you ship, or run the iOS/macOS cases in §13b on a Mac.

14. Cross-cutting security • Core

QA-SEC-01 — Tenant isolation • Smoke

Gherkin

```
Given two households owned by two different users
When user A is signed in
Then A can only ever see A's household, members, and invitations — never B's
```

Walkthrough

1. Create household A (owner A) and a separate household B (owner B, different account/incognito).
2. As A, open **Household**. **Expected:** only A's members/invites. Confirm B's data never appears. *(Deeper API-level probing — e.g. requesting another tenant's resource id — belongs in automated `Api.Tests`; this manual case verifies the UI never cross-contaminates.)*

QA-SEC-02 — Protected pages require auth • Core

Walkthrough: signed out, directly visit `/household`, `/settings`. **Expected:** each redirects to `/login`.

QA-SEC-03 — Session is gone after sign-out • Core Automated in CI

Gherkin

```
Given I sign out
When I press the browser Back button or reload a protected page
Then I am not able to access it — I am sent to /login
```

Walkthrough: sign out, press **Back** to a protected page / reload it. **Expected:** bounced to `/login`; no stale authenticated view.

Automated by `SessionJourneyTests.SignOut_ThenBack_LandsOnLogin_NotTheCachedHousehold`, which runs with the browser's back/forward cache on and fails if the page was not actually restored from it. The trade-off, by decision: a page restored from that cache is reloaded (`bfcache-guard.js`), so Back after sign-out costs a signed-in user one extra page load on any Back that hits the cache, in exchange for never showing the previous user's screen on a shared computer.

QA-SEC-04 — Native open-redirect guard • Edge (Desktop/Android)

Context/Expected: the native OAuth flow only honors loopback `http` callbacks or the configured `jiggerjot://` scheme; arbitrary redirect targets are rejected. This is unit-tested (`NativeRedirectPolicyTests`); no manual action needed unless probing the API directly — record as **covered by automated tests**.

QA-SEC-05 — Server-side hardening (automated) • Edge

Context/Expected: several invariants are enforced at the API/data layer and verified by `tests/Api.Tests`, not by manual UI steps — record as **covered by automated tests** unless probing the API directly:

- **Tenant write-stamping** — a new tenant-scoped row is stamped with the caller's tenant and a foreign-tenant write is rejected (`TenantStampingInterceptorTests`), so reads *and* writes are tenant-isolated.
- **Refresh-token reuse detection** — replaying an already-rotated refresh token revokes all the user's sessions (`RefreshTokenServiceTests`, `RefreshReplayTests`) — unless it arrives within 60 s of its rotation while its successor is still live (two tabs refreshing at once, a lost response): that race gets a fresh session and revokes nothing (ADR-002 addendum, 2026-09-18) — once per token: a third presentation is reuse (2026-09-28). Logout revokes the family of any known token, expired or just rotated out included (`RefreshReplayTests.Logout_With*`). Manually observable only by capturing and replaying a refresh cookie/token; out of scope for routine QA.
- **Unverified-email takeover guard** fails closed (`ClaimsExtractorTests` / `UserServiceTests`).
- **Legacy refresh-cookie self-heal** — a stale `Path=/` refresh cookie left by an older build can shadow the live `Path=/api/auth` cookie and wedge sign-in into a `/refresh` 401 -> "Authentication Failed" loop (seen in Firefox, due to cookie send-order). Setting the refresh cookie now also emits an expiry for the orphan, so the next successful sign-in / refresh sweeps it automatically — no manual cookie-clearing, and upgraded deployments self-heal (`CookieServiceTests`). *Manual repro needs a planted `Path=/` cookie; record as covered by automated tests. If a tester hits a stuck `/refresh` 401 loop in Firefox after a deploy, a single re-sign-in clears it.*

14a. Adversarial & tenant-isolation (QA-ADV-*) • Core

> This suite probes the failure classes a **v3 audit** surfaced: cross-tenant reads/writes at the **API** layer (not just the UI), impersonation attribution and confinement, billing idempotency, session/token lifecycle, preference bleed on shared devices, and deploy/native hardening. Most cases are **curl/Postman** driven with **two tenants A and B** — mint an **owner JWT for each** (sign in as each owner and copy its access token from `POST /api/auth/refresh` or the Swagger **Authorize** button, exactly as §14b describes) so you hold **two JWTs carrying different `tenant_id` claims**. A few cases need the **two browser contexts** of §1.2 (a normal window + an incognito/second profile); those are flagged in the title. Base URL `https://localhost:7360` (use `-k` for the dev cert) locally, or the staging host for Environment B. Grab each tenant's ids up front: `GET /api/household` as A and as B gives you A's/B's household id + member user-ids; note one **B** member user-id, one **B** notification/webhook/api-key id (create them if needed) for the cross-tenant probes.
>
> **(!) EXPECT-FAIL discipline.** Roughly half of these assert behaviour the v3 audit says is **broken at authoring time (2026-07); ALL are fixed as of the v3 remediation (PRs #147–#191)**. Each formerly-broken case carries a ■ v3-landed note naming the finding. When it fails, record **Blocked (known defect)** against the id in §16 — **never Pass**. A QA plan that green-ticks a broken isolation path is worse than no case at all. The remaining cases should **Pass on current code**; a failure there is a real regression.

QA-ADV-01 — Tenant A cannot read/export/erase Tenant B via the API • Smoke (curl)

Gherkin

```
Given I hold owner JWTs for two separate tenants A and B
When I use A's JWT to target B-scoped resource ids directly
Then every B id is refused (404/403), A's export contains only A, and no B row mutates
```

Walkthrough

1. With **A's JWT**, call `GET /api/household` -> **200** showing **only A** (never B's name/members).
2. `POST /api/household/export` with A's JWT -> the JSON bundle enumerates **only A's** tenant, members and invitations — grep it for B's household name/emails: **absent**.
3. Target B directly with A's JWT: `DELETE /api/household/members/{B-userId}`, `POST /api/notifications/{B-notificationId}/read`, `DELETE /api/notifications/{B-notificationId}`.
4. **Expected:** each B-scoped call returns **404** (or 403) — the global tenant filter + RLS backstop hide B entirely; ids from another tenant simply don't exist for A.
5. Re-verify as **B** (`GET /api/household`, bell): B's member list and notifications are **unchanged**. This promotes QA-SEC-01 from a UI-only check to a direct API probe (audit **RLS-2/4**).

QA-ADV-02 — Cross-tenant isolation on the newer tables (PUBAPI/HOOKS on) • Core (curl)

Precondition: set `PublicApi__Enabled=true` + `Webhooks__Enabled=true`, restart (§14b). In **each** tenant mint an API key (QA-API-02) and register a webhook + generate one delivery (QA-API-05/06). **Gherkin**

```
Given tenants A and B each own an API key, a webhook subscription and a delivery row
When A's JWT targets B's api-key / webhook / delivery ids
Then each is 404 and nothing cross-tenant is read, replayed or revoked
```

Walkthrough

1. With **A's JWT**: `GET /api/webhooks/{B-subscriptionId}/deliveries`, `POST /api/webhooks/deliveries/{B-deliveryId}/replay`, `DELETE /api/apikeys/{B-keyId}`.
2. **Expected: 404** for each — the delivery log/read side filters by `TenantId` and management routes are tenant-scoped, so B's rows are invisible and untouchable to A.
3. Confirm as B: B's key still authenticates, its subscription/deliveries are intact, nothing replayed (audit **LB-TEN-2 / RLS-8**).

QA-ADV-03 — Export includes EVERY tenant-scoped table, secret-free • Core (curl)

■ v3 REMEDIATION LANDED (2026-07, PRs #147–#191) — this case now expects PASS; re-run and record normally. Gherkin

```
Given PUBAPI + HOOKS are enabled and my tenant has an API key, a webhook and metered usage
When I export the household
Then the bundle covers api-keys (metadata only), webhook subscriptions (no secret) and usage counters
— or explicitly notes their exclusion
```

Walkthrough

1. Enable both flags; as owner mint an API key, register a webhook, and drive **one** metered `IQuotaService.TryConsumeAsync` consume (any endpoint an app has wired, or seed a `UsageCounter`).
2. `POST /api/household/export`; open the JSON.
3. **Expected (post-remediation):** it contains an **api-keys** section (name/prefix/scopes, **no hash**), a **webhook-subscriptions** section (url/event-types, **no signing secret**), and the **usage counters** — or an explicit documented exclusion note for each.
4. **Was (pre-v3 audit LB-TEN-1):** this step used to FAIL and was recorded Blocked. The finding is fixed (v3 remediation, PRs #147–#191) — the assertions above now hold; expect **Pass**. Also assert **no secret leaks** (hashes / `whsec_…`) regardless — the export stays secret-free.

QA-ADV-04 — Dissolve / erasure actually deletes api keys, webhook secrets, usage counters, delivery logs • Core (curl + DB)

■ v3 REMEDIATION LANDED (2026-07, PRs #147–#191) — this case now expects PASS; re-run and record normally. Gherkin

```
Given a tenant with an API key, a webhook subscription (+ deliveries) and usage counters
When the sole owner dissolves the household (or deletes the account)
Then all those rows are physically gone — no orphaned keys, secrets, counters or delivery logs remain
```

Walkthrough

1. Set up a throwaway tenant with the same artefacts as QA-ADV-03.
2. As sole owner, dissolve (QA-HH-07) / delete account (QA-SET-07).
3. Query the DB (or the staff console) for that tenant's `ApiKey`, `WebhookSubscription`, `WebhookDelivery`, `UsageCounter` rows.
4. **Expected (post-remediation): 0 rows** for the dissolved tenant — every contributor cleaned up.
5. **Was (pre-v3 audit LB-TEN-1):** this step used to FAIL and was recorded Blocked. The finding is fixed (v3 remediation, PRs #147–#191) — the assertions above now hold; expect **Pass**.

QA-ADV-05 — Writes during impersonation are attributed to the acting staff • Core (curl)

■ v3 REMEDIATION LANDED (2026-07, PRs #147–#191) — this case now expects PASS; re-run and record normally. Gherkin

```
Given a staff user is impersonating the household owner (token carries impersonated_by)
When they perform audited tenant writes (export the household, change a member's role)
Then each mutation's audit row records impersonated_by=<staff>, not just the target
```

> **Pick writes that are actually audited.** The audited-action catalog is small — `tenant.exported`, `member.role_changed`, `account.erased`, and the staff-only `admin.*` actions. Household rename (`PUT /api/household`) and mark-notification-read (`POST /api/notifications/{id}/read`) emit **no** `AuditEvent`, so they produce nothing to inspect; the two owner-gated writes below do. The stamp itself is ambient (`AuditLog` reads `ICurrentImpersonation`), so it lands on **every** audited event regardless of call site — the point is to exercise one that exists.

Walkthrough

1. As staff, `POST /api/admin/impersonate/{ownerUserId}` -> short-lived token with `impersonated_by`. Pick the household **owner** so the writes below are permitted; the tenant must also have **at least one other member** for the role change.
2. With that token, perform two **audited** tenant writes: `POST /api/household/export` (-> `tenant.exported`) and `PUT /api/household/members/{memberUserId}/role` (flip admin<->member -> `member.role_changed`).
3. Inspect the tenant's audit trail (staff-console audit view, or query the `AuditEvent` rows in the DB).

4. **Expected (post-remediation):** the `tenant.exported` **and** `member.role_changed` rows each carry **`impersonated_by=<staff-id>`** (with `actor_user_id=<owner>`), so an operator can tell staff-driven changes from the user's own — the stamp is applied to every event, not per call site.
5. **Was (pre-v3 audit LB-ADM-1):** this step used to FAIL and was recorded Blocked. The finding is fixed (v3 remediation, PRs #147–#191) — the assertions above now hold; expect **Pass**.

QA-ADV-06 — An impersonation session cannot reach staff-only actions • Core (curl)

■ v3 REMEDIATION LANDED (2026-07, PRs #147–#191) — this case now expects PASS; re-run and record normally. Gherkin

```
Given I hold an impersonation token (impersonated_by present, target is a normal member)
When I call staff-only admin endpoints with it
Then every one is refused at the staff gate — impersonation must not be a ladder back to staff power
```

Walkthrough

1. Impersonate a **plain member** (`POST /api/admin/impersonate/{memberUserId}`, as in QA-ADV-05 step 1 but targeting a non-owner) -> token with `impersonated_by`.
2. With it, attempt: `POST /api/admin/impersonate/{x}`, `PUT /api/admin/tenants/{id}/subscription`, `DELETE /api/admin/users/{x}/mfa`, `POST /api/admin/announce-all`.
3. **Expected (post-remediation): 403** on each — the staff gate treats a token bearing `impersonated_by` as **non-staff**, closing privilege re-escalation.
4. **Was (pre-v3 audit ADM-2):** this step used to FAIL and was recorded Blocked. The finding is fixed (v3 remediation, PRs #147–#191) — the assertions above now hold; expect **Pass**.

QA-ADV-07 — Impersonation cannot exceed the target's role • Core (curl)

Gherkin

```
Given I am impersonating a plain member (not owner)
When I attempt owner-only actions as that member
Then I get exactly the 403 the member themselves would get
```

Walkthrough

1. Impersonate a **plain member** of a tenant that has an owner + other members.
2. Attempt owner-only writes: `POST /api/household/transfer-ownership`, `PUT /api/household/members/{id}/role`, and (as staff-would-be) a comp — via the member token.
3. **Expected: 403** on each — impersonation inherits the **target's** role ceiling; a member seat confers no owner power (audit **TB-ADM-3**). Should **Pass** on current code.

QA-ADV-08 — Impersonation never rewrites target prefs; shared-device pref poisoning • Core (curl + Web — two contexts)

■ v3 REMEDIATION LANDED (2026-07, PRs #147–#191) — this case now expects PASS; re-run and record normally. Gherkin

```
Given I impersonate a user and change theme/language in their session
Then the target's server-stored preference is unchanged after I Stop impersonating
And on a shared browser, a fresh sign-in never inherits the previous user's saved preference
```

Walkthrough

1. **Leg A (should Pass):** impersonate a user, change theme/language in-session, **Stop impersonating**. Re-read the target's server pref (`GET /api/auth/me` / their Settings -> Preferences). **Expected: unchanged** — preference sync is suppressed while `impersonated_by` is set (PREFS-1/ADR-022).
2. **Leg B (EXPECT-FAIL):** in **context 1**, sign in as user A whose saved locale is **Español**; sign out. In the **same** browser, sign in as user B who has **never chosen** a language/theme.
3. **Expected (post-remediation):** B renders in B's own default (server never-set => B does not adopt A's leftover device value while a *different* account signs in).
4. **Was (pre-v3 audit LB-UI-4/5):** this step used to FAIL and was recorded Blocked. The finding is fixed (v3 remediation, PRs #147–#191) — the assertions above now hold; expect **Pass**.

QA-ADV-09 — A staff MFA-reset notification cannot be silenced by target prefs • Core (curl)

■ v3 REMEDIATION LANDED (2026-07, PRs #147–#191) — this case now expects PASS; re-run and record normally. Gherkin

```
Given a user has turned BOTH notification channels (in-app + email) off
When staff reset that user's MFA
Then the security.mfa_reset in-app row AND the email are still delivered (security events ignore
prefs)
```

Walkthrough

1. As the target, `PUT /api/notifications/preferences` with **in-app=false, email=false**.
2. As staff, `DELETE /api/admin/users/{targetUserId}/mfa`.
3. **Expected (post-remediation):** the target still gets the **`security.mfa_reset` bell row** and** the email — a security-critical notice is not suppressible by user prefs.
4. **Was (pre-v3 audit ADM-1):** this step used to FAIL and was recorded Blocked. The finding is fixed (v3 remediation, PRs #147–#191) — the assertions above now hold; expect **Pass**.

QA-ADV-10 — MFA step-up locks out after repeated wrong codes • Core (curl)

■ v3 REMEDIATION LANDED (2026-07, PRs #147–#191) — this case now expects PASS; re-run and record normally. Gherkin

```
Given an MFA-enabled account at the sign-in step-up
When I submit wrong 6-digit codes past the attempt cap, including across freshly-requested challenges
Then a per-user cumulative MFA lockout engages — not merely the per-IP 429
```

Walkthrough

1. Trigger step-up (sign in as an MFA-on user); `POST /api/auth/mfa/verify` with wrong codes.
2. Keep going **past** the OTP-style attempt cap, and request **new** challenges to reset any per-IP window, pacing under the 429 verify throttle.
3. **Expected (post-remediation):** after N cumulative failures the **account's** step-up is locked (distinct from QA-AUTH-11's per-IP 429), mirroring the OTP cumulative lockout (QA-AUTH-04).
4. **Was (pre-v3 audit ADM-3):** this step used to FAIL and was recorded Blocked. The finding is fixed (v3 remediation, PRs #147–#191) — the assertions above now hold; expect **Pass**.

QA-ADV-11 — A recovery code is single-use • Core (Web)

Gherkin

```
Given I used one MFA recovery code to complete step-up
When I sign out and submit the SAME recovery code again
Then it is rejected — recovery codes are consumed on first use
```

Walkthrough

1. On an MFA-on account, at step-up enter a **recovery code** (not the TOTP) -> signed in.
2. Sign out; start a fresh sign-in to the same account; at step-up submit the **same** recovery code.
3. **Expected:** rejected ("that code is incorrect or has expired"); the code was hashed + burned on first use (audit **TB-AUTH-6**). A different, unused code still works. Should **Pass**.

QA-ADV-12 — Webhook replay + out-of-order + same-second idempotency • Core (curl)

Precondition: billing wired to the fake provider; HOOKS not required (this exercises the billing webhook inbox).
Gherkin

```
Given the billing webhook endpoint
When I POST the same EventId twice, then two distinct same-second events (created then active)
Then the duplicate is a no-op (inbox dedup) and the later state (active) still applies — not dropped
as stale
```

Walkthrough

1. POST a provider webhook to `/api/billing/webhook`; **repeat the exact same `EventId`**.

2. **Expected:** the second POST is a **no-op** — the inbox dedups on event id; the subscription state is applied exactly once.
3. POST two **distinct** events with the **same whole-second** `OccurredAt`: first `created`, then `active`.
4. **Expected:** the flip to **`active` is applied (the strictly-older recency guard drops stale events; same-second ties apply in arrival order (exact redeliveries are inbox-deduped), not by a second-granular timestamp that would wrongly drop it as stale) (audit** LB-BILL-1**). Should** Pass**.

QA-ADV-13 — A first-ever webhook in a bad state does not false-notify • Core (curl)

■ v3 REMEDIATION LANDED (2026-07, PRs #147–#191) — this case now expects PASS; re-run and record normally. Gherkin

```
Given a tenant that has NEVER had a live subscription
When its first-ever billing webhook arrives as past_due (or canceled)
Then NO dunning notification is sent to the owner (there was nothing to lapse)
```

Walkthrough

1. Pick a tenant with no subscription history. POST a **first** webhook with `Status=past_due` (or `canceled`) to `/api/billing/webhook`.
2. **Expected (post-remediation):** the owner receives **no** billing/dunning notification — a dunning notice requires a transition **out of** an active/paid state.
3. **Was (pre-v3 audit LB-BILL-4):** this step used to FAIL and was recorded Blocked. The finding is fixed (v3 remediation, PRs #147–#191) — the assertions above now hold; expect **Pass**.

QA-ADV-14 — Comp / revert a churned (canceled) tenant • Core (curl)

■ v3 REMEDIATION LANDED (2026-07, PRs #147–#191) — this case now expects PASS; re-run and record normally. Gherkin

```
Given a tenant whose Stripe subscription is canceled (its subscription id still persisted)
When staff try to comp it to Pro (or revert)
Then the comp should succeed — a churned tenant is not permanently un-comp-able
```

Walkthrough

1. Take a tenant whose provider sub is **canceled** but whose `Subscription` row still holds the Stripe id. As staff, `PUT /api/admin/tenants/{id}/subscription` (comp to Pro).
2. **Expected (post-remediation):** comp **succeeds** — a canceled/churned sub is treated as not-provider-managed for comp purposes.
3. **Was (pre-v3 audit ADM-5):** this step used to FAIL and was recorded Blocked. The finding is fixed (v3 remediation, PRs #147–#191) — the assertions above now hold; expect **Pass**. (The guard now keys on *liveness*: 409 only for a live provider sub — see QA-ADMIN-06, where 409 on a live Stripe sub is correct.)

QA-ADV-15 — Concurrent acceptance of the last seat • Core (Web — two contexts)

Gherkin

```
Given a tenant at seat cap minus one with two pending invitations
When both invitees accept near-simultaneously
Then exactly one joins, the other gets 402 seat_limit_reached, and seats never exceed the cap
```

Walkthrough

1. Bring a Free tenant to **one seat below its cap** with **two** pending invitations outstanding (the cap is the M in the Billing page's "N of M seats"). A pending invitation reserves a seat, so a Free tenant cannot be put in that state directly: comp it to Pro from the staff console, add members until it has **cap** − **1**, send the two invitations, then revert it to Free — the downgrade setup QA-INV-10 uses.
2. In **two browser contexts**, sign in as each invitee and open both `/join` links; click **accept** as close to simultaneously as you can (or double-submit).
3. **Expected: one** join succeeds (the last seat), the other returns **402 `seat_limit_reached` with the "household is full" state; the member count settles at** exactly the cap **— the atomic seat check has no race (audit** TB-BILL-19 / BILLING-9**). Should** Pass**.

QA-ADV-16 — Magic-link / OTP double-redemption issues exactly one session • Core (Web — two contexts)

Gherkin

```
Given a single magic link or a single OTP code
When I redeem it from two clients at once (open the link in two tabs / submit the OTP twice)
Then exactly one sign-in succeeds and the second attempt is rejected
```

Walkthrough

1. Request a magic link; open the **same** link in two tabs nearly simultaneously (or double-click). Alternatively, request one OTP and `POST /api/auth/otp/verify` the same code from two clients at once.
2. **Expected: one** session is issued; the second redemption is rejected (token consumed single-use, no double-spend under concurrency) (audit **LB-AUTH-3**). Should **Pass**.

QA-ADV-17 — Replaying a rotated refresh token revokes all sessions • Core (curl)

Gherkin

```
Given I captured a refresh token, then refreshed once (rotating it)
When I replay the OLD (now-rotated) refresh token more than 60 seconds after that rotation
Then it is rejected AND all of the user's sessions are revoked
But a replay WITHIN 60 seconds, while the new token is still live, is a race and gets a fresh session
```

Walkthrough

1. Sign in; capture the refresh cookie/token. `POST /api/auth/refresh` once -> a **new** token (the old one is now rotated out).
2. **Grace window (ADR-002 addendum, 2026-09-18):** replay the **old** token within 60 s -> **200** with a fresh token, and nothing is revoked (the new token from step 1 still refreshes). This is the two-tab / lost-response race, deliberately not treated as theft (`RefreshToken:ReuseGraceSeconds`, default 60). **The grace is one-shot (ADR-002 addendum, 2026-09-28):** replay the same old token a **third** time, still within the 60 s -> **401**, and every session is revoked (both the step-1 token and the step-2 token now fail). Sign in again before step 3. Should **Pass**.
3. Wait **more than 60 s** after step 1, then replay the **old** token to `POST /api/auth/refresh`.
4. **Expected: 401** — reuse detected — **and** the whole token family is revoked: a legitimate silent refresh from another live session now **fails** too (forces re-auth). Promotes QA-SEC-05's automated `RefreshTokenServiceTests` note to a manual probe. Should **Pass**.
5. **Logout stays final:** sign in again, refresh once (rotating A -> B), `POST /api/auth/logout` with B, then replay A **within** 60 s -> **401**, no new session (`RefreshReplayTests`). Should **Pass**.

QA-ADV-18 — Spanish account on an English device accepts an invite • Smoke (Web — two contexts) Automated in CI

■ v3 REMEDIATION LANDED (2026-07, PRs #147–#191) — this case now expects PASS; re-run and record normally. Gherkin

```
Given a user whose saved locale is Español, on an English-default browser
When they open a valid /join?token=… (signed in, or signing in through it)
Then the locale-mismatch reload preserves the /join deep-link and the invite is accepted
```

Walkthrough

1. User's server locale = **Español**; use a **fresh English-default** browser profile.
2. Open a valid `/join?token=…` while signed in (or sign in through the join flow).
3. **Expected (post-remediation):** the one-time locale-mismatch reload (WASM satellite assemblies, PREFS-1) **preserves** the `/join?token=…` URL and the invite is **accepted**.
4. **Was (pre-v3 audit UX-1, LB-UI-1/2):** this step used to FAIL and was recorded Blocked. The finding is fixed (v3 remediation, PRs #147–#191) — the assertions above now hold; expect **Pass**.

QA-ADV-19 — Theme/locale doesn't revert after a soft (OTP/MFA) sign-in • Core (Web)

Gherkin

```
Given on the login page I pick Dark + Español, for a user whose server prefs differ
When I complete an OTP soft-navigation sign-in
Then there is no flip-back-then-correct flicker; the switcher reflects the applied value all session
```

Walkthrough

1. On `/login`, set **Dark + Español**. Complete an **OTP** sign-in (soft nav, no full reload) for a user whose stored prefs are e.g. Light/English.
2. **Expected:** the reconcile settles the applied theme/locale **once**, cleanly — no visible revert-then-reapply flash — and the header switcher shows the settled value for the rest of the session (PREFS-1 reconcile on **every** sign-in path, audit **UX-3/4**). Should **Pass**.

QA-ADV-20 — "Clear read" never deletes unread notifications • Core (Web)

Gherkin

```
Given my bell holds a mix of read and unread notifications
When I use "Clear read"
Then only the read ones are removed; every unread notification survives
```

Walkthrough

1. Seed a mix (mark some read, leave some unread). In the bell, click **Clear read**.
2. **Expected:** the client sends `DELETE /api/notifications?read=true` — **read-only** clear; unread rows remain. Confirm the bulk clear **requires an explicit scope**: `?read=true` clears read only, `?read=false` is the distinct **Clear all**, and an omitted scope returns **400 `scope_required` — so a dropped param can never silently wipe unread (audit** LB-UI-10**). Should** Pass**.

QA-ADV-21 — Security headers on the single-origin host (staging / Environment B) • Core (curl)

■ v3 REMEDIATION LANDED (2026-07, PRs #147–#191) — this case now expects PASS; re-run and record normally. Gherkin

```
Given the deployed single-origin host serving API + WASM
When I inspect the response headers for the SPA shell and framework assets
Then HSTS, nosniff, a CSP/frame-ancestors and Referrer-Policy are present, with correct
cache-control
```

Walkthrough

1. `curl -I https://<app>-staging.onrender.com/` and `curl -I https://<app>-staging.onrender.com/_framework/blazor.webassembly.js`.
2. **Expected (post-remediation):** `Strict-Transport-Security`, `X-Content-Type-Options: nosniff`, a `Content-Security-Policy` (or `frame-ancestors`), and `Referrer-Policy`; **index.html** served `no-cache`, `_framework/*` fingerprinted assets `immutable`.
3. **Was (pre-v3 audit DEP-2/3):** this step used to FAIL and was recorded Blocked. The finding is fixed (v3 remediation, PRs #147–#191) — the assertions above now hold; expect **Pass**.

QA-ADV-22 — Forged X-Forwarded-For cannot bypass the per-IP rate limit • Core (curl)

Gherkin

```
Given the deploy is reachable other than via its trusted proxy (test the assumption)
When I send OTP requests with a rotating X-Forwarded-For header
Then the per-IP limit + OTP/MFA lockout is NOT bypassed
```

Walkthrough

1. Hammer `POST /api/auth/otp/send` past the 5/min budget, rotating a spoofed `X-Forwarded-For` each request.
2. **Expected:** the throttle still trips **429** — the app only honours forwarded headers from the **configured trusted proxy** (DEPLOY-1's config-gated forwarded-headers), so a client-supplied XFF is ignored for rate-limit partitioning.

3. If the deploy is reachable **only** through its single trusted ingress, spoofing is moot — **document that sole-ingress assumption** in the run notes (audit **DEP-1 / ADM-10**). Should **Pass** (or be recorded N-A with the documented assumption).

QA-ADV-23 — A Release native build points at a real HTTPS base URL, not cleartext localhost • Core (Desktop/Android)

■ v3 REMEDIATION LANDED (2026-07, PRs #147–#191) — this case now expects PASS; re-run and record normally. Gherkin

```
Given a Release (non-dev) native build with no dev overrides
When I inspect its configured API base URL and network security config
Then it targets the configured HTTPS API (build fails if none) — no http://localhost cleartext ships
```

Walkthrough

1. Build a **Release** AAB/MSIX without dev overrides. Inspect the effective base URL + Android network security config.
2. **Expected (post-remediation):** base URL is the configured **HTTPS** API; a missing base URL **fails the build**; no `http://localhost:5438` and no cleartext-permitting network config is shipped.
3. **Was (pre-v3 audit NAT-3):** this step used to FAIL and was recorded Blocked. The finding is fixed (v3 remediation, PRs #147–#191) — the assertions above now hold; expect **Pass**.

QA-ADV-24 — Windows loopback OAuth binds state (login-CSRF guard) • Edge (Desktop)

Gherkin

```
Given the desktop loopback OAuth callback listener with a pending state
When a callback arrives whose state does not match the pending listener
Then it is rejected — no session is minted from an unsolicited/forged callback
```

Walkthrough

1. Start a desktop OAuth sign-in (loopback listener opens). Simulate a callback to the loopback URL with a **mismatched/blank `state`**.
2. **Expected:** rejected — the client only completes the exchange when `state` matches the value it generated for the pending listener (login-CSRF / session-fixation guard), consistent with the native redirect policy (audit **NAT-10**). Should **Pass** (unit-backed; manual probe optional).

14b. API surfaces — PUBAPI + HOOKS (config-gated; curl / Postman) • Core

> These two surfaces are **off by default** and have **no web UI** by design — they're for machines, so they're QA'd with an HTTP client. Any client works; the steps use `curl`. **Postman:** the repo ships a **complete, chained collection for the entire API** (all surfaces, not just these two) in `docs/postman/`, and CI mirrors it into the **Postman workspace** (`postman-sync` — open the workspace and pick an environment: local dev or staging/Render; importing the files by hand also works). It signs in via OTP (auto-fetching the code from Mailpit on local) and stores/rotates tokens per request. Alternatively, import `GET /api/public/openapi.json` (once PUBAPI is on) for just the public routes.
>
> **Preconditions (do once):** in the repo-root `.env` set `PublicApi__Enabled=true` and `Webhooks__Enabled=true`, then restart the API. Management of keys/webhooks is **owner-only**, so sign in as an owner and grab a JWT access token from `POST /api/auth/refresh` (the Swagger "Authorize" button shows one) to call the `/api/apikeys` and `/api/webhooks` management routes below. Base URL in these steps is `https://localhost:7360` (use `-k` for the dev cert).

QA-API-01 — Config gate: surfaces are 404 when disabled • Edge (curl)

Gherkin

```
Given PublicApi:Enabled and Webhooks:Enabled are false (the default)
When I call any /api/public, /api/apikeys or /api/webhooks route
Then it does not exist (404) — the routes aren't mapped and the API-key scheme isn't added
```

Walkthrough

1. With both flags **unset/false**, restart the API and call `curl -k https://localhost:7360/api/public/openapi.json` and `.../api/apikeys` (with a JWT).
2. **Expected: 404** for both. Now set the two flags true, restart, and re-check — they become live (401/200). Leave them **on** for the rest of this section.

QA-API-02 — Mint an API key (shown once) • Core (curl)

Gherkin

```
Given I am the owner with PUBAPI enabled
When I create an API key
Then I receive the raw pk_… key exactly once, and listing later shows only its prefix/metadata
```

Walkthrough

1. `curl -k -X POST https://localhost:7360/api/apikeys -H "Authorization: Bearer <JWT>" -H "Content-Type: application/json" -d '{"name":"qa","scopes":["read"]}'`
2. **Expected: 201** with a `key` field like `pk_…` — **copy it now** (never shown again). `GET /api/apikeys` lists it with `prefix`/`scopes` but **no** `key`.
3. **Non-owner:** repeat as a member/admin -> **403**.

QA-API-03 — Call the public API with the key; scopes + tenant-scoping • Core (curl / Postman)

Gherkin

```
Given a read-only API key for my tenant
When I call the public API with it
Then whoami returns my tenant, and a write-scoped route is refused (403 insufficient_scope)
```

Walkthrough

1. `curl -k https://localhost:7360/api/public/whoami -H "X-Api-Key: pk_…"` -> **200**, body shows my `tenant_id`, the key name, and `["read"]`.
2. `curl -k -X POST https://localhost:7360/api/public/echo -H "X-Api-Key: pk_…" -d '{"message":"hi"}'` with the **read-only** key -> **403 `insufficient_scope`**. (A key created with `"write"` succeeds.)
3. **No/blank/garbage key** -> **401**. **Revoked key** (`DELETE /api/apikeys/{id}`) -> **401** afterwards.
4. **Postman:** import `/api/public/openapi.json`, set an `X-Api-Key` header on the collection, run `whoami`.

QA-API-04 — Per-key rate limit • Core (curl)

Walkthrough

1. Fire `whoami` with one key ~65 times in a minute (`for i in $(seq 1 65); do curl -k -s -o /dev/null -w "%{http_code}\n" https://localhost:7360/api/public/whoami -H "X-Api-Key: pk_…"; done`).
2. **Expected:** the first 60 are **200**, then **429**. A **second** key still returns **200** (budgets are per key, not shared).

QA-API-05 — Register a webhook + send test + verify signature • Core (curl)

Precondition: a receiver URL that echoes requests — e.g. create one at **https://webhook.site** and copy it. **Gherkin**

```
Given HOOKS is enabled and I own the tenant
When I register my receiver and send a test event
Then my endpoint receives a signed POST I can verify with the secret
```

Walkthrough

1. `POST /api/webhooks` (JWT) with `{"url":"<webhook.site URL>","event_types":["ping"]}` -> **201** with a `secret` (`whsec_…`) **shown once** — copy it.
2. `POST /api/webhooks/{id}/test` (JWT) -> **200** `{ "delivered": true, "status_code": 200 }`.
3. On webhook.site, confirm the request has headers **`X-Webhook-Id`, `X-Webhook-Event: ping`, and `X-Webhook-Signature: sha256=…`. Verify: HMAC-SHA256 of the** raw body** with your `whsec_…` secret equals the signature (any HMAC tool, or the app's `WebhookSignature.Compute`).

4. **Non-owner** management -> **403**. **Bad URL / no event types** on create -> **400**.

QA-API-06 — Delivery log + replay • Core (curl)

Gherkin

```
Given I've sent test deliveries to my subscription
When I view its delivery log and replay one
Then I see per-attempt rows, and replay re-POSTs the same event to my endpoint
```

Precondition: send one or more **test** events first (QA-API-05 step 2) — the sync send-test records a delivery row per attempt (success and failure), so the log is populated in-template even though the sample app fires no published events. (Published events via `IWebhookPublisher` also log, through the outbox.) **Walkthrough**

1. `GET /api/webhooks/{id}/deliveries` (JWT) -> a list of attempts, newest first, each with `success`, `status_code`, `error`, `event_id`. Each send-test from QA-API-05 shows up here.
2. `POST /api/webhooks/deliveries/{deliveryId}/replay` (JWT) -> **202**; webhook.site receives the **same** event again (same `X-Webhook-Id`, so a real receiver can dedup). Replay goes through the outbox, so its own attempt is logged too.
3. **Failure path:** point the subscription at a URL that returns 500, send a test -> the response is `{ "delivered": false, "status_code": 500 }` and the log gains a `success:false` row. (The sync send-test does **not** retry; the outbox retry/backoff + dead-lettering applies to *published*/replayed deliveries — watch the API logs on a replay.) Replay of an unknown/other-tenant delivery id -> **404**.

14c. Pre-launch gates — billing off + signup green list (GATES) • Core

> Both gates are **deployment configuration**, read at startup (ADR-027) — every change below needs an **API restart**, and neither can be flipped from inside the app. They exist so a deployment can run **private and free** before it is published: nothing offers to sell a tester anything, and a stranger who finds the URL cannot create an account. Hiding the deployment itself is explicitly out of scope.

QA-GATE-01 — Billing off: the surface does not exist • Smoke (Web + curl) Automated in CI

Automated by `GateJourneyTests.BillingOff_TheSurfaceDoesNotExist_NoLink_NoPage_ApiAnswers404` in the gates-off E2E lane (`Billing__Enabled` unset, `Signup__AllowedDomains__0` = one listed domain; `pwsh tools/e2e.ps1 -Gates` locally). The manual walkthrough stays for a real deployment.

Precondition: `Billing__Enabled` unset in the repo-root `.env` (the shipped default); restart. **Gherkin**

```
Given Billing:Enabled is unset
When I look at the header and call the billing routes
Then there is no billing link, /billing does not render, and the API answers 404
```

Walkthrough

1. Sign in as an owner. **Expected:** the header shows Household / Settings but **no Billing** button.
2. Type `/billing` in the address bar. **Expected:** you land on the home page — the page refuses to render.
3. `curl -k https://localhost:7160/api/billing` -> **404** (not 401 — the route is gone, not protected).
4. `curl -k -X POST https://localhost:7160/api/billing/webhook -d '{}'` -> **404**. The provider callback is gated too: signature authentication is not a reason to accept events for a deployment with no provider.
5. Check the API startup log for one line reading `Pre-launch gates: billing disabled....`

QA-GATE-02 — Billing off: nothing offers an upgrade • Core (Web) Automated in CI

Automated by `GateJourneyTests.BillingOff_AFullHousehold_SaysFull_AndNeverOffersAnUpgrade` in the gates-off E2E lane (`Billing__Enabled` unset, `Signup__AllowedDomains__0` = one listed domain; `pwsh tools/e2e.ps1 -Gates` locally). The manual walkthrough stays for a real deployment.

Precondition: QA-GATE-01's state, plus a household at its seat cap (see QA-HH-14). **Gherkin**

```
Given billing is gated off and my household is full
When I invite one more member, and when an invitee opens a valid invitation
Then both are told the household is full, and neither is told to upgrade a plan
```

Walkthrough

1. As owner of a full household, invite another address. **Expected:** "Your household is full. Remove a member or revoke a pending invitation to free a seat." — **no** mention of upgrading.
2. Open a still-valid invitation to that household as the invitee. **Expected:** the full-household page says to ask the owner to free a seat, **not** to ask them to upgrade their plan.
3. Switch the UI to Spanish and repeat step 1 — the Spanish copy must be the no-upgrade wording too.

QA-GATE-03 — Billing on: the surface comes back whole • Core (Web)

Precondition: set `Billing__Enabled=true`; restart. **Gherkin**

```
Given Billing:Enabled is true
When I sign in as an owner
Then the billing link is back, /billing renders the plan summary, and the seat-limit copy offers an upgrade
```

Walkthrough

1. The header shows **Billing**; `/billing` renders the plan + seat usage (§10c applies in full).
2. A seat-capped invite now reads "...Upgrade your plan to invite more members."
3. `curl -k https://localhost:7160/api/features` -> `{"billing":true}` (anonymous — this is what the client reads).

QA-GATE-04 — Green list: a stranger cannot create an account • Smoke (Web) Automated in CI

Automated by `GateJourneyTests.GreenList_AStrangerIsRefusedAtRedemption_WithTheRightWords` in the gates-off E2E lane (`Billing__Enabled` unset, `Signup__AllowedDomains__0` = one listed domain; `pwsh tools/e2e.ps1 -Gates` locally). The manual walkthrough stays for a real deployment.

Precondition: in `.env` set `Signup__AllowedEmails__0=` **your own address**; restart. Use a *second* address you control that is NOT listed and has **no account yet**. **Gherkin**

```
Given a green list that does not contain my address
When I complete a first-time sign-in with a valid code
Then I am told the app is in private testing and no account is created
```

Walkthrough

1. On `/login`, request a code for the unlisted address. **Expected:** the send succeeds — the gate runs at redemption, not at issue, deliberately (refusing at issue would reveal whether an address has an account).
2. Enter the **correct** code from Mailpit. **Expected:** "This app is in private testing and isn't accepting new accounts yet..." (HTTP **403**, `signup_not_allowed`) — NOT the "incorrect or expired" line.
3. Confirm in the DB (or by retrying) that **no** user row and **no** household were created.
4. Repeat with the same unlisted address via **Sign in with Google**. **Expected:** the same private-testing message on the login page — the provider button is not a side door.
5. Repeat via a **magic link**. **Expected:** the same message.

QA-GATE-05 — Green list: a listed person founds a household and can bring their family • Core (Web) Automated in CI

Automated by `GateJourneyTests.GreenList_AListedOwnerFoundsAHousehold_AndAnUnlistedInviteeMayJoinIt` in the gates-off E2E lane (`Billing__Enabled` unset, `Signup__AllowedDomains__0` = one listed domain; `pwsh tools/e2e.ps1 -Gates` locally). The manual walkthrough stays for a real deployment.

Precondition: QA-GATE-04's state (your address listed, one unlisted address available). **Gherkin**

```
Given my address is on the green list
When I sign in, and then invite the unlisted address into MY household
Then I get my own household, and my invitee may sign up even though they are not listed
```

Walkthrough

1. Sign in with your listed address. **Expected:** normal sign-in; you own a household.
2. Invite the unlisted address from **Household**. Open the invitation as that address and sign in for the first time. **Expected:** sign-in succeeds and they join **your** household — the invitation admits them because the household's **owner** (you) is green-listed.
3. Have that member press **Leave**. They are re-homed into their own empty household — expected, and harmless: now have them invite a **third** unlisted address. **Expected:** the third address is refused at sign-in, because the owner of that household is not green-listed. This is the one-hop boundary.

QA-GATE-06 — Green list never locks out an existing account • Smoke (Web)

Precondition: an account that already exists and is **not** on the green list (e.g. the one from QA-GATE-05 step 2, after removing it from the list if you added it). **Gherkin**

```
Given an account that already exists and is not on the green list
When it signs in
Then the sign-in succeeds
```

Walkthrough

1. Sign out, then sign in again with that existing, unlisted account. **Expected:** success — the gate is on account **creation** only. This is the case that makes editing the list safe once people have data.
2. Clear `Signup__AllowedEmails__0` entirely and restart. **Expected:** the startup log reads `signup open to anyone` and any new address can sign up again — this is what publishing looks like.

14d. v4 audit — adversarial, forge & regression (QA-ADV-25+, QA-DEP-*) • Core

The platform's v4 delta audit (2026-09-23; its reports live in the perezosoft-platform repo, the rule book it produced is `docs/audits/v4-2026-09/FOUNDATION_RULES_v3.md` here) found the classes below. When this section was written, eleven of its cases asserted behaviour the audit had proved broken and were pre-recorded **Blocked**. **All eleven are remediated** (v4 batches B3–B7, merged 2026-09-24 -> 2026-10-01): each case below names the finding and the task that fixed it, and **every case in this section is expected to Pass**. A failure here is a regression — record it **Fail** with the finding id in Notes. (The discipline for a future audit is unchanged: a case that asserts known-broken behaviour is marked **(!) PENDING remediation** and recorded Blocked, never Pass, until its task lands.)

QA-ADV-25 — Joining another household wipes your old one completely • Core (curl + DB)

■ **Remediated** — v4 LB-AUTH-5 / LB-BILL-19, task T21 (R123) landed; expected **Pass**. **Gherkin**

```
Given I am the only member of household A, with an API key and a webhook subscription (PUBAPI/HOOKS
on)
When I accept an invitation to household B
Then household A is gone AND its API key no longer authenticates AND no row of A survives
```

Walkthrough

1. Environment with `PublicApi__Enabled=true`, `Webhooks__Enabled=true`. Sign in as a fresh user (a tenant-of-one is founded). Create an API key (`POST /api/apikeys`, capture the raw key) and a webhook subscription.
2. From a second account, invite the first user to household B; accept it as the first user.
3. `GET /api/public/…` with the captured key. **Expected: 401**. **Today:** 200 — the key still authenticates against a tenant that no longer exists.
4. DB: `SELECT count(*) FROM "ApiKeys"/"WebhookSubscriptions"/"UsageCounters" WHERE "TenantId" = '<A>'`. **Expected:** 0 each. **Today:** rows survive (the accept path deletes the tenant row directly instead of running the dissolution contributors). With a Stripe-backed A, also expect a `billing.cancel` outbox row.

QA-ADV-26 — Dissolving a household leaves no queued email or attachment behind • Core (curl + DB)

■ **Remediated** — v4 JOBS-2 / ADV-P4-7, tasks T22–T23 (R90/R91) landed; expected **Pass**. **Gherkin**

```
Given household A has emails in the outbox (an invitation, a notification)
When A is dissolved (or its owner erases their account)
Then no outbox row for A keeps a recipient address, a body or attachment bytes
```

Walkthrough

1. As A's owner, send an invitation to `x@example.com`. Stop the outbox dispatcher (or read quickly) and note the `OutboxMessages` row for it.
2. Dissolve A (`POST /api/household/leave` as sole owner, or delete the account).
3. DB: `SELECT "Payload" FROM "OutboxMessages" WHERE "Payload" LIKE '%x@example.com%'`. **Expected:** no rows (pending rows wiped, sent rows scrubbed). **Today:** the row survives with `TenantId` null and the full payload — and sent rows are never purged.

QA-ADV-27 — Signing out while a background renewal is in flight really signs out • Core (curl)

■ **Remediated** — v4 LB-AUTH-4 / LB-UI-12, tasks T29/T31 (R124/R125) landed; expected **Pass**. **Gherkin**

```
Given I refreshed once, rotating token A into B
When I sign out presenting the rotated-out token A (the cookie a racing tab still holds)
Then B is revoked too, and no further refresh succeeds
```

Walkthrough

1. Sign in; capture refresh token A. `POST /api/auth/refresh` with A -> capture B.
2. Within 60 s, `POST /api/auth/logout` presenting **A**. Response is 200 either way.
3. `POST /api/auth/refresh` with **B**. **Expected: 401** (the family was revoked). **Today:** 200 — a logout carrying the just-rotated token revokes nothing. Repeat step 2 with an **expired** token and a second live session: the second session should be revoked as well.

QA-ADV-28 — A rotated token is forgiven once, not forever • Core (curl)

■ **Remediated** — v4 AUTH-1, task T28 (R81) landed; expected **Pass**. **Gherkin**

```
Given token A was rotated into B less than 60 seconds ago
When A is presented a second time inside the window
Then it is treated as theft: 401 and every session revoked
```

Walkthrough

1. Sign in -> A. Refresh with A -> B.
2. Refresh with A again (inside 60 s) -> **200** (the benign race, QA-ADV-17 step 2).
3. Refresh with A a **third** time (still inside 60 s). **Expected: 401** and B stops working. **Today:** 200 again — each replay mints another live chain.

QA-ADV-29 — An expired impersonation never turns into the staff account mid-page • Core (Web)

■ **Remediated** — v4 LB-UI-13, task T31 (R125) landed; expected **Pass**. **Gherkin**

```
Given staff is impersonating a member on the member's Household page
When the 15-minute impersonation token expires and staff clicks an action
Then the action is refused or staff is returned home — it never runs as the staff user in the staff's
household
```

Walkthrough

1. As staff (`Admin__StaffEmails__0`), impersonate a member from `/admin`. Stay on the member's `/household`.
2. Wait 15+ minutes (or shorten the impersonation lifetime in a test config).

3. Rename the household. **Expected:** a visible refusal or a return to your own identity with a message; the member's household is untouched and the staff household is untouched. **Today:** the bearer handler silently renews into the staff identity and the rename lands on the **staff's own** household.

QA-ADV-30 — A webhook endpoint that redirects is a failed delivery • Core (curl)

■ **Remediated** — v4 LB-JOBS-1 / JOBS-9 / ADV-P4-9, task T37 (R130) landed; expected **Pass**. **Gherkin**

```
Given a webhook subscription whose URL answers 302 to another host
When a delivery or a send-test runs
Then the redirect is not followed, and the delivery log records a failure with status 302
```

Walkthrough

1. Point a subscription at a public endpoint you control that returns `302 Location: https://<other>/ok` (e.g. a request-bin with a redirect rule); make `<other>` answer 200.
2. `POST /api/webhooks/{id}/test`. **Expected:** `delivered: false`, `status_code: 302`; `<other>` receives nothing. **Today:** `<other>` receives a body-less **GET** carrying the signature headers and the log says delivered (200). With a 307, today the signed body itself reaches `<other>`.

QA-ADV-31 — The delivery log never shows raw server exception text • Edge (curl)

■ **Remediated** — v4 JOBS-1, task T39 (R89) landed; expected **Pass**. **Walkthrough**

1. Subscribe a webhook to `https://does-not-exist.invalid/hook`; send a test.
2. `GET /api/webhooks/{id}/deliveries`. **Expected:** `error` is a short code (`dns`, `timeout`, `connection`, `http_<status>`, `url_refused`). **Today:** the raw exception message (resolver text, IPs, the SSRF verdict).

QA-ADV-32 — Billing off leaves no billing-shaped surface anywhere • Core (curl)

■ **Remediated** — v4 BILL-2/BILL-3, task T45 (R86) landed; expected **Pass**. **Walkthrough**

1. Environment with `Billing__Enabled` unset. As staff: `PUT /api/admin/tenants/{id}/subscription {"plan_key":"pro"}`. **Expected: 404** (the comp write goes with the billing surface) and the console's Comp/Revert buttons are hidden. **Today:** 200, the tenant becomes Pro while billing is "off".
2. Complements QA-GATE-01 (which checks `/api/billing*` only).

QA-ADV-33 — Expired invitations stop holding seats • Core (Web)

■ **Remediated** — v4 LB-AUTH-7 / LB-BILL-20, task T34 (R127) landed; expected **Pass**. **Walkthrough**

1. Free household at `cap − 1` members plus pending invitations that are **past their expiry** (seed `ExpiresAt` in the past in the DB, or wait out `LifespanDays`).
2. Invite a new address. **Expected:** allowed (an expired invitation reserves nothing). **Today:** 402 `seat_limit_reached` / "household full" — dead invitations keep their seats forever.

QA-ADV-34 — The native export download never sends your sign-in token to the file host • Edge (Android/Desktop)

■ **Remediated** — v4 NAT-12, task T49 (R102) landed; expected **Pass**. **Walkthrough**

1. Environment with `Files__Provider=S3` (MinIO in compose). On the native app, Household -> Export data.
2. Watch the MinIO/S3 access log (or a proxy). **Expected:** the presigned GET carries **no** `Authorization: Bearer` header and the download succeeds. **Today:** the bearer JWT is attached (and AWS S3 rejects a presigned GET that also carries an Authorization header).

QA-ADV-35 — Exported logs carry no email addresses • Edge (Ops)

■ **Remediated** — v4 OBS-1, task T41 (R93) landed; expected **Pass**. **Walkthrough**

1. Set `OpenTelemetry__Otlp__Endpoint` to a collector you can read (Grafana Cloud or a local otel-collector with a file exporter). Sign in with OTP twice.
2. Search the exported log records for `@`. **Expected:** none — records carry user ids only. **Today:** every token issue exports `{Email}` as an attribute.

QA-DEP-01 — Deploy staging from GitHub • Smoke (Operator)

Gherkin

```
Given develop on GitHub
When I run the CI workflow on develop with deploy = staging
Then every web gate runs, and only when all pass the Render hook fires and the version-gated smoke
passes
```

Walkthrough

1. GitHub -> Actions -> `CI` -> *Run workflow* on `develop`, `deploy` = staging, `devices` = none (ADR-031; DEPLOYMENT §6).
2. **Expected:** `build-test`, `secret-scan`, `qa-artifacts`, `license-scan`, `docker-build` and `e2e` run, then `deploy-staging` fires `RENDER_DEPLOY_HOOK_STAGING`, waits until `/api/version` reports the commit and runs the post-deploy smoke green. No device leg runs. A merge alone deploys nothing.

QA-DEP-02 — A deploy run on the wrong branch fails loudly • Core (Operator)

Walkthrough

1. *Run workflow* on `main` with `deploy` = staging. **Expected:** the web gates run, then `deploy-staging` fails at its first step: "staging deploys from develop". Nothing is deployed.
2. *Run workflow* on `develop` with `deploy` = prod. **Expected:** `deploy-prod` fails the same way ("prod deploys from main").

QA-DEP-03 — A red gate blocks the deploy • Edge (Operator)

Walkthrough

1. On a throwaway branch off `develop`, break a unit test; run the workflow on that branch with `deploy` = staging. **Expected:** `build-test` is red, `deploy-staging` is skipped, and the Render hook never fires.
2. Delete the throwaway branch.

QA-DEP-04 — The device legs run only when asked for • Core (Operator)

Walkthrough

1. Open a pull request that changes `src/Shared.Ui`. **Expected:** only the web gates run — no `native-*` job.
2. *Run workflow* on `develop` with `deploy` = none, `devices` = android. **Expected:** `native-build` (Android leg only), `native-release-android` and `native-smoke-android` run; no web gate, no Windows or Apple job.
3. Repeat with `devices` = all before a release (macOS bills 10×). **Expected:** every native job green.

QA-SET-09 — A theme saved just before a reload survives the reload • Core (Web + Android)

Walkthrough

1. Signed in, switch theme Light -> Dark in Settings; immediately reload (web) or background/foreground the Android app so the WebView reloads. **Expected:** Dark stays (platform PR #233 — the switcher remembers the saved value; `PreferenceSyncClaimTests`). Repeat with the language switcher.

QA-AND-16 — An API base configured with a trailing slash still signs in • Edge (Android)

Walkthrough

1. Build the Android app with `-p:ApiBaseUrl=https://<staging>/` (trailing slash). Sign in with OTP and with Google. **Expected:** both succeed; no request goes to `//api/…` (platform PR #230 — `TrimEnd('/')` in `MauiProgram` and both OAuth initiators).

15. Traceability matrix (feature -> cases -> API)

Feature area	Test cases	Key API endpoints
OAuth sign-in (Google/MS)	SMK-02, AUTH-02, DSK-02, AND-02	`GET /api/auth/login/{provider}`, `GET /api/auth/callback/{provider}`, native `login/callback/exchange`
Magic link (web)	AUTH-01, 05, 06, MAIL-02	`POST /api/auth/magic-link/send`, `GET /api/auth/magic-link/verify`
Email OTP	SMK-01/05/06, AUTH-03/04, DSK-01, AND-01, MAIL-01	`POST /api/auth/otp/send`, `POST /api/auth/otp/verify`

Feature area	Test cases	Key API endpoints
Rate limiting / abuse guard	AUTH-11	`POST /api/auth/otp/send`, `…/magic-link/send`, `…/otp/verify` (429)
Session / refresh / sign-out	SMK-03/04, DSK-03/06, AND-03, SEC-03, SEC-05 (legacy-cookie self-heal)	`POST /api/auth/refresh`, `POST /api/auth/logout`, `GET /api/auth/me`
Enumeration / error handling	AUTH-04/07/08/09	(send + verify endpoints; `/login` query states)
Onboarding (auto tenant)	ONB-01/02, SMK-01	(provisioned on first auth)
Household view/rename	HH-01/02	`GET /api/household`, `PUT /api/household`
Members (remove/leave/transfer/dissolve)	HH-03..08	`DELETE /api/household/members/{id}`, `POST /api/household/leave`, `POST /api/household/transfer-ownership`
Invitations	INV-01/06/07/08, MAIL-03	`POST /api/household/invitations`, `GET /api/household/invitations`, `POST …/{id}/regenerate`, `DELETE …/{id}`
Join / accept	INV-02/03/04/05, **INV-10** (seat re-check after downgrade — BILLING-9, E2E), **DSK-08 / AND-08** (paste invite code — NATIVE-4b; web variant E2E)	`POST /api/household/invitations/accept` (402 `seat_limit_reached` when the tenant is over its downgraded cap)
Linked accounts	SET-01..06, DSK-05	`GET /api/auth/logins`, `POST /api/auth/link/{provider}`, `DELETE /api/auth/logins/{provider}`
Localization	I18N-01..04, **DSK-09 / AND-09** (native restart persistence — NATIVE-5)	`PUT /api/auth/locale` (+ resx)
Theme / dark mode (THEME-1 + PREFS-1)	**SET-08** (E2E `ThemeJourneyTests`) + **DSK-15 / AND-14** (native restart persistence)	`PUT /api/auth/theme` ("system" stored verbatim, null = never chose — ADR-022; `theme` JWT claim; pre-paint `theme.js` -> `data-bs-theme`; sign-in reconcile + device adoption)
Emails / branding	MAIL-01..04, I18N-04	(SMTP via Mailpit)
Tenant isolation / auth guards	SEC-01..05	(all `[Authorize]` endpoints; write-stamping + reuse detection are automated)
Platform health / readiness	SMK-07	`GET /health`, `GET /health/ready`
Transactional email delivery	(all email cases)	async via the outbox dispatcher (`OutboxMessages`)
Billing — checkout/portal/webhook + billing page	**BILL-01/02** (§10c, E2E `BillingJourneyTests`) + **DSK-11 / AND-11** (native refresh-on-return — NATIVE-4) + `Api.Tests` (Billing*/Entitlement* tests)	`POST /api/billing/checkout`, `…/portal`, `…/webhook`
Billing — quotas (BILLING-5)	**HH-14** (seat limit blocks invite -> 402 upgrade message) + `Api.Tests` (`QuotaServiceTests`)	seats (members + pending invites vs `Plan.SeatLimit`) enforced on `POST /api/household/invitations` -> 402 `seat_limit_reached`; metered usage via `IQuotaService.TryConsumeAsync` (monthly `UsageCounter`). Limits in `PlanCatalog` (null = unlimited).
Billing — trial/dunning (BILLING-6)	covered by `Api.Tests` (`BillingWebhookHandlerTests`, `SubscriptionLapseSweepJobTests`); manual via Stripe test triggers	webhook transition into `active`/`trialing` from nothing or a lapsed state -> owner **"Subscription active"** notification (`billing.activated`, plan + renewal date) once — a renewal or a trial converting is silent; transition into `past_due`/`canceled` -> owner **notification** (in-app bell + outbox email, NOTIFY) once; `SubscriptionLapseSweepJob` (6h) nudges the owner once when a paid period lapses without a webhook (`LapseNotifiedAt`). Verify with `stripe trigger invoice.payment_failed` (test mode) -> owner sees a billing notification in the bell.
Feature area	**Test cases**	**Key API endpoints**
Billing — dissolve cleanup (BILLING-7)	covered by `Api.Tests` (`BillingDissolveTests`)	on tenant dissolve, `BillingDataContributor` wipes the `Subscription` projection **and** enqueues a `"billing.cancel"` outbox message -> `IBillingProvider.CancelSubscriptionAsync` (a deleted tenant stops being billed). `HasDataAsync`=false (billing never blocks leaving); export gains a `billing` section

Feature area	Test cases	Key API endpoints
		(plan/status/period, no Stripe ids). Manual (Stripe test mode): subscribe a throwaway tenant, delete the account, confirm the Stripe subscription is canceled.
Public API + API keys (PUBAPI, **config-gated off**)	**QA-API-01..04** (curl/Postman) + `Api.Tests` (`ApiKeyServiceTests`, `RateLimitingTests`); boot-verified on/off	`PublicApi:Enabled` toggles it. Owner-only `/api/apikeys` (create -> raw `pk_…` once, list, revoke; `Permission.ManageApiKeys`); API-key auth scheme mints a `tenant_id`-scoped principal; demo `/api/public/whoami` (read scope) + `/api/public/echo` (write scope) via `.RequireApiScope`. **PUBAPI-2:** per-key rate limit (60/min, isolated per key -> 429) + a leak-free public OpenAPI doc at `/api/public/openapi.json` (only the public routes). **Off (default) => routes 404.** Manual: `PublicApi__Enabled=true`, mint a key, `curl -H "X-Api-Key: pk_…" /api/public/whoami`; fetch `/api/public/openapi.json`.
Outbound webhooks (HOOKS, **config-gated off**)	**QA-API-01, 05, 06** (curl/webhook.site) + `Api.Tests` (`WebhookSubscriptionServiceTests`, `WebhookDeliveryTests`, `WebhookDeliveryLogTests`) + `Core.Tests` (`WebhookSignatureTests`); boot-verified on/off	`Webhooks:Enabled` toggles it. Owner-only `/api/webhooks` (register -> signing secret `whsec_…` once, list, delete, **send test**; `Permission.ManageWebhooks`). `IWebhookPublisher.PublishAsync` fans out to matching active subs -> one `"webhook"` **outbox** message each -> signed POST (`X-Webhook-Signature`), retry/dead-letter via the outbox. **HOOKS-2:** a delivery log (`GET /api/webhooks/{id}/deliveries` — one row per attempt, success/status/error) + **replay** (`POST /api/webhooks/deliveries/{id}/replay` — re-enqueue the exact payload). **Off (default) => routes 404.** Manual: `Webhooks__Enabled=true`, register a receiver (e.g. a webhook.site URL), hit **send test**, view deliveries, replay one.
Pre-launch gates (GATES, **both deployment config**)	**QA-GATE-01..06** (§14c) + `Api.Tests` (`BillingGateTests`, `SignupGateTests`, `SignupRefusalSurfacingTests`, `ConfigPostureTests`) + `Ui.Tests` (`BillingGateUiTests`, `SeatLimitCopyTests`, `SignupRefusedCopyTests`)	`Billing:Enabled` off (default) removes the billing controllers from the application model, so `GET /api/billing`, `POST …/checkout`, `…/portal` and `…/webhook` all **404** and every tenant resolves Free; the client reads `GET /api/features` (anonymous) to hide the link and refuse `/billing`, and the seat-limit copy drops the upgrade pitch. `Signup:AllowedEmails` / `Signup:AllowedDomains` (empty = open) gate account **creation** at `UserService.CreateUserWithTenantAsync` — refusal is **403** `signup_not_allowed` on `POST /api/auth/otp/verify` and `?error=signup_not_allowed` on the OAuth / magic-link redirects. An invitation admits its addressee only when that invitation's tenant **owner** is green-listed. ADR-027.
v4 audit — adversarial (§14d)	**ADV-25..35** (all remediated in v4 B3–B7 — each names the task that fixed it)	accept-path dissolve, outbox erasure, logout/grace, impersonation expiry, webhook redirects + error codes, billing-off admin comp, expired-invite seats, native bearer scope, exported-log PII
Audit log (API-only)	covered by `Api.Tests` (`AuditLogTests`)	append-only `IAuditLog` + interceptor
RBAC roles (admin tier)	HH-09/10/11/12 (web roster promote/demote + admin capability/limits); `Api.Tests` (`RolePermissionsTests`, `PermissionServiceTests`, `MemberRoleManagementTests`)	`PUT /api/household/members/{id}/role` (owner-only; admin<->member, owner via transfer only); permission seam gates tenant writes
File storage (API-only)	covered by `Api.Tests` (`LocalDiskFileStorageTests`, `FileDownloadTokenizerTests`, `FilesControllerTests`, `S3FileStorageMinioTests` [real MinIO], `FileStorageRegistrationTests`)	`IFileStorage` (tenant-scoped keys; local disk / S3-compatible — AWS/MinIO/R2/B2, config-gated); local signed `GET /api/files/{token}` (expiring, single-key, tenant-checked -> 404 on any failure); S3 native presigned URLs
GDPR data export	**HH-13** (owner Household -> Data -> download, E2E `GdprExportJourneyTests`) + **DSK-10 / AND-10** (native share — NATIVE-3) + `Api.Tests` (`TenantExportTests`)	`POST /api/household/export` (owner-only `ExportData` -> 403 else; JSON bundle via `IFileStorage`, signed URL; secret-free, tenant-scoped, audited)
GDPR account erasure	**SET-07** (Settings -> Danger zone) + `Api.Tests` (`AccountErasureTests`)	`DELETE /api/auth/me` (wipes identity/PII in one tx; owner-with-members -> 400, solo owner -> 409 without `confirm_dissolve`; member removed not re-homed; audited; audit trail survives)

Feature area	Test cases	Key API endpoints
MFA / TOTP	**MFA-01..05**, **DSK-12 / AND-12** (native) (Settings enroll/QR/confirm/recovery + disable; step-up on OTP, OAuth/magic-link, **and** native logins) + `Api.Tests` (`MfaServiceTests`, `MfaChallengeServiceTests`, `MfaLoginServiceTests`)	`GET|POST /api/auth/mfa[/enroll|/confirm|/disable]` (enroll/manage; secret encrypted, hashed single-use recovery codes) + **login step-up** `POST /api/auth/mfa/verify` (MFA-on logins get a signed challenge instead of a session; verify a TOTP/recovery code to complete). **Every sign-in path enforces it** (web + native): OTP returns the challenge as JSON; OAuth callback + magic-link redirect to `/login?mfa=<challenge>`; native OTP/OAuth-exchange return the challenge in the body and the MAUI client steps up in-app.
In-app notifications	**NOTIF-01..04**, **DSK-13** (header bell: list/unread-count/mark-read/delete/clear; Settings delivery-preference switches) + `Api.Tests` (`NotificationServiceTests`, `NotificationFanOutTests`)	`GET /api/notifications` (+ `?before=&limit=`), `/unread-count`, `POST /{id}/read`, `/read-all`, `DELETE /{id}`, `DELETE /api/notifications?read=true|false` (scope REQUIRED — read-only vs all; omitted -> 400), and `GET|PUT /api/notifications/preferences` — **per-user** (scoped to the caller). `NotifyAsync` fans out to in-app + email (outbox-backed) per prefs (default both on).
Admin back-office	**ADMIN-01..06**, **DSK-14** (native spot) (staff `/admin` console: tenant list/detail + impersonate w/ banner + stop; targeted/broadcast announce; plan comp/revert) + `Api.Tests` (`PlatformStaffServiceTests`, `AdminControllerTests`)	`GET /api/admin/me` (staff probe, 200 `{is_staff}` for any caller — drives the nav/gate), `GET /api/admin/tenants` (+ `/{id}` — returns `plan_key` + `provider_managed`), `POST /api/admin/impersonate/{userId}`, `POST /api/admin/tenants/{id}/announce` (optional `user_ids[]` subset, intersected with membership), `POST /api/admin/announce-all` (202; outbox fan-out to **every** user), `PUT|DELETE /api/admin/tenants/{id}/subscription` (comp/revert; 409 only for a LIVE provider sub — canceled Stripe-backed subs are comp-able) — **platform-staff only** (config `Admin:StaffEmails`; non-staff -> 403). Detail enters the target tenant (filter never loosened); impersonation returns a **short-lived, non-refreshable** token with an `impersonated_by` claim, **audited in the target's tenant**.

JiggerJot's own flows (§10d–§10i). Everything above this block is the platform; everything in it is the product. Cited decisions are `JJ-nnn` in `docs/DECISIONS.md`.

Feature area	Test cases	Key API endpoints
The shelf (INV-1)	**SHELF-01/02/03/05/06** (E2E `ShelfJourneyTests`) + SHELF-04/12/13	`GET /api/inventory` (the WHOLE catalog with an availability flag — you cannot tick what you cannot see), `PUT /api/inventory/{ingredientId}`. Unticking UPDATES the row rather than deleting it, so "checked and I do not have it" stays apart from "never looked" (JJ-023); every reader filters on `is_available`, so absence and false behave identically either way. Ice and water are not in the catalog at all (JJ-020).
Custom ingredients (INV-2)	**SHELF-07/08** (E2E)	`POST /api/inventory/ingredients` (created ticked; 409 + `existingIngredientId` on a case-insensitive duplicate against the shared catalog AND the household's own), `GET /api/inventory/categories`. The app's first write to a dual-natured table, so the handler sets `TenantId` by hand — nothing stamps it (JJ-031).
The shelf, reworked (INV-3)	SHELF-09/10/11	no new endpoint. Counts are over the whole category, never over what search left visible; card and jump bar read ONE value. The payoff total re-asks `GET /api/cocktails?makeable=true&pageSize=1` **once per burst of ticks**, not once per tick — a slice may not reach into another slice (R7/TR-9), and duplicating the query would give the app two definitions of makeable (JJ-003).
Bulk shelf write (ONBOARD-1)	**START-03/05** (E2E `OnboardJourneyTests`) + `Api.Tests` (`BulkInventoryTests`)	`PUT /api/inventory` — the whole shelf in one request. State is STATED rather than toggled, safe to send twice, and an id this household cannot see comes back in `unknown` rather than failing the request.
Browse / search / paging (CKTL-2)	**CAT-01** (E2E `CocktailBrowseJourneyTests`)	`GET /api/cocktails` (paged; a bad page or size is clamped, never a 400). Ordered name-then-id, because name alone is not a total order in this catalog; the SOURCE is shown per row because four names appear in both books.

Feature area	Test cases	Key API endpoints
Recipe detail (CKTL-3, CKTL-4)	**CAT-02/03/08** (E2E) + **MAKE-07** (E2E)	`GET /api/cocktails/{id}` — lines in order, `makeability` plus per-line `availability`/`substituteWith`. Another household's cocktail is a **404, never a 403** (JJ-031). Glass and method may be null and are rendered as nothing at all (JJ-034).
Measurement preference (PREFS-2, amended by PREFS-3)	**CAT-07** (E2E)	`GET`/`PUT /api/unit-preference` (the UnitPreference slice, Arch A3) — Metric or Imperial only, empty is a 400; conversion is server-side in Core's `AmountDisplay`, so a second front end inherits the rules. Teaspoons and neutral units never convert; metric reads at 30 ml to the ounce in 2.5 ml steps, never a fraction; null reads as imperial and `/me` says so (JJ-041, JJ-008).
Every amount in oz or ml (PREFS-3)	**CAT-10** + CAT-07 (E2E) + `Core.Tests` (`BarMeasureTests`, `AmountDisplayTests`) + `Api.Tests` (`OunceMigrationTests`, `CatalogSeederTests`, `CocktailAuthoringTests`)	`GET /api/cocktails/lookups` (units per reader), `POST /api/cocktails` (stored in oz). Every volume stored in ounces on the quarter marks through Core's `BarMeasure`: parts as their share of 3 oz, a glass or wineglass 2 oz, a liqueur glass 1 oz; the seeder and the authoring handler write through it; a one-off SQL migration converts existing rows and is held to `BarMeasure` by a test (JJ-041).
Filters (FILTER-1)	**CAT-04/05** (E2E) + CAT-06/09	`GET /api/cocktails?ingredient=&method=&glass=&serving=`, `GET /api/cocktails/filters`. The ingredient filter reads the RECIPE LINES — there is no main-spirit column and never will be (JJ-014) — and matches name, category and subcategory at once, so a parent catches every child (JJ-015, JJ-016). Options are derived from the catalog so no filter is a dead end.
Makeable (MAKE-1)	**MAKE-01/02** (E2E `MakeableJourneyTests`) + MAKE-05	`GET /api/cocktails?makeable=true`. Derived at query time, never stored (JJ-003). A required line is satisfied by the exact ingredient or by a substitute **in that direction** — cognac stands in for brandy, not the reverse (JJ-006). Optional lines never block (JJ-009). Each row carries the swaps in play.
One ingredient away (ALMOST-1)	**MAKE-03/04** (E2E) + MAKE-05	`GET /api/cocktails?almost=true` — exactly ONE required line unsatisfied after substitutions (JJ-019), with `missingIngredient` named on every row. The same predicate as makeable, counted rather than negated, so the two lists are adjacent and can never overlap; asking for both returns an empty page rather than one flag winning.
The unlocking bottle (ALMOST-2)	**MAKE-06/08** (E2E)	`GET /api/cocktails/unlocks?limit=` — the almost set grouped by MISSING INGREDIENT and ranked (JJ-035). `unlocks` always equals the length of `cocktails` (one value read twice, which is why the names are never truncated in the DATA); ties break on name; an empty shelf ranks nothing.
Where to start (MARGA-3)	**MAKE-10**, CAT-09	`GET /api/cocktails/starters?limit=` — a THIRD reading of the catalog, not a variant of `/unlocks`: which bottles the recipes lean on most, minus what the household has. Required lines only (JJ-009), substitutions ignored. **`appears` is how many recipes ASK for it, never how many it would unlock.**
Forking (FORK-1)	**MINE-01** (E2E) + MINE-02	`POST /api/cocktails/{id}/fork` — a SNAPSHOT copy, never a reference (JJ-002, JJ-013): a new tenant-owned `Cocktail` plus copies of every line, `TenantId` set by hand on both tables. The source credit is deliberately NOT copied; provenance rides on `ForkedFromCocktailId`, which is not a foreign key, so deleting the original leaves the copy standing.
Authoring (AUTHORING-1)	**MINE-03/04** (E2E) + MINE-05/06	`POST /api/cocktails`, `GET /api/cocktails/lookups`. **Two lookup endpoints that must not be merged:** `/filters` is catalog-derived so no filter is a dead end, `/lookups` is the whole curated set (JJ-022) so a form can reach a glass no recipe uses. Refuses a lineless recipe, a unit with no amount, and an ingredient the household cannot see. Request enums cross the wire BY NAME.

Feature area	Test cases	Key API endpoints
The ingredient suggests the role (AUTHORING-3)	**MINE-07** (E2E) + `Core.Tests` (`RecipeRolesTests`) + `Api.Tests` (`SeedRolesParityTests`, `CocktailAuthoringTests`) + `Ui.Tests` (`WriteRoleSuggestionTests`)	`GET /api/cocktails/roles?ingredient=…` — roles in the order asked, from the ingredient's top-level category through Core's `RecipeRoles` (the seed script's rule, held to it by a parity test). First spirit Base, later spirits Modifier, a garnish optional; another household's ingredient is Other (JJ-031). The form never overwrites a role or required box set by hand.
A searchable ingredient picker (AUTHORING-4)	**MINE-08** (E2E) + `Ui.Tests` (`IngredientPickerTests`)	none — presentation only. `IngredientPicker` in the RCL: an ARIA combobox (input `role="combobox"`, `aria-controls` -> listbox, `aria-activedescendant` -> the arrowed option) over the bottles this household can see; names starting with the typed text first, then name or category contains it; at most fifty shown; arrows + Enter + Escape; picks only from the list (JJ-031); on-shelf bottles marked. Test ids `new-line-ingredient` / `-option` / `-empty`.
A glass on every recipe (AUTHORING-6, JJ-043)	**MINE-11** (E2E) + **CAT-03** + `Api.Tests` (`CocktailAuthoringTests`, `CocktailEditingTests`, `CocktailAuthoringEndpointTests`, `CatalogSeederTests`) + `Ui.Tests` (`WriteLayoutTests`)	`POST`/`PUT /api/cocktails` refuse a recipe without a glass: **400 `glass_required`**, checked after the name and the lines. `Cocktails.GlassTypeId` NOT NULL (`CocktailGlassRequired`, which stops with a message naming the rows if any cocktail has none). The form: *Choose a glass* in place of *Not stated*, Marga's compact line beside the field (`new-glass-marga`) for the client's check and the server's code alike. Method stays optional (JJ-034).
Editing a cocktail (AUTHORING-2)	**MINE-09** (E2E) + `Api.Tests` (`CocktailEditingTests`) + `Core.Tests` (`BarMeasureTests`) + `Ui.Tests` (`WriteEditTests`)	`GET /api/cocktails/{id}/draft` (the request's shape, volumes in the caller's writing unit), `PUT /api/cocktails/{id}` (same body and 400 codes as POST). Household-owned only; the shared catalog **403 `catalog_read_only` **(JJ-002), another household** 404** (JJ-031). A fork keeps `forkedFrom` (JJ-013). Every line replaced; one `PrepareAsync` behind writing and editing; last save wins. `/cocktails/{id}/edit` is the same page as `/cocktails/new`; `cocktail-edit` on the recipe page for own cocktails only.
Onboarding wizard (ONBOARD-1)	**START-01/02/03/05** (E2E) + START-04/06/07	`GET /api/inventory` + `GET /api/cocktails/starters?limit=12` + `PUT /api/inventory`. Offered, never forced: **no redirect and no dismissal flag**, so there is no "has this household been onboarded" fact to store — `Tenant` is the platform's. Members joining by invitation skip it for free, because they already have a shelf (FEATURES §7, JJ-021).
Marga (MARGA-1/2/3/5/6)	CHROME-01/02/03/**10/11/12/13/26**, MAKE-08/09/10 + `Ui.Tests` (`MargaPresenceTests`, `MargaEverywhereTests`)	none of her own. **She is a drawn character, not an assistant**: every line is a localized resource string with real data in its placeholders, picked whole rather than assembled, from queries that already exist. Her component takes a FINISHED sentence — it cannot build one, pick one or fetch anything. She is `alt=""`/`aria-hidden`.
She pages through the next bottles (MARGA-7)	**CHROME-27** + `Ui.Tests` (`BottlePagerTests`, `MargaBottlePagingTests`)	`GET /api/cocktails/unlocks?limit=10` and `GET /api/cocktails/starters?limit=10` — no new endpoint; every screen had asked for one. One `BottlePager` component (‹ "2 of N" ›, nothing for a single bottle, stopping at both ends, `MaxBottles` = 10) under every line where she names a bottle: Home's panel (her line follows), the shelf (the outlined pill follows), the catalog's one-away panel (its drink names follow, the list below does not), and both "where to start" lines. A later bottle says "or"; a re-rank resets to the best; the line sits in an `aria-live` region.
The responsive shell (SHELL-1)	**CHROME-04/06** (E2E `ShellJourneyTests`) + CHROME-05	none. ONE element repositioned by CSS, never a second copy hidden at one width — two `nav-shelf` in the DOM fails every journey that clicks it. The current tab is weight plus a drawn indicator, never colour alone, plus `aria-current`.
Feature area	**Test cases**	**Key API endpoints**
The boot state (SHELL-2)	CHROME-07/08, and the native legs of §12/§13	none — it renders before the app exists. It lands in **two** `index.html` files (`NATIVE_PARITY.md`), held by a CI gate; the WebView hosts sweep instead of filling, having no

Feature area	Test cases	Key API endpoints
		download to measure. A second gate caps the illustration's size, since this is the one place it is fetched before the app is usable.
Marga in the emails (MARGA-4)	**MAIL-05/06/07** + `Api.Tests` (`MargaInEmailTests`)	none — `BrandedEmail` renders her the way it renders the logo: a CID inline image, the one approach Gmail and Outlook both show (they block data-URIs). She is on the three emails a person ASKED for and deliberately not on the notification wrapper, which carries failed payments and security alerts. Attached only when shown, so her 19 KB never rides along unused. Her line is one whole resource string from `EmailStrings.resx`, same contract as in the app.
The app in Spanish	**CHROME-09**, I18N-01..04	resx only. Her lines are voiced, so the Spanish is a writing job rather than a translation, and the placeholders have to survive the rewrite.
Billing, Admin and the small screens (BACKBAR-8)	**CHROME-24** + BILL-*, ADM-* (E2E `BillingJourneyTests`, `SeatQuotaJourneyTests`, `AnnouncementJourneyTests`, unchanged) + `Ui.Tests` (`UndrawnScreensTests`, `BillingGateUiTests`)	none — presentation only (JJ-039), derived by analogy from pages 02 and 16–17 since the handoff never drew these. Billing as two groups with the status pill in the status colours; the console's six cards and one table as groups with tenants and members as rows and every action a text link; statuses as one `.notice`; not-found as a `NotFoundView` on the empty-state layout; the callback's spinner a brass bar; the impersonation banner a hairline strip; the error bar in the error colours. No admin write added (ADR-021); every gate and id intact.
The sweep (BACKBAR-9)	**CHROME-25** + `Api.Tests` (`RestyleGateTests` — no `.card` outside the unlock panel, every transition and animation named again under `prefers-reduced-motion`, the primary button's disabled tokens set) + `Ui.Tests` (`AnonymousLayoutTests`, Join's heading)	none — presentation only (JJ-039). Ninety-six frames (sixteen screens × three widths × two themes) rendered against mocked responses and read one by one; three findings, each fixed with a test first: the bell's dropdown was still a card, every primary button that STARTS disabled painted in Bootstrap's blue because the copper rule never set the disabled tokens, and Join's headings were the bold sans the split's borrowers had kept. The boot arc's 50ms transition gained the reduced-motion rule the other motions already had. **Amended 2026-09-14** by a second pass against the REAL API with a stocked shelf: every drink name was copper in dark theme (the dark link rule's `:not()` exclusions put it at 0,4,1, above every page's own rule — now inside `:where()`, gated by `RestyleGateTests` and asserted as a computed colour in `ShellJourneyTests`), and the wizard's second step stretched her scene to the height of 191 pills (now one sticky block inside her column, held by `MargaSplitTests` and a geometry assertion in `OnboardJourneyTests`).
Settings and Household restyled (BACKBAR-7)	**CHROME-22/23** + SET-01..08, MFA-01..03, NOTIF-01..04, HH-01..14, INV-01..10 (E2E, one journey's two dropdown selects become label clicks) + `Ui.Tests` (`SettingsLayoutTests`, `HouseholdLayoutTests`, `SwitcherStateTests`)	none — presentation only (JJ-039). `ThemeSwitcher`/`UnitSwitcher` gain `Segmented`, radios on their own ids (`theme-choice-*`, `unit-choice-*`) sharing the select's handler and PUT; the header's select keeps `theme-switcher`, so a page no longer carries that id twice. `.settings-group`/`.settings-row` in `app.css`, shared by both pages and both cards; the ⋯ row menu toggles visibility only, so no id doubles. Owner badge copper, Admin/Member neutral.
The write page restyled (BACKBAR-6)	**CHROME-21** + MINE-03/04 (E2E, unchanged) + `Ui.Tests` (`WriteLayoutTests`)	none — presentation only (JJ-036 F4). Two columns; the amount in the display face beside its unit; role and required a visible sub-line, no popover; the two client-side checks attach to the name field and to the lines (new id `new-lines-error`, additive), `new-error` kept above Save for the server's answer; a fixed Save bar on a phone. Every `new-*` id intact.
Feature area	**Test cases**	**Key API endpoints**
Login, Welcome, Join, auth error (BACKBAR-5)	**CHROME-20** + SMK-01, AUTH-01..10, START-01..07, INV-02..05 (E2E, unchanged) + `Ui.Tests` (`MargaSplitTests`, `AnonymousLayoutTests`)	none — presentation only (JJ-036). One `MargaSplit` component: her scene as half the screen with the line on it, night ground in both themes, the working side beside; Login and Welcome are her two full appearances, Join and the auth error page reuse it. The wizard's step 2 renders the shelf's sections, whose rules moved to `app.css` so two pages share them; a 3px brass progress rule; every id intact. Her art, gradient and line are one sticky block inside her

Feature area	Test cases	Key API endpoints
		column (2026-09-14), so a long panel beside her — step 2 is the whole shelf — never stretches the scene; `OnboardJourneyTests` asserts her scene is never taller than the viewport there. Login is a framed card again (2026-09-14, the maintainer's call, the 2026-09-10 proposal's placement): `MargaSplit Whole` keeps the scene square so the whole drawing shows, the form is centred with the subtitle back, and on a phone the form sits under the square rather than sliding over her; `AuthFlowTests` holds the card's width and the phone geometry.
Catalog and recipe restyled (BACKBAR-4)	**CHROME-18/19** + MAKE-01..10 (E2E `MakeableJourneyTests` through the new `SetCatalogFilterAsync` helper) + `Ui.Tests` (`CatalogChipsTests`, `RecipeLayoutTests`)	none — presentation only, but the one slice that touches a journey's MECHANICS (JJ-036): the two switches are three radios (`cocktail-makeable`, `cocktail-almost`, new `cocktail-all`), driven by label like the shelf pills; only the selected chip carries the response's own total, no new fetch. Rows keep their list-group classes. Amounts lead in the serif; two marks only; optional is a dash and plain text; provenance in the facet line; not-found on the empty-state layout; `@media print` on the recipe.
Home and the shelf restyled (BACKBAR-3)	**CHROME-16/17** + `Ui.Tests` (`HomeLayoutTests`, `ShelfSectionsTests`) + the shelf and makeable journeys unchanged	none — presentation only (JJ-036). Home in two columns, rules not cards, the unlock panel the only box, the count in the serif, her scene on an empty shelf; the shelf's cards become sections on a hairline with `#cat-{slug}` intact, the named bottle outlined in the subtle copper and never ticked, the add form a panel in place, the payoff count in the serif.
The chrome off copper (BACKBAR-2)	**CHROME-15** + CHROME-04 (E2E `ShellJourneyTests`, colour lines rewritten to the hierarchy) + `Ui.Tests` (`AppHeaderChromeTests`) + `Api.Tests` (`RestyleGateTests`, the two marks)	none — presentation only (JJ-037). The bar and the tab bar on the surface with a hairline in both themes; destinations in the ink, account cluster muted, one theme-aware button variant for all of them so an anchor-shaped button and a `<button>` can never differ; 2px copper indicator; `icon_dark.svg` for the dark surface, swapped by `app.css`, named in `REBRANDING.md` and `build_assets.py`.
The restyle's foundation (BACKBAR-1)	**CHROME-14** + `Api.Tests` (`RestyleGateTests`) + `Ui.Tests` (`MargaSaysToneTests`)	none — presentation only (JJ-036). Both token sets in `app.css`; one self-hosted display serif, one weight, display only, reached through `.font-display` and never bold (JJ-038, gated); the primitives as CSS on Bootstrap's classes so no markup churns; `MargaSays` gains `Tone` with the old `Size`/`Compact` intact.

Adversarial & tenant-isolation (§14a, QA-ADV-*) — v3-audit hardening probes. **Rows tagged (!) v3 **were authored against then-broken behaviour and sat** Blocked **until their finding landed;** every tagged finding is fixed **(v3 remediation, PRs #147–#191), so ALL rows now expect** Pass**.

Probe (finding)	Test case(s)	Key API endpoint(s) / surface
Cross-tenant read/export/erase (RLS-2/4)	ADV-01	`GET /api/household`, `POST /api/household/export`, `DELETE /api/household/members/{id}`, `POST|DELETE /api/notifications/{id}[/read]` (foreign id -> 404/403)
Cross-tenant isolation, newer tables (LB-TEN-2/RLS-8)	ADV-02	`GET /api/webhooks/{id}/deliveries`, `POST /api/webhooks/deliveries/{id}/replay`, `DELETE /api/apikeys/{id}` (foreign id -> 404)
Export completeness, secret-free (**(!) v3** LB-TEN-1)	ADV-03	`POST /api/household/export` (must cover api-keys/webhooks/usage, no hashes/`whsec_…`)
Dissolve/erasure cleanup (**(!) v3** LB-TEN-1)	ADV-04	dissolve/`DELETE /api/auth/me` -> 0 rows in `ApiKey`/`WebhookSubscription`/`WebhookDelivery`/`UsageCounter`
Impersonation write attribution (**(!) v3** LB-ADM-1)	ADV-05	`POST /api/admin/impersonate/{id}` then `PUT /api/household`, `POST /api/notifications/{id}/read` (audit `impersonated_by`)
Impersonation ≠ staff ladder (**(!) v3** ADM-2)	ADV-06	impersonation token vs `POST /api/admin/impersonate/{x}`, `PUT /api/admin/tenants/{id}/subscription`, `DELETE /api/admin/users/{x}/mfa`, `POST /api/admin/announce-all` (-> 403)

Probe (finding)	Test case(s)	Key API endpoint(s) / surface
Impersonation role ceiling (TB-ADM-3)	ADV-07	member impersonation vs `POST /api/household/transfer-ownership`, `PUT /api/household/members/{id}/role` (-> 403)
Pref bleed: impersonation + shared device (**(!) v3** partial, ADM-8/9, LB-UI-4/5)	ADV-08	`GET /api/auth/me` prefs; localStorage theme/locale bootstrap across accounts
Security notice bypasses prefs (**(!) v3** ADM-1)	ADV-09	`PUT /api/notifications/preferences` (both off) + `DELETE /api/admin/users/{id}/mfa` -> `security.mfa_reset` in-app + email
Per-user MFA lockout (**(!) v3** ADM-3)	ADV-10	`POST /api/auth/mfa/verify` (cumulative cap, not just per-IP 429)
Recovery code single-use (TB-AUTH-6)	ADV-11	`POST /api/auth/mfa/verify` (recovery code burned on first use)
Webhook replay/order/same-second idempotency (LB-BILL-1)	ADV-12	`POST /api/billing/webhook` (dup EventId no-op; same-second created->active applies)
No false dunning on first bad webhook (**(!) v3** LB-BILL-4)	ADV-13	`POST /api/billing/webhook` first-ever `past_due`/`canceled` -> no owner notification
Comp/revert a churned sub (**(!) v3** ADM-5)	ADV-14	`PUT|DELETE /api/admin/tenants/{id}/subscription` (canceled sub should be comp-able, not stuck 409)
Concurrent last-seat accept (TB-BILL-19/BILLING-9)	ADV-15	`POST /api/household/invitations/accept` (one 200, one 402 `seat_limit_reached`)
Magic-link/OTP double-redemption (LB-AUTH-3)	ADV-16	`GET /api/auth/magic-link/verify`, `POST /api/auth/otp/verify` (exactly one session)
Rotated refresh-token replay revokes family (SEC-05 promoted)	ADV-17	`POST /api/auth/refresh` (reuse past the 60 s grace window -> 401 + all sessions revoked; within it, successor live -> 200, nothing revoked — once; a third presentation -> 401 + revoke-all)
Locale-reload preserves deep-link (**(!) v3** UX-1/LB-UI-1/2)	ADV-18 (E2E `LocaleMismatchJoinTests`)	`/join?token=…` under a PREFS-1 locale-mismatch reload
No pref revert after soft sign-in (UX-3/4)	ADV-19	OTP soft-nav sign-in reconcile (`theme`/`locale` claims)
"Clear read" spares unread (LB-UI-10)	ADV-20	`DELETE /api/notifications?read=true` vs `?read=false`; omitted scope -> 400 `scope_required`
Security/cache headers on single-origin (**(!) v3** DEP-2/3)	ADV-21	`curl -I /` + `/_framework/*` (HSTS/nosniff/CSP/Referrer-Policy; cache-control)
Forged XFF ≠ rate-limit bypass (DEP-1/ADM-10)	ADV-22	`POST /api/auth/otp/send` w/ rotating `X-Forwarded-For` (trusted-proxy only)
Release native build HTTPS base URL (**(!) v3** NAT-3)	ADV-23	Release AAB/MSIX base URL + Android network-security-config (no cleartext localhost)
Loopback OAuth state binding (NAT-10)	ADV-24	desktop loopback callback rejects mismatched `state`

Per-client coverage: Web = full (all suites). Desktop = DSK-01..15 (auth + per-feature parity). Android = AND-01..15 (auth + per-feature parity incl. hardware back, share sheet, OAuth kill drill). iOS = IOS-01..04 and macCatalyst = MAC-01..03 (first-run smoke — first passed 2026-07-06; needs a Mac). Magic link is **web-only** by design; the §13c checklist is the per-release native subset.

Automated (E2E CI): the following manual cases have an equivalent Playwright/NUnit journey in `tests/E2E.Tests`, run on every push by the `e2e` job (`.github/workflows/ci.yml`) against a booted Postgres + Mailpit + API + Web stack — so they are continuously regression-guarded on **Web**:

Manual case	E2E test
QA-SMK-01 (OTP sign-in happy path)	`AuthFlowTests.Otp_SignIn_LandsInTheApp` (+ `Login_Page_Renders`)
QA-SMK-03 (sign out)	`AuthFlowTests.SignOut_ReturnsToLogin`
QA-AUTH-09 (email-format validation)	`AuthFlowTests.Invalid_Email_IsRejected_NoCodeStep`
QA-MFA-01 (enable TOTP)	`MfaJourneyTests.Enroll_ThenStepUp_OnNextSignIn` (enroll leg)
QA-MFA-02 (step-up enforced at sign-in)	`MfaJourneyTests.Enroll_ThenStepUp_OnNextSignIn` + `StepUp_WithWrongCode_DoesNotSignIn`
QA-I18N-01 (switch language on login)	`I18nTests.Switching_Language_ReRendersTheUi`
QA-I18N-02 (language follows the user across browsers)	`I18nTests.LocaleChoice_FollowsTheUser_AcrossBrowsers`

Manual case	E2E test
QA-SET-08 (dark mode applies/persists/follows, incl. Auto propagation)	`ThemeJourneyTests.ThemeChoice_AppliesLive_PersistsLocally_AndFollowsTheUser`
QA-INV-10 (accept refused after a downgrade — seat re-check)	`SeatQuotaJourneyTests.Accepting_An_Invite_After_A_Downgrade_Shows_The_HouseholdFull_State`
QA-AUTH-01/05 (magic link happy + single-use)	`MagicLinkJourneyTests`
QA-HH-02/03/05/07/08 (rename, invite/join, leave, dissolve, delete account)	`MembershipLifecycleTests`
QA-HH-09..12 (roster: promote/demote/remove/owner-protection)	`RosterJourneyTests`
QA-HH-13 (data export download)	`GdprExportJourneyTests`
QA-INV-01/02/08 (invite create/accept/regenerate)	`MembershipLifecycleTests` (+ invalid-code inline error)
QA-NOTIF-01..03 (bell list/unread/mark-read via announcements)	`NotificationJourneyTests` + `AnnouncementJourneyTests`
QA-ADMIN-01/04 (staff console gate, announce)	`AnnouncementJourneyTests`
QA-BILL-01/02 (billing page + fake-provider upgrade loop)	`BillingJourneyTests`
QA-ADV-18 (locale-mismatch × invite acceptance)	`LocaleMismatchJoinTests`
QA-DSK-01 (desktop boot)	the `native-smoke-windows` job boots the REAL Windows exe as a process-alive + provider-probe canary (WebView2 150 strips CDP under elevation — PRs #170/#171); the OTP journey is CI-driven on Android only. `NativeSmokeTests` remains for local non-elevated runs
QA-AND-01 (Android OTP sign-in)	`tests/native-smoke-android/smoke.js` — the `native-smoke-android` job boots a real emulator and drives the app via playwright-core's `_android` module

The native smoke jobs run from *Run workflow* -> `devices` (ADR-031) — they are boot-and-sign-in canaries, not the per-feature native regression, which stays manual (§12–13b). All other cases remain manual-only or API-test-backed as noted per row.

16. Sign-off sheet

Record one row per executed case. Build = API/web commit SHA (`git rev-parse --short HEAD`).

Case ID	Client	Result (P/F/Blocked/N-A)	Tester	Build (SHA)	Date	Notes / defect link
QA-SMK-01	Web					
QA-SMK-02	Web					
QA-SMK-03	Web					
QA-SMK-04	Web					
QA-SMK-05	Desktop					
QA-SMK-06	Android					
QA-SMK-07	Platform					
QA-AUTH-01	Web					
QA-AUTH-02	Web					
QA-AUTH-03	Web					
QA-AUTH-04	Web					
QA-AUTH-11	Web					
QA-AUTH-05	Web					
QA-AUTH-06	Web					
QA-AUTH-07	Web					
QA-AUTH-08	Web					

Case ID	Client	Result (P/F/Blocked/N-A)	Tester	Build (SHA)	Date	Notes / defect link
QA-AUTH-09	Web					
QA-AUTH-10	Web					
QA-ONB-01	Web					
QA-ONB-02	Web					
QA-HH-01	Web					
QA-HH-02	Web					
QA-HH-03	Web					
QA-HH-04	Web					
QA-HH-05	Web					
QA-HH-06	Web					
QA-HH-07	Web					
QA-HH-08	Web					
QA-HH-09	Web					
QA-HH-10	Web					
QA-HH-11	Web					
QA-HH-12	Web					
QA-HH-13	Web					
QA-HH-14	Web					
QA-INV-01	Web					
QA-INV-02	Web					
QA-INV-03	Web					
QA-INV-04	Web					
QA-INV-05	Web					
QA-INV-06	Web					
QA-INV-07	Web					
QA-INV-08	Web					
QA-INV-09	Web					
QA-INV-10	Web					
QA-SET-01	Web					
QA-SET-02	Web					
QA-SET-03	Web					
QA-SET-04	Web					
QA-SET-05	Web					
QA-SET-06	Web					
QA-SET-07	Web					
QA-SET-08	Web					
QA-MFA-01	Web					
QA-MFA-02	Web					
QA-MFA-03	Web					
QA-MFA-04	Web					
QA-MFA-05	Desktop/Android					
QA-NOTIF-01	Web					
QA-NOTIF-02	Web					
QA-NOTIF-04	Web					
QA-NOTIF-03	Web					

Case ID	Client	Result (P/F/Blocked/N-A)	Tester	Build (SHA)	Date	Notes / defect link
QA-I18N-01	Web					
QA-I18N-02	Web					
QA-I18N-03	Web					
QA-I18N-04	Web					
QA-ADMIN-01	Web					
QA-ADMIN-02	Web					
QA-ADMIN-03	Web					
QA-ADMIN-04	Web					
QA-ADMIN-05	Web					
QA-ADMIN-06	Web					
QA-ADMIN-07	Web					
QA-BILL-01	Web					
QA-BILL-02	Web					
QA-MAIL-01	Web					
QA-MAIL-02	Web					
QA-MAIL-03	Web					
QA-MAIL-04	Web					
QA-DSK-01	Desktop					
QA-DSK-02	Desktop					
QA-DSK-03	Desktop					
QA-DSK-04	Desktop					
QA-DSK-05	Desktop					
QA-DSK-06	Desktop					
QA-DSK-07	Desktop					
QA-DSK-08	Desktop					
QA-DSK-09	Desktop					
QA-DSK-10	Desktop					
QA-DSK-11	Desktop					
QA-DSK-12	Desktop					
QA-DSK-13	Desktop					
QA-DSK-14	Desktop					
QA-DSK-15	Desktop					
QA-AND-01	Android					
QA-AND-02	Android					
QA-AND-03	Android					
QA-AND-04	Android					
QA-AND-05	Android					
QA-AND-06	Android					
QA-AND-07	Android					
QA-AND-08	Android					
QA-AND-09	Android					
QA-AND-10	Android					
QA-AND-11	Android					
QA-AND-12	Android					
QA-AND-13	Android					

Case ID	Client	Result (P/F/Blocked/N-A)	Tester	Build (SHA)	Date	Notes / defect link
QA-AND-14	Android					
QA-AND-15	Android					
QA-IOS-01	iOS					
QA-IOS-02	iOS					
QA-IOS-03	iOS					
QA-IOS-04	iOS					
QA-MAC-01	macCatalyst					
QA-MAC-02	macCatalyst					
QA-MAC-03	macCatalyst					
QA-SEC-01	Web					
QA-SEC-02	Web					
QA-SEC-03	Web					
QA-SEC-04	Desktop/Android					
QA-SEC-05	Web					
QA-API-01	curl					
QA-API-02	curl					
QA-API-03	curl/Postman					
QA-API-04	curl					
QA-API-05	curl					
QA-API-06	curl					
QA-GATE-01	Web + curl					
QA-GATE-02	Web					
QA-GATE-03	Web					
QA-GATE-04	Web					
QA-GATE-05	Web					
QA-GATE-06	Web					

§10d–§10i — JiggerJot's own flows. All rows **Not-run** (blank) until executed. Run **§10d first**: the rest of the app has nothing to work with until a shelf exists, so a failure there will cascade.

Case ID	Client	Result (P/F/Blocked/N-A)	Tester	Build (SHA)	Date	Notes / defect link
QA-MAIL-05	Web					
QA-MAIL-06	Web					|Needs a Spanish reader
QA-MAIL-07	Web					|Images blocked
QA-SHELF-01	Web					
QA-SHELF-02	Web					
QA-SHELF-03	Web					
QA-SHELF-04	Web					Needs the API stopped mid-case
QA-SHELF-05	Web					
QA-SHELF-06	Web					
QA-SHELF-07	Web					

Case ID	Client	Result (P/F/Blocked/N-A)	Tester	Build (SHA)	Date	Notes / defect link
QA-SHELF-08	Web					
QA-SHELF-09	Web					
QA-SHELF-10	Web					
QA-SHELF-11	Web					
QA-SHELF-12	Web					Screen reader
QA-SHELF-13	Web					
QA-SHELF-14	Web					Needs a cocktail using the bottle (QA-MINE-01)
QA-CAT-01	Web					
QA-CAT-02	Web					
QA-CAT-03	Web					
QA-CAT-04	Web					
QA-CAT-05	Web					
QA-CAT-06	Web					
QA-CAT-07	Web					
QA-CAT-08	Web					Needs two households
QA-CAT-09	Web					
QA-CAT-10	Web					
QA-MAKE-01	Web					
QA-MAKE-02	Web					
QA-MAKE-03	Web					
QA-MAKE-04	Web					
QA-MAKE-05	Web					
QA-MAKE-06	Web					
QA-MAKE-07	Web					
QA-MAKE-08	Web					
QA-MAKE-09	Web					
QA-MAKE-10	Web					note the catalog size if it is not the curated 644
QA-MINE-01	Web					
QA-MINE-02	Web					
QA-MINE-03	Web					
QA-MINE-04	Web					
QA-MINE-05	Web					
QA-MINE-06	Web					
QA-MINE-07	Web					
QA-MINE-08	Web					Keyboard only, and a screen reader
QA-MINE-09	Web					A Metric member for step 3; two browsers for step 7
QA-MINE-10	Web					A phone width for step 2
QA-MINE-11	Web					Both languages for step 2
QA-START-01	Web					Needs a brand-new household

Case ID	Client	Result (P/F/Blocked/N-A)	Tester	Build (SHA)	Date	Notes / defect link
QA-START-02	Web					
QA-START-03	Web					
QA-START-04	Web					
QA-START-05	Web					
QA-START-06	Web					Needs a second brand-new household
QA-START-07	Web					Needs an invitation into a filled household
QA-CHROME-01	Web					
QA-CHROME-02	Web					Needs the API stopped mid-case
QA-CHROME-03	Web					Screen reader
QA-CHROME-04	Web					
QA-CHROME-05	Web					Screen reader
QA-CHROME-10	Web					
QA-CHROME-11	Web					
QA-CHROME-12	Web					
QA-CHROME-13	Web					Needs the API stopped mid-case
QA-CHROME-06	Web					
QA-CHROME-07	Web/Desktop/Android					
QA-CHROME-08	Web					OS reduce-motion setting
QA-CHROME-09	Web					Needs a Spanish reader
QA-CHROME-14	Web					Browser network panel, both themes
QA-CHROME-15	Web					Both themes, 1280 and 390 wide
QA-CHROME-16	Web					1280 and 390 wide; a throttled reload
QA-CHROME-17	Web					A shelf one bottle away from something
QA-CHROME-18	Web					Keyboard + screen reader for the radio group
QA-CHROME-19	Web					Print preview; 1280 and 390 wide
QA-CHROME-20	Web					Both themes; 390×812; a fresh household for the wizard
QA-CHROME-21	Web					1280 and 390 wide

Case ID	Client	Result (P/F/Blocked/N-A)	Tester	Build (SHA)	Date	Notes / defect link
QA-CHROME-22	Web					A linked provider; 1280 and 390 wide
QA-CHROME-23	Web					Owner and member accounts; 1280 and 390 wide
QA-CHROME-24	Web					Billing on; a staff account; a provider sign-in
QA-CHROME-25	Web					Both themes; 1440, 768 and 390 wide; OS reduce-motion on
QA-CHROME-26	Web					A stocked household and an empty one; both languages
QA-CHROME-27	Web					A stocked household and an empty one; keyboard and screen reader; both languages

§14a adversarial / tenant-isolation (QA-ADV-*). **All rows are Not-run **(blank) until executed. The formerly pre-seeded** Blocked (known defect) **rows were reset when the v3 remediation completed (2026-07, PRs #147–#191) — their Notes record which defect each was blocked on, and every one of those cases now expects** Pass** on re-run.

Case ID	Client	Result (P/F/Blocked/N-A)	Tester	Build (SHA)	Date	Notes / defect link
QA-ADV-01	API					
QA-ADV-02	API					
QA-ADV-03	API					Was pre-seeded Blocked (LB-TEN-1 (export incomplete)) — fixed in v3; re-run
QA-ADV-04	API					Was pre-seeded Blocked (LB-TEN-1 (orphaned rows on dissolve)) — fixed in v3; re-run
QA-ADV-05	API					Was pre-seeded Blocked (LB-ADM-1 (impersonation write attribution)) — fixed in v3; re-run
QA-ADV-06	API					Was pre-seeded Blocked (ADM-2 (staff gate re-escalation)) — fixed in v3; re-run
QA-ADV-07	API					
QA-ADV-08	API/Web					Was pre-seeded Blocked (LB-UI-4/5 shared-device pref bleed) — fixed in v3 (ADM-9 sign-out clear); re-run both legs
QA-ADV-09	API					Was pre-seeded Blocked (ADM-1 (security notice suppressible by prefs)) — fixed in v3; re-run
QA-ADV-10	API					Was pre-seeded Blocked (ADM-3 (no per-user MFA lockout)) — fixed in v3; re-run
QA-ADV-11	Web					
QA-ADV-12	API					
QA-ADV-13	API					Was pre-seeded Blocked (LB-BILL-4 (false dunning on first bad webhook)) — fixed in v3; re-run
QA-ADV-14	API					Was pre-seeded Blocked (ADM-5 (comp stuck 409 on churned sub)) — fixed in v3; re-run
QA-ADV-15	Web					
QA-ADV-16	Web					

Case ID	Client	Result (P/F/Blocked/N-A)	Tester	Build (SHA)	Date	Notes / defect link
QA-ADV-17	API					
QA-ADV-18	Web					Was pre-seeded Blocked (UX-1/LB-UI-1/2 (locale reload drops /join deep-link)) — fixed in v3; re-run
QA-ADV-19	Web					
QA-ADV-20	Web					
QA-ADV-21	API					Was pre-seeded Blocked (DEP-2/3 (no security/cache headers)) — fixed in v3; re-run
QA-ADV-22	API					
QA-ADV-23	Desktop/Android					Was pre-seeded Blocked (NAT-3 (Release build cleartext localhost base URL)) — fixed in v3; re-run
QA-ADV-24	Desktop					
QA-ADV-25	API					
QA-ADV-26	API					
QA-ADV-27	API					
QA-ADV-28	API					
QA-ADV-29	Web					
QA-ADV-30	API					
QA-ADV-31	API					
QA-ADV-32	API					
QA-ADV-33	Web					
QA-ADV-34	Android/Desktop					
QA-ADV-35	Ops					
QA-DEP-01	Operator					
QA-DEP-02	Operator					
QA-DEP-03	Operator					
QA-DEP-04	Operator					
QA-SET-09	Web + Android					
QA-AND-16	Android					

Release gate (suggested): all • Smoke Smoke + all • Core Core cases Pass on Web; the §13c native checklist Pass on every platform being shipped (iOS/macCatalyst once those ship); no open Critical/High defects. • Edge Edge cases triaged (Pass or accepted-known-issue).

17. Notes for maintainers

- This plan is the manual counterpart to the automated `tests/E2E.Tests` (Playwright/NUnit) suite. That suite now covers the **OTP auth happy-path + guards** (`AuthFlowTests`), the **MFA enroll -> step-up** journey incl. the wrong-code negative (`MfaJourneyTests`), and the **language switch** (`I18nTests`) — all over the page objects in `Pages/`, reading codes from Mailpit via the `Mailpit` REST client, and **run in CI** by the `e2e` job (`.github/workflows/ci.yml`, Web only). See `tests/E2E.Tests/README.md` for the run procedure (it documents the same Mailpit SMTP override noted in §1.1). The cases these journeys mirror are marked **Automated in CI** and listed in §15. The Gherkin blocks here are written to be lifted directly into new E2E scenarios — keep the two in sync as automation grows.
- When you add an app-specific domain feature on top of this platform, add a matching suite here and a row in the traceability matrix (§15) so "entire functionality" stays honest. **That instruction sat here unfollowed through nineteen merged app slices** — this document covered the platform only while every flow in `FEATURES.md` was built on top of it, and its scope section went on saying the app had no features. The app suites are §10d–§10i; the lesson is that the reminder is not enough on its own, so the slice ritual in `docs/PLAN.md` now carries the step.
- **Each app story file carries Gherkin that maps onto these cases almost directly.** When a slice changes behaviour, its story is the place the new acceptance criteria are written first; copying them here is a translation rather than an invention.
- `docs/FEATURES.md` describes the same JWT-based flows at the design level; this plan is their step-by-step verification. Keep the two in sync when behavior changes.
- **Updated 2026-06-22** for the security/quality remediation: OTP lockout is now **cumulative per email** and resend-proof (QA-AUTH-04); passwordless send/verify are **rate-limited** (QA-AUTH-11); the Settings **Unlink** now requires a fail-closed confirmation (QA-SET-04); and server-side hardening (write-stamping, refresh-reuse detection, fail-closed takeover guard) is captured as automated coverage (QA-SEC-05).
- **Updated 2026-06-23:** QA-AUTH-02 now documents the **tenant-gated same-email auto-link** rule (Microsoft trusted only on the `consumers` tenant; work/school + unknown providers fail closed — audit MITI-3); and QA-SEC-05 adds the **legacy refresh-cookie self-heal** as automated coverage (`CookieServiceTests`).
- **Updated 2026-06-26** for the platform-foundation work (JOBS/BILLING/OBS, ADR-006/007/008):
- **Transactional email is now delivered asynchronously** via the outbox dispatcher — a few-second delay, and the send request always succeeds (reliability moved to the background). Affects every email-based case; see the §1.1 and §11 notes. The E2E `AuthFlowTests` (which reads OTP codes from Mailpit) now traverses this async path — confirm it waits/polls for the email rather than assuming instant delivery.
- Added **API health/readiness** endpoints + smoke case **QA-SMK-07** (`/health`, `/health/ready`).
- The **billing API** (checkout/portal/webhook), the **append-only audit log**, and **OpenTelemetry** telemetry are API-/operational-level with **no client UI** — covered by `tests/Api.Tests`, E2E pending; manual cases will follow when UI exists (see §2 + §15).
- **Updated 2026-06-30** for RBAC (ADR-009): the tenant role model gains an **`admin` **tier behind a permission seam (owner > admin > member). RBAC-1 added the seam (no behavior change); RBAC-2 added the** owner-only role-change endpoint** (`PUT /api/household/members/{id}/role`, admin<->member; owner is conferred only via transfer) and hardened member-removal so the **owner can't be removed**. This was API-only at first; **RBAC-3 added the web UI** — the Household roster now has owner-only **Make admin / Make member** controls, and the page is admin-aware (admin can rename + invite/remove members, but sees no role/transfer/dissolve controls). New web cases **QA-HH-09..12**; §7 intro updated for the three-tier model. API coverage stays automated (`tests/Api.Tests`).
- **Updated 2026-06-30** for file storage (ADR-010): an `IFileStorage` seam (tenant-scoped keys, local-disk dev default / S3-compatible prod) with a signed, time-limited download endpoint `GET /api/files/{token}`. **API-only** (no UI consumer yet) — covered by `tests/Api.Tests` (`LocalDiskFileStorageTests`, `FileDownloadTokenizerTests`, `FilesControllerTests`); see §2 + §15. Manual cases will follow when a feature (avatars/attachments) wires it to UI. FILES-3 added the **S3-compatible** backend (AWS/MinIO/R2/B2, config-gated by `Storage:S3:Bucket`), tested against a real **MinIO** container (`S3FileStorageMinioTests`); prod config is documented in `.env.example`. **This completes Wave 1** (RBAC + File storage).
- **Updated 2026-07-01** for GDPR data export (ADR-011, Wave 2): owner-only `POST /api/household/export` assembles the tenant's data (core + each feature's contributor section) into a JSON bundle stored via file storage and returns a **signed download URL**; **secret-free** (no token hashes), tenant-scoped, audited. **API-only** (no UI button yet) — covered by `Api.Tests` (`TenantExportTests`); see §2 + §15.

- **Updated 2026-07-01** for GDPR account erasure (ADR-011): `DELETE /api/auth/me` ("delete my account") wipes the caller's identity/PII (`User`/`UserLogin`/`RefreshToken`/`LoginToken`) in one transaction, single-owner-safe (owner-with-members -> transfer first; solo owner -> confirm to dissolve; member removed, not re-homed), audited (the audit trail survives — actor ids, never PII). **API-only** — covered by `Api.Tests` (`AccountErasureTests`). **GDPR epic complete (export + erasure).**
- **Updated 2026-07-01** for MFA enrollment (ADR-012, Wave 2): authenticator-app **TOTP** (Otp.NET) — `/api/auth/mfa/*` (enroll -> provisioning URI/QR; confirm with a code -> enable + one-time recovery codes; disable; status). Secret **encrypted at rest** (Data Protection); recovery codes **hashed + single-use**; MFA rows wiped by account erasure. **API-only** (MFA-1) — covered by `Api.Tests` (`MfaServiceTests`). **MFA-2 (login step-up):** an MFA-on login returns a **signed challenge** instead of a session; `POST /api/auth/mfa/verify` completes it with a TOTP/recovery code. Wired into **OTP-verify + native-exchange**; the OAuth/magic-link **redirect** paths route to a client `/mfa` page (needs the MFA UI) and are a flagged **follow-up** — no platform-web user can enable MFA via those paths today (no enrollment UI). Covered by `MfaChallengeServiceTests` + `MfaLoginServiceTests`. **This completes Wave 2** (GDPR + MFA).
- **Updated 2026-07-01** for in-app notifications (ADR-013, Wave 3): a **per-user** notification center — `GET /api/notifications` (paginated), `/unread-count`, `POST /{id}/read`, `/read-all` — scoped to the caller (never cross-user); features produce via `NotifyAsync` (staged in-app row). Notifications are user PII (wiped by account erasure). **API-only** — covered by `Api.Tests` (`NotificationServiceTests`). **NOTIFY-2:** per-user **delivery preferences** (`GET|PUT /api/notifications/preferences`, default both on) + `NotifyAsync` **fan-out** — in-app row + email via the outbox-backed `IEmailSender`, gated by prefs. Covered by `NotificationFanOutTests`. Bell-menu UI is an API-first follow-up. **NOTIFY epic + Wave 3's first item complete.**
- **Updated 2026-07-01** for the platform-staff admin surface (ADR-014, Wave 3): `GET /api/admin/tenants` (+ `/{id}`) — **staff-only** (config allowlist `Admin:StaffEmails`, out-of-band; non-staff -> 403). Read-only cross-tenant inspection; the per-tenant detail **enters the target tenant** (the global filter is never loosened) and is **audited in that tenant**. **API-only** — covered by `Api.Tests` (`PlatformStaffServiceTests`, `AdminControllerTests`).
- **Updated 2026-07-01** for admin **impersonation** (ADR-014, ADMIN-2): `POST /api/admin/impersonate/{userId}` (staff-only) returns a **short-lived (15-min), non-refreshable** access token carrying the target's identity + an `impersonated_by` claim; **loudly audited in the target's tenant**. Unknown target -> 404. Covered by `AdminControllerTests`. **This completes the ADMIN epic — and the planned platform** (nine epics, ADRs 006–014).
- **Updated 2026-07-01** — UI pass (putting a face on the API-first surfaces). **UI-1 (GDPR):** the owner **Data -> Download household data** button on Household (**QA-HH-13**) and **Settings -> Danger zone -> Delete my account** (**QA-SET-07**) — wired to `POST /api/household/export` and `DELETE /api/auth/me` (single-owner-safe, with the dissolve second-confirm). EN/ES localized.
- **Updated 2026-07-01** — **UI-2 (MFA):** a **Two-factor authentication** card in Settings (`MfaCard` component) — enroll (`POST /api/auth/mfa/enroll`) renders a **client-side QR** of the `otpauth://` URI (vendored `qrcode-generator`, MIT, in `Shared.Ui/wwwroot/js/`; the secret never leaves the browser) alongside a manual key, confirm (`/confirm`) reveals one-time **recovery codes**, and **disable** (`/disable`) needs a live code. The **Login** page now handles the OTP-verify `mfa_required` response with a **step-up code prompt** -> `POST /api/auth/mfa/verify` (TOTP or recovery code) -> `/auth-callback`. **QA-MFA-01..03**; EN/ES localized. Native OTP + OAuth/magic-link step-up remain follow-ups (web-first).
- **Updated 2026-07-01** — **UI-3 (Notifications):** a **header bell** (`NotificationBell` component) — unread-count badge (polled ~60s), a dropdown list (newest-first, unread dot + relative time), click-to-mark-read and **Mark all read** via `GET /api/notifications[/unread-count]`, `POST /{id}/read`, `/read-all`. A **Notifications** card in Settings (`NotificationPrefsCard`) with **In-app**/**Email** switches (optimistic save, reverts on error) via `GET|PUT /api/notifications/preferences`. **QA-NOTIF-01..03**; EN/ES localized. (No built-in producer — items appear once a feature calls `NotifyAsync`.)
- **Updated 2026-07-01** — **UI-4 (Admin console):** a staff-only `/admin` page (`AdminConsole`) — tenant list + detail (members/subscription/audit count) and **Sign in as** (impersonation). Staff detection uses a new **non-gating** probe `GET /api/admin/me` (`{is_staff}` for any authenticated caller — the only API addition in the UI pass; the allowlist stays config-only, actions still 403 for non-staff), so the header shows an **Admin** link only to staff. Impersonation swaps the in-memory session to the short-lived token (`AuthService.BeginImpersonation`), pins an **impersonation banner** in `MainLayout` (reads the `impersonated_by` claim), and **Stop impersonating** restores the staff identity from the refresh cookie; a reload also reverts (the token is non-refreshable). **QA-ADMIN-01..03**; EN/ES localized. **This completes the UI pass (UI-1..4).**

- **Updated 2026-07-01** — **MFA-3 (redirect step-up, security fix):** OAuth callback and magic-link verify now route through `IMfaLoginService.CompleteOrChallengeAsync` (like the OTP path) instead of issuing a session directly — closing the gap where an MFA-enabled user could sign in via Google/ magic link and **skip the second factor**. When MFA is on, the server redirects to `/login?mfa=<challenge>` (a signed, single-use, 5-min Data-Protection token — no secret), and `Login.razor` reuses the UI-2 step-up prompt -> `POST /api/auth/mfa/verify` -> `/auth-callback`. Property covered by `MfaLoginServiceTests` (MFA-on -> challenge, never a session); **QA-MFA-04**. The dead `IssueRefreshCookieAsync` helper was removed.
- **Updated 2026-07-01** — **MFA-4 (native step-up):** the MAUI client now handles the `{mfa_required, challenge}` response on the native **OTP** and **OAuth-exchange** paths. `AuthService.VerifyOtpAsync`/`SignInWithOAuthAsync` now return a `SignInResult` (Success / Failed / MfaRequired+challenge) and a new `VerifyMfaAsync` completes the step-up (tokens in the body, native transport); `Login.razor`'s native branches reuse the same code prompt. Client-only — no API change; build-verified (native is E2E/manual per `MOBILE_TESTING.md`); **QA-MFA-05**. **MFA is now enforced on every sign-in path, web and native — the epic is fully closed, no open gaps.**
- **Updated 2026-07-01** — **BILLING-5 (quotas):** `IQuotaService` adds plan **seat** limits (members + pending invites vs `Plan.SeatLimit`, enforced on the invite path -> **402 `seat_limit_reached`, with an upgrade message in the Household invite UI) and** metered usage** (`TryConsumeAsync` against a monthly, self-resetting `UsageCounter`). Limits are `PlanCatalog` data — `null`/absent = unlimited, so it's inert until set (platform shipped example caps Free 3/3, Pro 10/100; Free seats were raised to 5 by GATES-1). New entity + migration `AddUsageCounter`. Covered by `QuotaServiceTests` (10 cases); **QA-HH-14**; EN/ES. Only **BILLING-6** (trial/dunning) and a billing-dissolve contributor remain from the BILLING epic.
- **Updated 2026-07-01** — **BILLING-6 (trial/dunning):** the owner-facing reaction to the subscription lifecycle. `IBillingNotifier` notifies the tenant **owner** via the notification center (in-app bell + outbox email). The **webhook** notifies once on a transition into `past_due`/`canceled` (no spam; inbox dedups). A **`SubscriptionLapseSweepJob`** (6h) nudges once when a paid period lapses without a webhook, recording `Subscription.LapseNotifiedAt` (migration `AddSubscriptionLapseNotifiedAt`) — Stripe stays source of truth, no fabricated status. Covered by `BillingWebhookHandlerTests` + `SubscriptionLapseSweepJobTests`; manual via `stripe trigger`. **The BILLING epic is now complete** (1–6); only optional follow-ups remain (advance trial nudge; billing-dissolve contributor).
- **Updated 2026-07-01** — **PUBAPI-1 (public API + API keys, config-gated OFF):** a programmatic surface for machines (the user reversed the earlier "no public API" stance). `ApiKey` (hash-only, `pk_…` revealed once; migration `AddApiKey`); a second **API-key auth scheme** mints a `tenant_id`-scoped principal (so tenant isolation applies for free); owner-only `/api/apikeys` management (`Permission.ManageApiKeys`); demo `/api/public/whoami` (read) + `/echo` (write) gated by `.RequireApiScope`. **`PublicApi:Enabled` (default false) — strong gating: off => the scheme isn't added and the routes 404.** Covered by `ApiKeyServiceTests` (9); boot-verified on (401 without a key) and off (404). HOOKS (outbound webhooks) is the companion outbound half — next.
- **Updated 2026-07-01** — **HOOKS-1 (outbound webhooks, config-gated OFF):** the outbound integration half. `WebhookSubscription` (url + event types + **encrypted** signing secret, revealed once; migration `AddWebhookSubscription`); `IWebhookPublisher.PublishAsync` fans out to matching active subs -> one `"webhook"` **outbox** message each (durable, retried, atomic with the change); `WebhookOutboxHandler` signs (HMAC-SHA256, `X-Webhook-Signature`) + POSTs, throwing on non-2xx so the outbox retries/dead-letters. Owner-only `/api/webhooks` (`Permission.ManageWebhooks`) with a synchronous **send test**. **`Webhooks:Enabled` (default false) — off => routes 404.** Covered by `WebhookSignatureTests` (3) + `WebhookSubscriptionServiceTests` + `WebhookDeliveryTests` (11 in `Webhooks/`); boot-verified on (401) and off (404). **This completes the integration story (PUBAPI inbound + HOOKS outbound), both default-off.**
- **Updated 2026-07-01** — **PUBAPI-2 (hardening): per-key rate limiting** (`RateLimiting.PublicApiPolicy` partitions on the key id — 60/min, one key can't exhaust another's budget -> 429; `RateLimitingTests` proves per-key isolation) and a **leak-free public OpenAPI doc** (`GET /api/public/openapi.json`, anonymous, emits only the `/api/public` routes so the internal `v1` surface is never exposed; boot-verified it serves in Production when enabled and 404s when off).
- **Updated 2026-07-01** — **HOOKS-2 (delivery log + replay):** a tenant-facing debug trail. `WebhookDelivery` records **one row per delivery attempt** (event, success, status/error, and the sent body; not `ITenantScoped` — written from both the tenant-less outbox dispatcher and the request-scoped send-test, read side filters by `TenantId`; migration `AddWebhookDelivery`). Owner routes `GET /api/webhooks/{id}/deliveries` + `POST /api/webhooks/deliveries/{id}/replay` (re-enqueue the exact stored payload, same event id). Covered by `WebhookDeliveryLogTests`. **Candidate HOOKS-3 (not built):** a Blazor management UI for webhooks/API keys.

- **Updated 2026-08-28** — **HOOKS-2 fix (send-test now records a delivery):** the synchronous `POST /api/webhooks/{id}/test` delivered inline but wrote **no** `WebhookDelivery` row, and the sample app fires no published events — so `GET /api/webhooks/{id}/deliveries` was always empty and replay had nothing to replay (QA-API-06 unrunnable). The send-test now records one row per attempt (success **and** failure) via `IWebhookSubscriptionService.SendTestAsync`, so the log doubles as the in-template debug trail and replay is testable. Response shape unchanged (`{ delivered, status_code }`, or `{ delivered:false, error:"delivery_failed" }` on a transport error — internal detail stays in the row, GAP-3). New `WebhookDeliveryLogTests` (`SendTest_RecordsDelivery_*`).
- **Updated 2026-07-01** — **BILLING-7 (dissolve cleanup):** closed the one real gap — deleting a billing-enabled tenant left the Stripe subscription active (still charging). `BillingDataContributor` now wipes the `Subscription` projection on dissolve and **cancels the provider subscription** via a `"billing.cancel"` outbox message (`BillingCancelOutboxHandler` -> new idempotent `IBillingProvider.CancelSubscriptionAsync`) — out-of-band, not an external call inside the teardown tx. `HasDataAsync`=false so billing never blocks leaving; export gains a secret-free `billing` section. Covered by `BillingDissolveTests` (6). **Closes the BILLING epic (1–7).**
- **Updated 2026-07-01** — added **§14b — manual QA for the API surfaces (PUBAPI + HOOKS)**: curl/Postman cases (QA-API-01..06) for the config gate, minting/using API keys (scopes, tenant-scoping, rate limit), and registering webhooks (send-test, signature verification, delivery log, replay). These surfaces have **no web UI by design** (they're for machines), so this is the human-testable complement to the automated tests — the Postman path is one import of `/api/public/openapi.json` away. Referenced from the §15 PUBAPI/HOOKS rows and the §2 "no client UI" note.
- **Updated 2026-07-02** — **E2E-in-CI (v2 audit B8-5/B8-6):** the Playwright suite is now booted and run on every push (the `e2e` job in `.github/workflows/ci.yml` — Postgres + Mailpit + API + Web) and was expanded beyond the auth happy-path to the **MFA enroll -> step-up** journey (`MfaJourneyTests`, incl. a wrong-code negative) and the **login-page language switch** (`I18nTests`). Six manual cases now have a continuously-run Web equivalent and are marked **Automated in CI**: QA-SMK-01, QA-SMK-03, QA-AUTH-09, QA-MFA-01, QA-MFA-02, QA-I18N-01 (mapping table added under §15; how-to note in §2/§1 "How to use"). These stay in the manual plan for **Desktop/Android** (CI runs Web only) and area-change re-runs; human QA can spot-check them on Web rather than run them in full each cycle. `data-testid` hooks were added to `MfaCard` and `LanguageSwitcher` to keep the selectors stable.
- **Updated 2026-07-03** for NATIVE Wave 2 (ADR-018, `docs/NATIVE_PARITY.md`): the parity audit's six gaps were fixed (join-by-code, culture persistence, export download/share, billing return-refresh, Android hardware back — plus **G7**, iOS/macCatalyst crashing at boot, found while writing this expansion) and this plan grew the per-feature native coverage that verifies them on hardware: **QA-DSK-08..14**, **QA-AND-07..13**, the first-ever **iOS/macCatalyst smoke** (§13b, QA-IOS-01..04 + QA-MAC-01..03), and the **per-release native checklist** (§13c). The web halves of the fixes are already -automated (`Member_Joins_By_Pasting_The_Invite_Code`, `Owner_Downloads_The_Data_Export`); the OS-chrome halves (share sheets, hardware back, real focus-return, safe areas) are exactly what these manual cases exist for.
- **Updated 2026-07-06** — **first Apple run of §13b** (maintainer's MacBook Air M1, iOS 26.5 simulator + Mac Catalyst): **QA-IOS-01 PASS** (boots to login — validates the G7 fix on a real Apple runtime), **QA-IOS-02 PASS**, **QA-IOS-04 PASS** (first-ever exercise of the `ASWebAuthenticationSession` -> `jiggerjot://auth` path; also verified on iPhone 17 Pro Max + iPad Air 11" simulators), **QA-MAC-01 PASS**, **QA-MAC-02 PASS** and the OAuth leg of QA-MAC-03 PASS. Remaining: the core-flows spot-checks (QA-IOS-03, rest of QA-MAC-03 — share sheet, language + restart persistence). The pass surfaced two platform gaps, fixed in the same PR as this entry: (1) macOS trust evaluation fails Brevo's SMTP TLS handshake ("incomplete certificate revocation check") -> new `Email:Smtp:CheckCertificateRevocation` setting, default **on**, dev-box opt-out documented in `.env.example` (`SmtpSettingsTests`); (2) sign-in on ad-hoc-signed Mac Catalyst Debug builds died storing the refresh token — MAUI SecureStorage needs the restricted `keychain-access-groups` entitlement (MissingEntitlement without it, SIGKILL at launch with it) -> store entitlement added to `Entitlements.plist` for signed builds, Debug builds swap to `Entitlements.Debug.plist` (unsandboxed) + a `DebugFileSessionStore` fallback (`MACCATALYST && DEBUG` only); SecureStorage must be re-verified under real signing at the downstream first native release (ADR-024; `NEW_APP_GUIDE.md` Phase 9).
- **Updated 2026-07-09** — first findings + features from the manual QA pass (`bugfix/qa-manual-pass`): (1) **rate-limit split** — passwordless *verify* endpoints got their own per-IP budget (attempt cap + headroom, default 10/min) so the cumulative-lockout 401 (QA-AUTH-04) surfaces before the 429 can mask it, and the login UI now shows a dedicated "Too many requests…" message on 429 (QA-AUTH-11 names the exact copy; §1.1 pacing note updated). (2) **MFA recovery codes fixed** — they were 88-char opaque tokens that overflowed the maxlength-14 inputs (unusable); now short `xxxxx-xxxxx` codes, matched case-/separator-insensitively (QA-MFA-01/03). (3)

Notification delete/clear — per-row trash + "Clear read"/"Clear all" in the bell, backed by new per-user `DELETE /api/notifications` endpoints (new **QA-NOTIF-04**). (4) **Staff announce upgrades** — optional member targeting on the per-tenant announce (checkbox column -> `user_ids[]`) and a platform-wide **Announce to everyone** card backed by an outbox fan-out (`POST /api/admin/announce-all`, 202 queued) (QA-ADMIN-04 extended, new **QA-ADMIN-05**). (5) **Staff plan comp** — `PUT|DELETE /api/admin/tenants/{id}/subscription` + console buttons simulate a completed checkout (Pro) and revert to Free; refused 409 for Stripe-backed subs (new **QA-ADMIN-06**). Suite 117 -> **120** cases.

- **Updated 2026-07-10** — **THEME-1 (dark mode)**: per-user Light/Dark/System theme on Bootstrap's `data-bs-theme` — header + login switcher, pre-paint `theme.js` bootstrap (no flash), device persistence (`localStorage["app_theme"]`), server sync (`PUT /api/auth/theme`, `theme` JWT claim, cold-start reconcile like locale). New **QA-SET-08** (web, `ThemeJourneyTests`) + **QA-DSK-15** / **QA-AND-14** (native restart persistence — also verifies WebView localStorage survives a restart per platform). Suite 120 -> **123** cases.
- **Updated 2026-07-10** — **staff MFA reset (ADR-021 addendum, `feat/admin-mfa-reset`)**: a user who loses both the authenticator and the recovery codes was locked out permanently (self-serve disable demands a valid code; every sign-in path steps up). New staff-gated `DELETE /api/admin/users/{userId}/mfa` wipes `UserMfa` + recovery codes (no code required — out-of-band identity verification first), audited in the target's tenant (`admin.mfa.reset`) with the user notified in-app + email (`security.mfa_reset`); confirm-gated **Reset MFA** button on the admin member row (new **QA-ADMIN-07**). Suite 123 -> **124** cases.
- **Updated 2026-07-14** — **PREFS-1 (ADR-022, per-user preference sync)**: fixed the QA-I18N-02 failure (locale was never persisted server-side — the only switcher lived on the anonymous login page) and the "theme applies only after a manual reload" instability (the reconcile ran only on cold starts, missing OTP/MFA soft-navigation sign-ins). Language + theme now live in a **Settings -> Preferences** card; sign-in reconciles both ways (server value wins; a never-set server value adopts the device choice); a locale mismatch persists + reloads once (WASM satellite assemblies); **"system" is stored verbatim** so Auto propagates across devices. QA-I18N-02 is now automated (`I18nTests.LocaleChoice_FollowsTheUser_AcrossBrowsers`); QA-SET-08 updated (no workaround reload; Auto-propagation leg added). No new manual cases — suite stays **124**.
- **Updated 2026-07-14** — **BILLING-9 (ADR-006 addendum, seat re-check at accept)**: a downgrade (dunning lapse, cancel, or an ADR-021 comp revert) left pending invitations that could each still join — BILLING-5 checked seats only at invitation *creation*, so a Pro->Free tenant could grow past its cap by redeeming stale invites. `AcceptAsync` now refuses when the tenant is already over its limit (402 `seat_limit_reached`; accepts at exactly the cap stay allowed — the joiner consumes the seat their invite reserved) and `/join` shows a "This household is full" state (EN/ES); the token stays pending and self-heals on re-upgrade. New **QA-INV-10** (automated — `SeatQuotaJourneyTests`, fake-provider webhook downgrade). Suite 124 -> **125** cases.
- **Updated 2026-07-14** *(recorded 2026-08-25 — the branch carrying this entry never merged and was recovered during branch housekeeping)* — **FULL MANUAL PASS COMPLETE**: the maintainer finished the entire manual QA process as the plan stood that day — the web suite (the 2026-07-09/10 sessions' findings all fixed and merged), the §12–13 native columns (Android + Windows device pass), and the remaining §13b Apple spot-checks (QA-IOS-03 + rest of QA-MAC-03, on top of the 2026-07-06 first run) — **all 125 cases passing, no open findings**. With epic NATIVE closed the same day (ADR-024: distribution is downstream work), this pass is the platform's verification baseline; future passes follow §13c per release and the full plan on native-glue/toolchain changes. *Rider:* everything added below this entry postdates the baseline — the §14a adversarial cases (12 §16 re-run rows) and QA-AND-15 (NATIVE-12) are the open device items on the current plan.
- **Updated 2026-07-15** — **§14a Adversarial & tenant-isolation (QA-ADV-*)**: **24 new curl/Postman + two-browser-context cases from the** v3 audit**, probing cross-tenant read/write isolation at the** API **layer (RLS-2/4, LB-TEN-1/2), impersonation attribution/confinement (LB-ADM-1, ADM-2/3/5), billing-webhook idempotency + false-dunning (LB-BILL-1/4), session/token lifecycle (LB-AUTH-3, refresh-reuse), preference bleed on shared devices + the locale-reload deep-link break (UX-1, LB-UI-1/2/4/5/10), and deploy/native hardening (DEP-1/2/3, NAT-3/10).** 12 cases are EXPECT-FAIL **(assert not-yet-fixed behaviour): ADV-03, 04, 05, 06, 08 (partial), 09, 10, 13, 14, 18, 21, 23 — each formerly carried a** PENDING v3 REMEDIATION **banner (now ■ v3-landed notes) and their §16 rows are reset for a re-run (never Pass while the finding is open). The other 12 should Pass on current code. §15 gains a QA-ADV traceability block; §16 gains 24 sign-off rows. Suite 125 ->** 149** cases.
- **Updated 2026-07-17** — **NATIVE-12 (OAuth process-death resilience)** merged onto develop: an OS kill mid-consent no longer loses the sign-in — an `IOAuthResumeStore` marker (MAUI Preferences) brackets the browser round-trip, the cold-started callback activity stashes the redirect, and `AuthService.TryCompletePendingOAuthAsync` completes the exchange on startup (MFA handoff + 5-min TTL

guard; interrupted links land on Settings' banner). New **QA-AND-15** (on-device kill test; renumbered from the branch's QA-AND-14 — that slot went to THEME-1's restart test in the interim). Suite 149 -> **150** cases.

- **Updated 2026-09-07** — **Billing: the checkout return addresses exist.** The hosted checkout sends the browser back to `/billing/success` or `/billing/cancel`, and neither route existed — the first real (test-mode) Stripe checkout on a downstream app's staging ended on the 404 page. Both now route to the Billing page with a banner ("Payment received…" / "Checkout cancelled — nothing changed"); after success the page refetches quietly until the webhook has flipped the plan. QA-BILL-02 rewritten around the return; `NotifyBillingTests.Billing_ReturnFromCheckout_*` pin it. `DEPLOYMENT.md` §3 gains the real-Stripe walkthrough (where the price id hides, the three look-alikes, the webhook events, the test cards). Also the lifecycle's good-news notice: `billing.activated` ("Subscription active", plan + renewal date) on a transition into a granting status (first activation, resubscribe; renewals and trial conversion silent) — `BillingWebhookHandlerTests.SubscriptionActivated_*` pin it; the dunning tests now count dunning notices only. The cancelled state reads honestly: "Your paid subscription ended on <date>" instead of "renews", and no portal button (nothing live to manage; Upgrade is the way back) — `NotifyBillingTests.Billing_CancelledSubscription_*`.
- **Updated 2026-09-15** — **PREFS-3 (JJ-041): every amount in ounces or millilitres.** Every volume is stored in ounces on the quarter marks — the Savoy's parts as their share of a 3 oz drink, its glasses as 2 oz (a liqueur glass 1 oz) — and metric reads them at the bar's 30 ml to the ounce. "As written" is gone: two choices, imperial the default. QA-CAT-07 rewritten around the two choices; new **QA-CAT-10** (a pour, never parts or glasses; the metric write form; the migrated staging rows). Suite 230 -> **231** cases.
- **Updated 2026-09-15** — **AUTHORING-3: the ingredient suggests the line's role.** Picking a line's ingredient on the write form fills in its role and required box from the same rule the seeded catalog was built with, now in Core; a choice made by hand is never overwritten. New **QA-MINE-07**. Suite 231 -> **232** cases.
- **Updated 2026-09-15** — **MARGA-7: she pages through the next bottles.** Wherever Marga names a bottle to buy — Home, the shelf, the catalog's one-away panel and both "where to start" lines — ‹ › pages through up to ten, stopping at both ends, with everything that named the bottle following it. New **QA-CHROME-27**. Suite 232 -> **233** cases.
- **Updated 2026-09-15** — **AUTHORING-4: a searchable ingredient picker.** A line's ingredient is a combobox rather than a select of two hundred bottles: type part of a name or a category, pick with the mouse or the keyboard, on-shelf bottles marked. New **QA-MINE-08**. Suite 233 -> **234** cases.
- **Updated 2026-09-15** — **AUTHORING-2: editing a cocktail, including a fork.** An Edit button on a household's own recipes opens the same form, filled in and in the writer's units; the shared catalog stays read-only, a fork keeps "Based on", and the last save wins. New **QA-MINE-09**. Suite 234 -> **235** cases.
- **Updated 2026-09-15** — **AUTHORING-5 and INV-4: deleting what a household made.** A household's own cocktail can be deleted behind a confirmation, with the recipe's actions now one row (Create my own version, Edit, Delete); a bottle it added can be deleted from the shelf, refused while its recipes use it and naming them. The shared catalog has neither. New **QA-MINE-10** and **QA-SHELF-14**. Suite 235 -> **237** cases.
- **Updated 2026-09-16** — **Android on a real emulator.** The app's screens were looked at on a device for the first time: the status bar wore the platform template's green, the top inset was applied twice, bars fixed above the tab bar floated 14px up (also on the web), Cocktails' selected chip sat past the edge, and a theme or language picked in the app came undone at the next reload (a stale token claim). All fixed; **QA-AND-09**, **QA-AND-13** and **QA-AND-14** carry the new checks. Suite stays **237**.
- **Updated 2026-10-02** — **CI on billed minutes (ADR-031).** GitHub is the only forge again (ADR-030). A pull request runs the web gates only; the MAUI builds and native smokes run from *Run workflow* -> `devices`; deploys run from *Run workflow* -> `deploy` behind every web gate. The deploy gate paragraph says GitHub; the Apple-cadence note became the device-legs note.
- **Updated 2026-10-04 (v4 Later D, ported from the platform)** — new **§14d**: eleven adversarial cases **QA-ADV-25..35** (each names the v4 finding and the task that fixed it; all expected to **Pass**), four operator drills **QA-DEP-01..04** for the deploy-on-request pipeline (ADR-031), and two regressions, **QA-SET-09** (a theme save across a reload) and **QA-AND-16** (an API base with a trailing slash). On the platform these were first recorded Blocked while the findings were open; every remediation had landed here before the cases did, so they arrive unseeded. QA-ADV-15 now counts seats relative to the cap (T47). 237 -> 254 cases.
- **Updated 2026-10-09 (JJ-043, milestone *Curated catalog*) — the catalog is curated: 644 recipes, one per drink, every one with a glass; the 31-recipe starter set is retired and the notes that described it are rewritten.** QA-CAT-03 **now checks that every recipe names its glass;** QA-MINE-05 **no longer expects a glassless save; new** QA-MINE-11** — no glass, no cocktail, and Marga says so beside the field. 254 -> 255 cases.